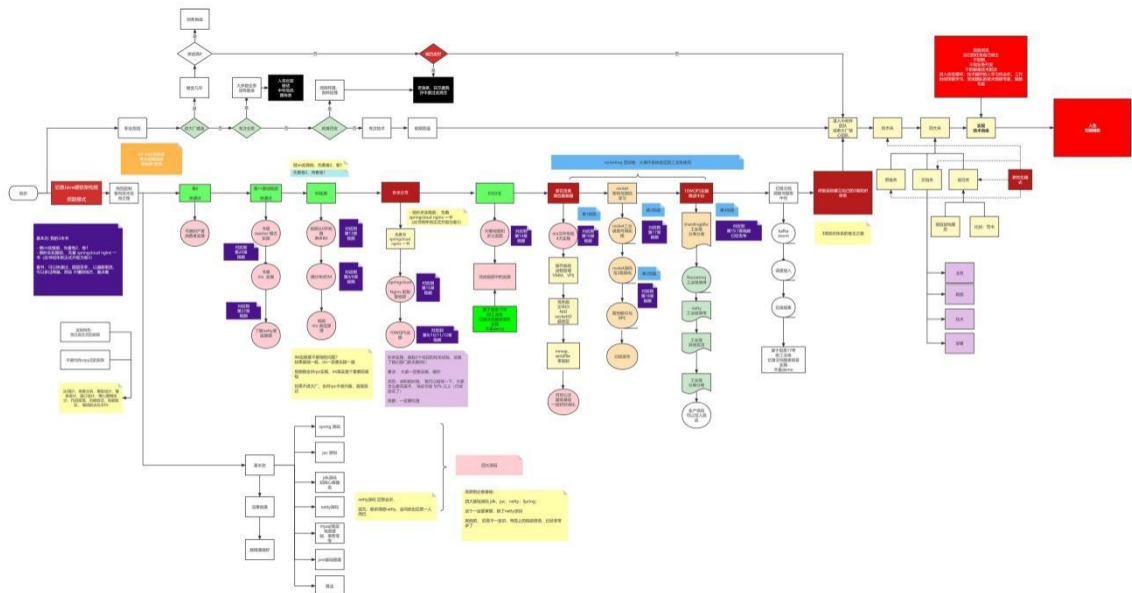


牛逼的职业发展之路

40 岁老架构尼恩用一张图揭秘：Java 工程师的高端职业发展路径，走向食物链顶端的之路

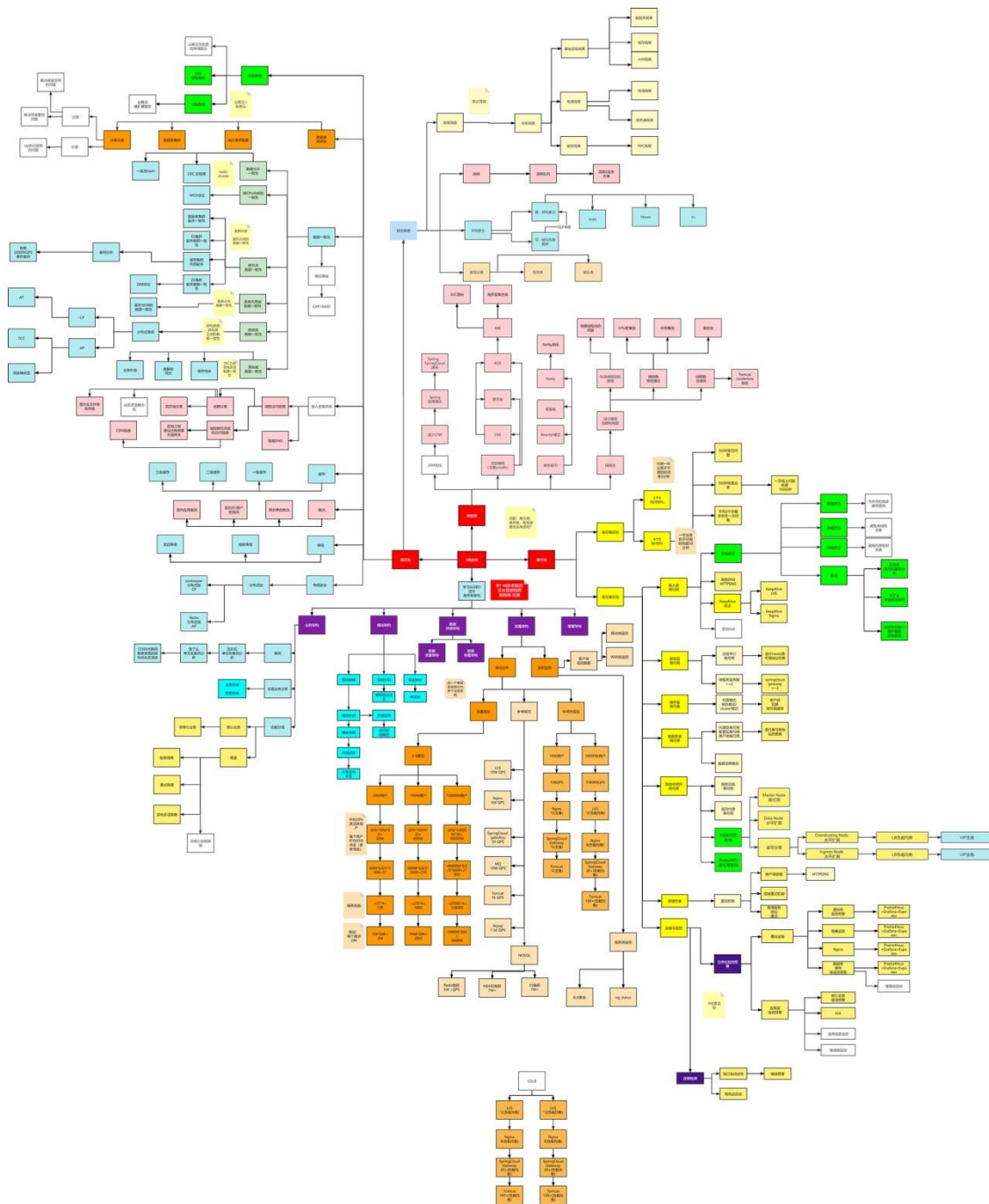
链接：<https://www.processon.com/view/link/618a2b62e0b34d73f7eb3cd7>



史上最全：价值10W的架构师知识图谱

此图梳理于尼恩的多个 3 高生产项目：多个亿级人民币的大型 SAAS 平台和智慧城市项目

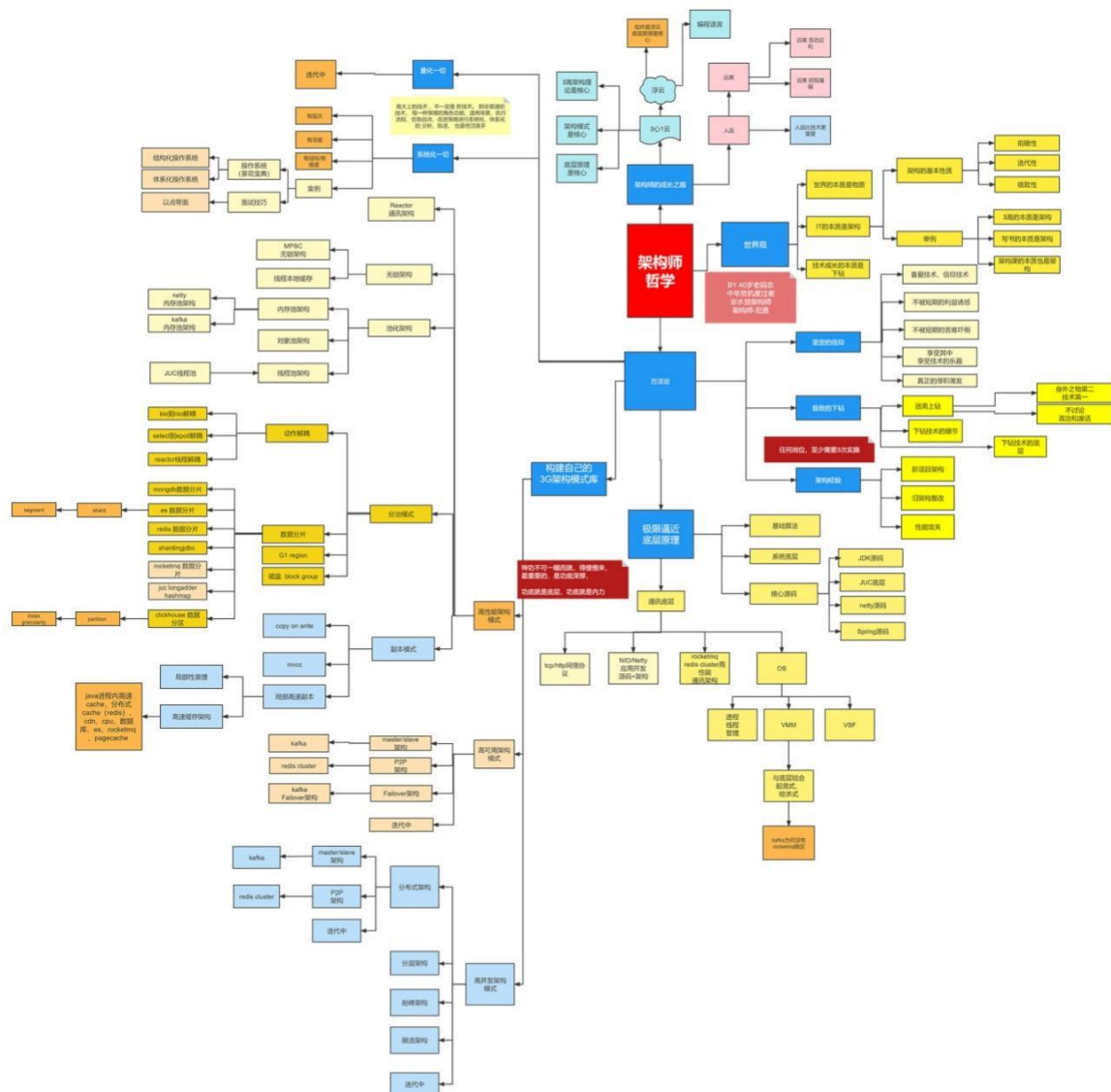
链接：<https://www.processon.com/view/link/60fb9421637689719d246739>



牛逼的架构师哲学

40 岁老架构师尼恩对自己的 20 年的开发、架构经验总结

链接: <https://www.processon.com/view/link/616f801963768961e9d9aec8>



牛逼的3高架构知识宇宙

尼恩 3 高架构知识宇宙，帮助大家穿透 3 高架构，走向技术自由，远离中年危机

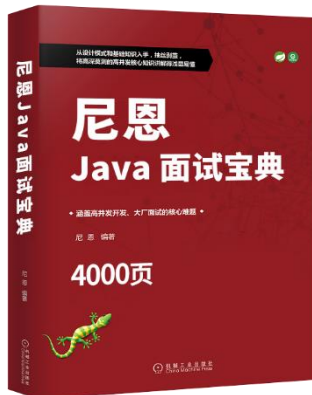
链接: <https://www.processon.com/view/link/635097d2e0b34d40be778ab4>



尼恩Java面试宝典

40 个专题（卷王专供+ 史上最全 + 2023 面试必备）

详情：<https://www.cnblogs.com/crazymakercircle/p/13917138.html>



名称

- ❏ 专题01: JVM面试题 (卷王专供 + 史上最全 + 2022面试必备) -V81-from-尼恩Java面试宝典.pdf
- ❏ 专题02: Java算法面试题 (卷王专供 + 史上最全 + 2022面试必备) -V80-from-Java面试红宝书.pdf
- ❏ 专题03: Java基础面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题04: 架构设计面试题 (卷王专供+ 史上最全 + 2023面试必备) -V86-from-尼恩Java面试宝典.pdf
- ❏ 专题05: Spring面试题_专题06: SpringMVC_专题07: Tomcat面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-尼恩面试宝典-release.pdf
- ❏ 专题08: SpringBoot面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题09: 网络协议面试题 (卷王专供+ 史上最全 + 2023面试必备) -V46-from-尼恩Java面试宝典-release.pdf
- ❏ 专题10: TCP/IP协议 (卷王专供+ 史上最全 + 2022面试必备) -V57-from-Java面试红宝书.pdf
- ❏ 专题11: JUC并发包与容器类 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题12: 设计模式面试题 (卷王专供+ 史上最全 + 2022面试必备) -V84-from-Java面试红宝书.pdf
- ❏ 专题13: 死锁面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题14: Redis 面试题 (卷王专供+ 史上最全 + 2022面试必备) -V65-from-Java面试红宝书.pdf
- ❏ 专题15: 分布式锁 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题16: Zookeeper 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题17: 分布式事务面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题18: 一致性协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题19: Zab协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题20: Paxos 协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题21: raft 协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题22: Linux面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题23: Mysql 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V82-from-尼恩Java面试宝典.pdf
- ❏ 专题24: SpringCloud 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V12-from-Java面试红宝书-release.pdf
- ❏ 专题25: Netty 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题26: 消息队列面试题: RabbitMQ、Kafka、RocketMQ (卷王专供+ 史上最全 + 2023面试必备) -V10-from-Java面试红宝书-release.pdf
- ❏ 专题27: 内存泄漏 内存溢出 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题28: JVM 内存溢出 实战 (卷王专供+ 史上最全 + 2023面试必备) -V17-from-Java面试红宝书-release.pdf
- ❏ 专题29: 多线程面试题 (卷王专供+ 史上最全 + 2023面试必备) -V66-from-Java面试红宝书.pdf
- ❏ 专题30: HR面试题: 过五关斩六将后, 小心阴沟翻船! (史上最全、避坑宝典) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题31: Hash链表面试题 (卷王专供+ 史上最全 + 2022面试必备) -V68-from-Java面试红宝书.pdf
- ❏ 专题32: 大厂面试的基本流程和面试准备 (阿里、腾讯、网易、京东、头条.....) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题33: BST、AVL、RB红黑树、三大核心数据结构 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题34: Elasticsearch面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-Java面试红宝书-release.pdf
- ❏ 专题35: Mybatis面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-尼恩Java面试宝典-release.pdf
- ❏ 专题36: Dubbo面试题 (卷王专供+ 史上最全 + 2023面试必备) -V21-from-尼恩Java面试宝典-release.pdf
- ❏ 专题37: Docker面试题 (卷王专供+ 史上最全 + 2023面试必备) -V47-from-尼恩Java面试宝典.pdf
- ❏ 专题38: K8S面试题 (卷王专供+ 史上最全 + 2023面试必备) -V59-from-尼恩Java面试宝典.pdf
- ❏ 专题39: Nginx面试题 (卷王专供+ 史上最全 + 2023面试必备) -V27-from-尼恩Java面试宝典-release.pdf
- ❏ 专题40: 操作系统面试题 (卷王专供+ 史上最全 + 2023面试必备) -V28-from-尼恩Java面试宝典-release.pdf
- ❏ 专题41: 大厂面试真题 (卷王专供+ 史上最全 + 2023面试必备) -V84-from-尼恩Java面试宝典.pdf

未来职业，如何突围：三栖架构师

未来职业，如何突围？

技术自由圈



——未来超级架构师社区

领路式指导

FSAC 三栖合一架构师

Future Super Architect Community

- 第一栖：Java 架构
- 第二栖：GO 架构
- 第三栖：大数据 架构

尼恩JAVA硬核架构班

会员制

提供技术方向指导，
职业生涯指导，少坑坑，少弯路

简历指导

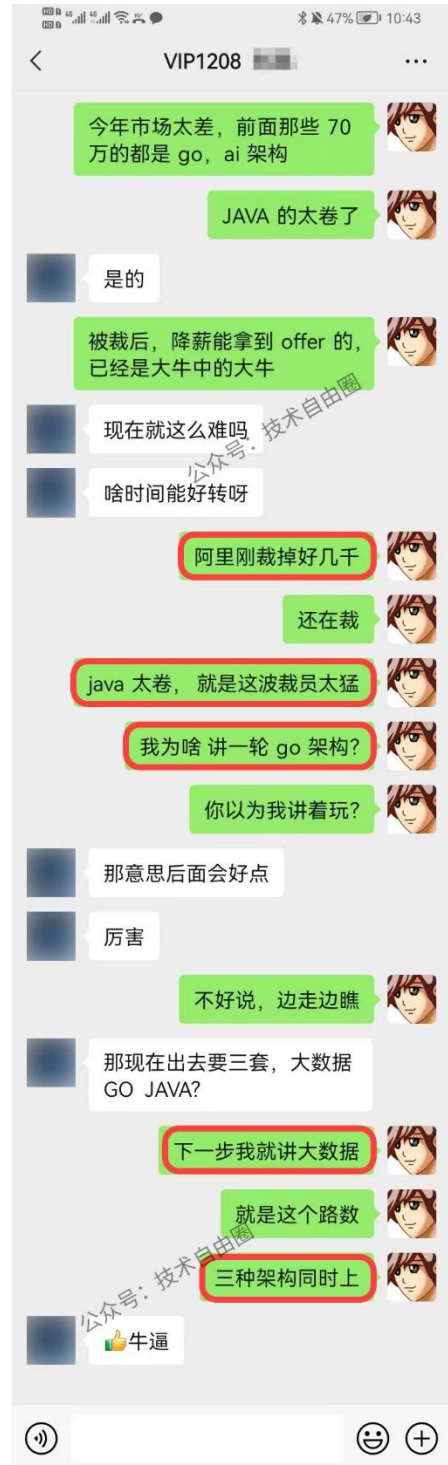
有助成功就业、跳槽大厂
挪窝涨薪必备

实操性

项目都是老架构师
在生产上实操过的项目

非水货

老架构师，不是水货架构师
《Java高并发三部曲》为证



专题24：SpringCloud 面试题（史上最全、定期更新）

本文版本说明：V108

此文的格式，由markdown 通过程序转成而来，由于很多表格，没有来的及调整，出现一个格式问题，尼恩在此给大家道歉啦。

由于社群很多小伙伴，在面试，不断的交流最新的面试难题，所以，《[尼恩Java面试宝典](#)》，后面会不断升级，迭代。

本专题，作为《尼恩Java面试宝典》专题之一，《尼恩Java面试宝典》一共**41个面试专题**，后续还会增加

升级说明：

V108升级说明（2023-09-12）：

微博一面：RPC怎么做零呼损？

V12升级说明：

聊聊：你对微服务的理解？微服务架构和单体架构有何不同？单体架构如何演进的微服务架构？

聊聊：分布式应用AKF拆分原则和扩展原则

聊聊：Feign的工作原理

聊聊：Ribbon的工作原理

聊聊：Hystrix的工作原理

聊聊：gateway的工作原理

聊聊：Nginx 和 Zuul 的区别和共同点

央企真题：Feign Ribbon Hystrix 三者关系 （重点题目）

《尼恩Java面试宝典》升级的规划为：

后续基本上，**每一个月，都会发布一次**，最新版本，可以扫描扫架构师尼恩微信，发送“领取电子书”获取。

尼恩的微信二维码在哪里呢？请参见文末

面试问题交流说明：

如果遇到面试难题，或者职业发展问题，或者中年危机问题，都可以来 疯狂创客圈社群交流，
加入交流群，加尼恩微信即可，
入交流群，加尼恩微信即可， 发送“入群”



微服务理论方面的面试题

1、聊聊：你对微服务的理解？ 微服务架构和单体架构有何不同？ 单体架构如何演进的微服务架构？

微服务架构（Microservice Architecture）是一种架构概念，

微服务架构主要作用是将功能分解到离散的服务当中，从而降低系统的耦合性，并提供更加灵活的服务支持。

相对于单体应用（所谓单体应用是指所有的模块、业务都包含在一个应用中）而言，微服务把各个模块拆分成不同的项目，每个模块都只关注一个特定的业务功能，发布时每一个项目都是一个独立的包，运行在独立的进程上。

它解决了单体应用造成的一些服务升级、服务扩展、性能提升等多个维度的难题。

概念：

把一个大型的单个应用程序和服务拆分为数个甚至数十个的支持微服务，它可扩展单个组件而不是整个的应用程序堆栈，从而满足服务等级协议。增强可用性、服务易扩展、减少开发成本、减少服务发布对整个平台的影响

定义：

围绕业务领域组件来创建应用，这些应用可独立地进行开发、管理和迭代。在分散的组件中使用云架构和平台式部署、管理和服务功能，使产品交付变得更加简单。实现有很多方式，企业转由单个系统转向微服务就要考虑很多问题，比如技术选型、业务拆分问题、高可用、服务通信、服务发现和治理、集群容错、配置管理、数据一致性问题

本质：

用一些功能比较明确、业务比较精练的服务去解决更大、更实际的问题

优点：

- 易于开发，可维护性高：一个服务只会关注一个特定的业务模块，代码比较少，可维护性就高
- 服务之间可以独立部署：发布风险低，发布单个服务不需要重新发布整个应用
- 易于扩展：每个服务可以各自进行负载均衡扩展和数据库扩展，而且每个服务可以根据自己的需要部署到合适的硬件服务器上
- 提高容错性：经过负载的服务有更好的容错率，挂了一台实例不会导致整个系统瘫痪
- 技术栈不受限（异构）：微服务之间通过轻量级的通信机制进行通信，比如RESTful API，因此不同项目可以随意选择合适的技术来实现

缺点：

- 运维要求高：
对于单体应用只要部署一个服务，微服务化后可能需要部署几十几百个服务
- 分布式固有的复杂性：
开发人员需要考虑分布式事务、系统的容错性等，比如服务A的某个接口依赖服务B，通过RESTful API调用服务B的接口但发生了错误，需要提供重试等机制
- 修改接口成本增加：
修改接口本就是一件繁琐的事，微服务化后成本更高，因为各个服务之间只通过轻量级通信机制访问，耦合度比较低，需要排查哪些服务的接口受到了影响，而在单体应用中通常只是方法间的依赖，如果修改了某个方法的签名，那么在编译时就会报错
- 重复劳动：
当一个单体应用微服务化后，不同的服务之间会有重复代码产生，比如 Gradle脚本、Maven依赖都非常类似，使用到的某些函数每个服务都造了一遍等，如果都是用同一种语言实现的还能用共享库解决，如果有多种语言那就难以避免重复劳动了。
还有各种实体类可能会有重复

2、聊聊：微服务和模块划分原则

- 微服务设计四个原则
 - AKF拆分原则
 - 前后端分离

- 无状态服务
- Restful通信风格
- 微服务设计目标
 - 架构必须稳定
 - 服务必须高内聚，服务应该实现一小组强相关的功能
 - 服务必须符合开闭原则，将一同变更的内容打包在一起，以确保每个更改仅影响一个服务
 - 服务必须松耦合，每个服务都可以在不影响客户端的情况下更改实现
- 微服务划分方法
 - 纵向拆分
 - 从 **业务维度** 进行拆分。标准是按照业务的关联程度来决定，关联比较密切的业务适合拆分为一个微服务，而功能相对比较独立的业务适合单独拆分为一个微服务
 - 横向拆分
 - 从 **公共且独立功能** 维度拆分。标准是按照是否有公共的被多个其他服务调用，且依赖的资源独立不与其他业务耦合

3、聊聊：分布式应用AKF拆分原则和扩展原则

当我们需要分布式系统提供 stronger 的性能时，该怎样扩展系统呢？什么时候该加机器？什么时候该重构代码？扩容时，究竟该选择哈希算法还是最小连接数算法，才能有效提升性能？

在面对 Scalability 可伸缩性问题时，我们必须有一个系统的方法论，才能应对日益复杂的分布式系统。

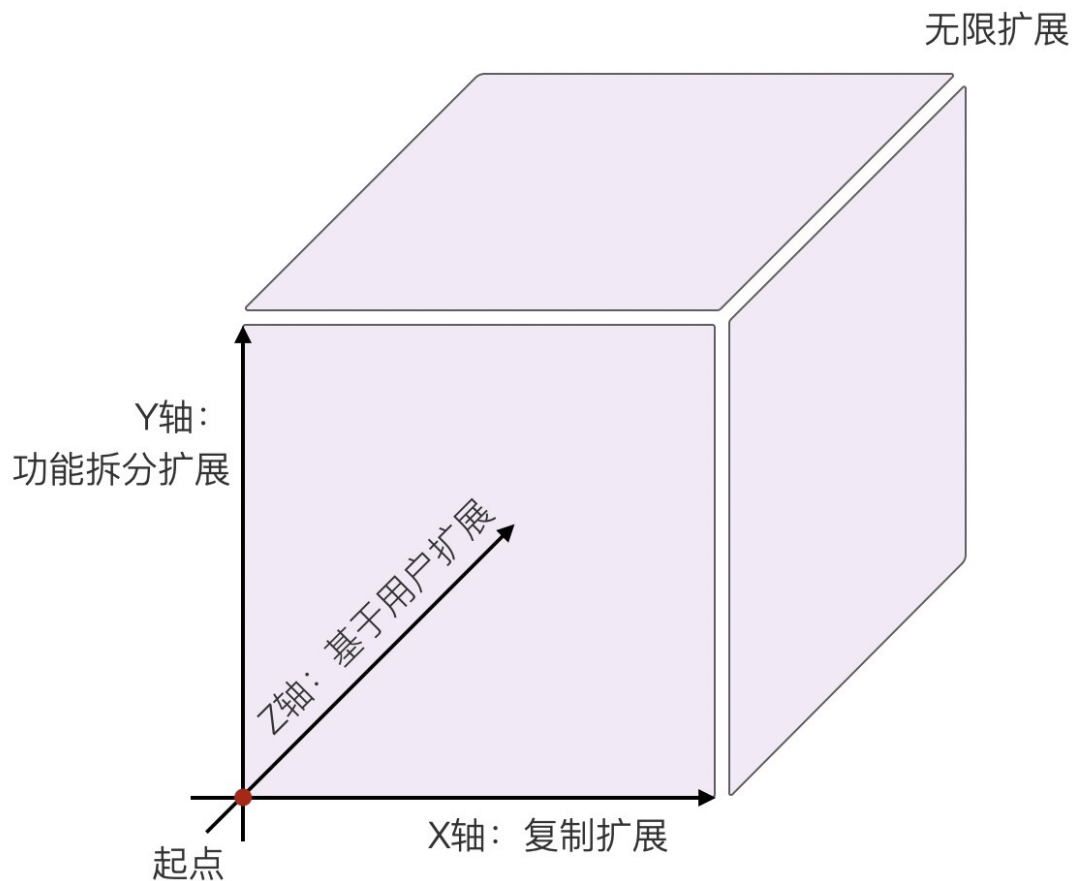
AKF 立方体理论，它定义了扩展系统的 3 个维度，是进行优化性能的一种理论依据。

什么是AKF

AKF 立方体也叫做scala cube，它在《The Art of Scalability》一书中被首次提出，旨在提供一个系统化的扩展思路。AKF 把系统扩展分为以下三个维度：

- X 轴：直接水平复制应用进程来扩展系统。
- Y 轴：将功能拆分出来扩展系统。
- Z 轴：基于用户信息扩展系统。

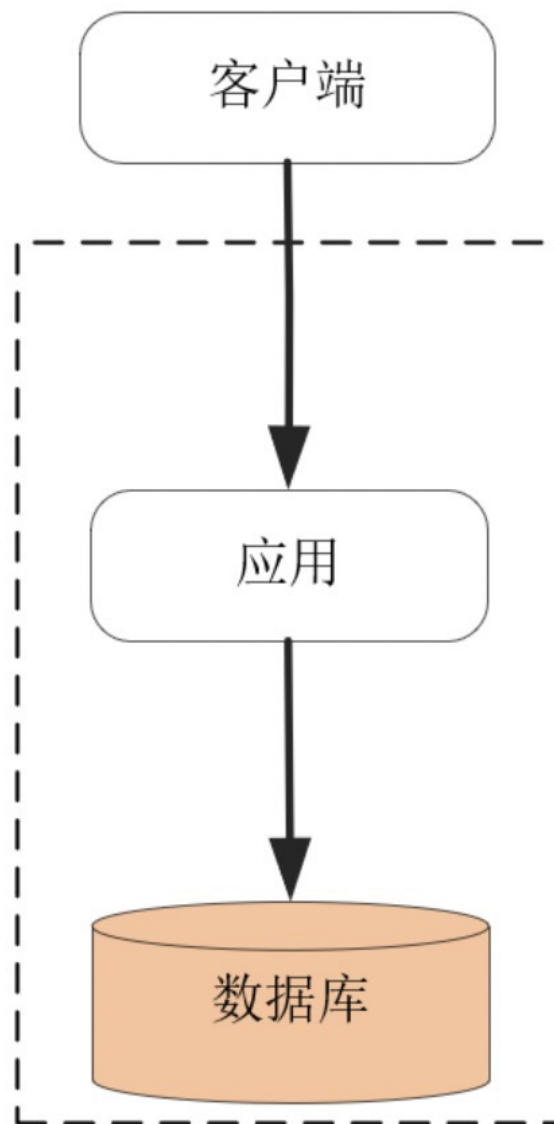
如下图所示：



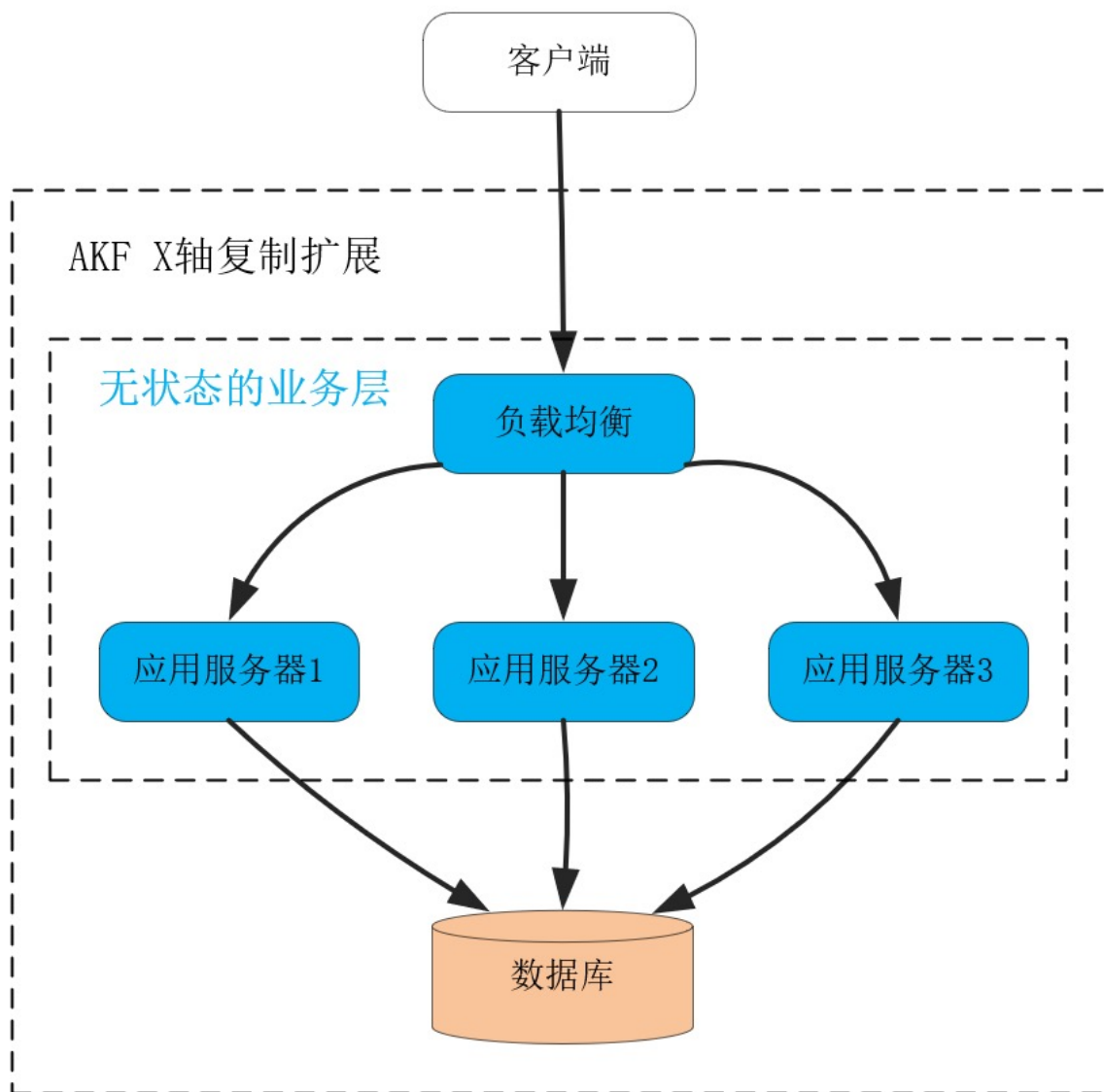
如何基于 AKF X 轴扩展系统？

我们日常见到的各种系统扩展方案，都可以归结到 AKF 立方体的这三个维度上。而且，我们可以同时组合这 3 个方向上的扩展动作，使得系统可以近乎无限地提升性能。为了避免对 AKF 的介绍过于抽象，下面我用一个实际的例子，带你看看这 3 个方向的扩展到底该如何应用。

假定我们开发一个博客平台，用户可以申请自己的博客帐号，并在其上发布文章。最初的系统考虑了 MVC 架构，将数据状态及关系模型交给数据库实现，应用进程通过 SQL 语言操作数据模型，经由 HTTP 协议对浏览器客户端提供服务，如下图所示：



在这个架构中，处理业务的应用进程属于无状态服务，用户数据全部放在了关系数据库中。因此，当我们在应用进程前加 1 个负载均衡服务后，就可以通过部署更多的应用进程，提供更大的吞吐量。而且，初期增加应用进程，RPS 可以获得线性增长，很实用，如下图：



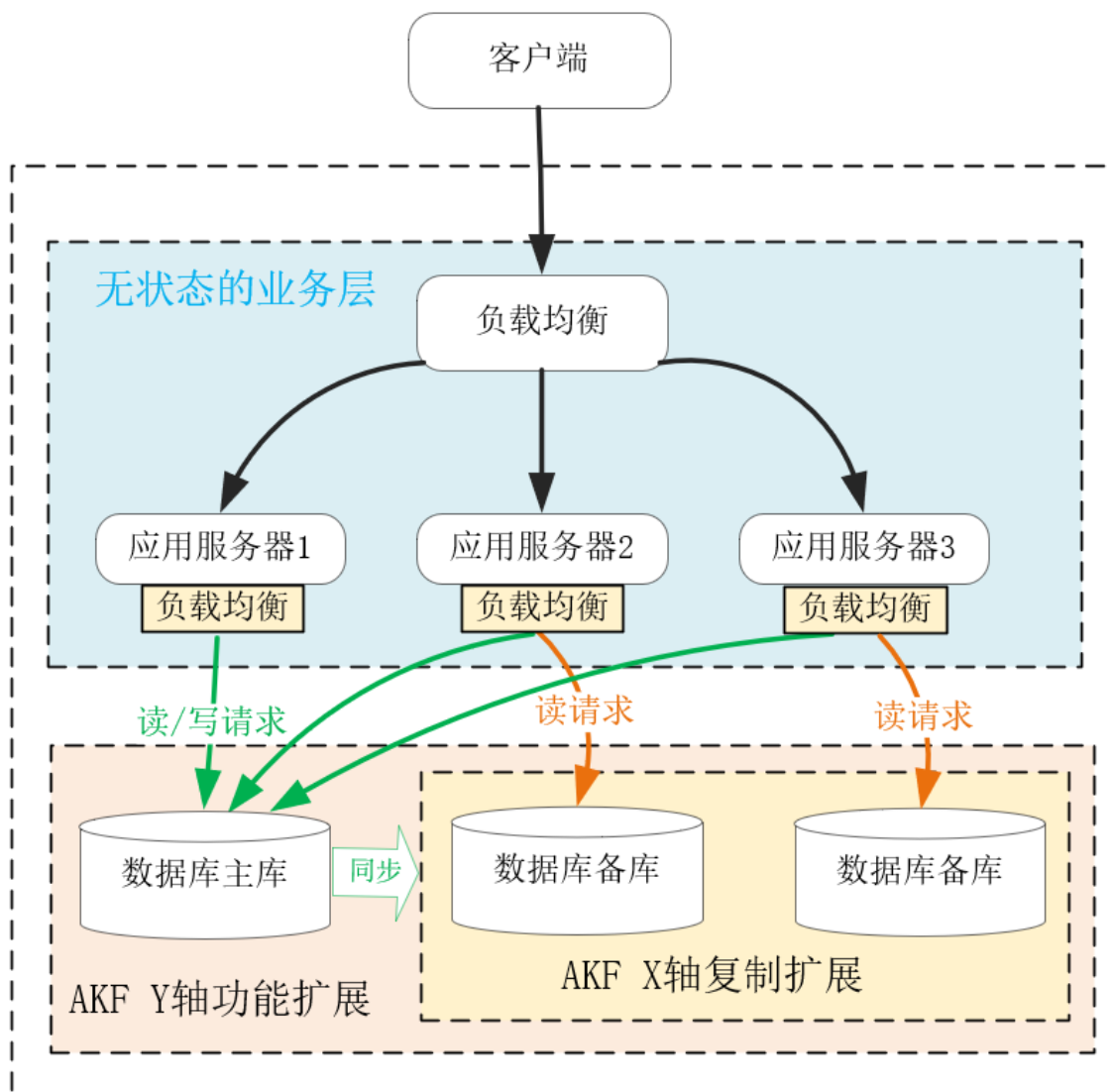
这就叫做沿 AKF X 轴扩展系统。这种扩展方式最大的优点，就是开发成本近乎为零，而且实施起来速度快！在搭建好负载均衡后，只需要在新的物理机、虚拟机或者微服务上复制程序，就可以让新进程分担请求流量，而且不会影响事务 Transaction 的处理。

当然，AKF X 轴扩展最大的问题是只能扩展无状态服务，当有状态的数据库出现性能瓶颈时，X 轴是无能为力的。例如，当用户数据量持续增长，关系数据库中的表就会达到百万、千万行数据，SQL 语句会越来越慢，这时可以沿着 AKF Z 轴去分库分表提升性能。又比如，当请求用户频率越来越高，那么可以把单实例数据库扩展为主备多实例，沿 Y 轴把读写功能分离提升性能。下面我们先来看 AKF Y 轴如何扩展系统。

如何基于 AKF Y 轴扩展系统？

当数据库的 CPU、网络带宽、内存、磁盘 IO 等某个指标率先达到上限后，系统的吞吐量就达到了瓶颈，此时沿着 AKF X 轴扩展系统，是没有办法提升性能的。

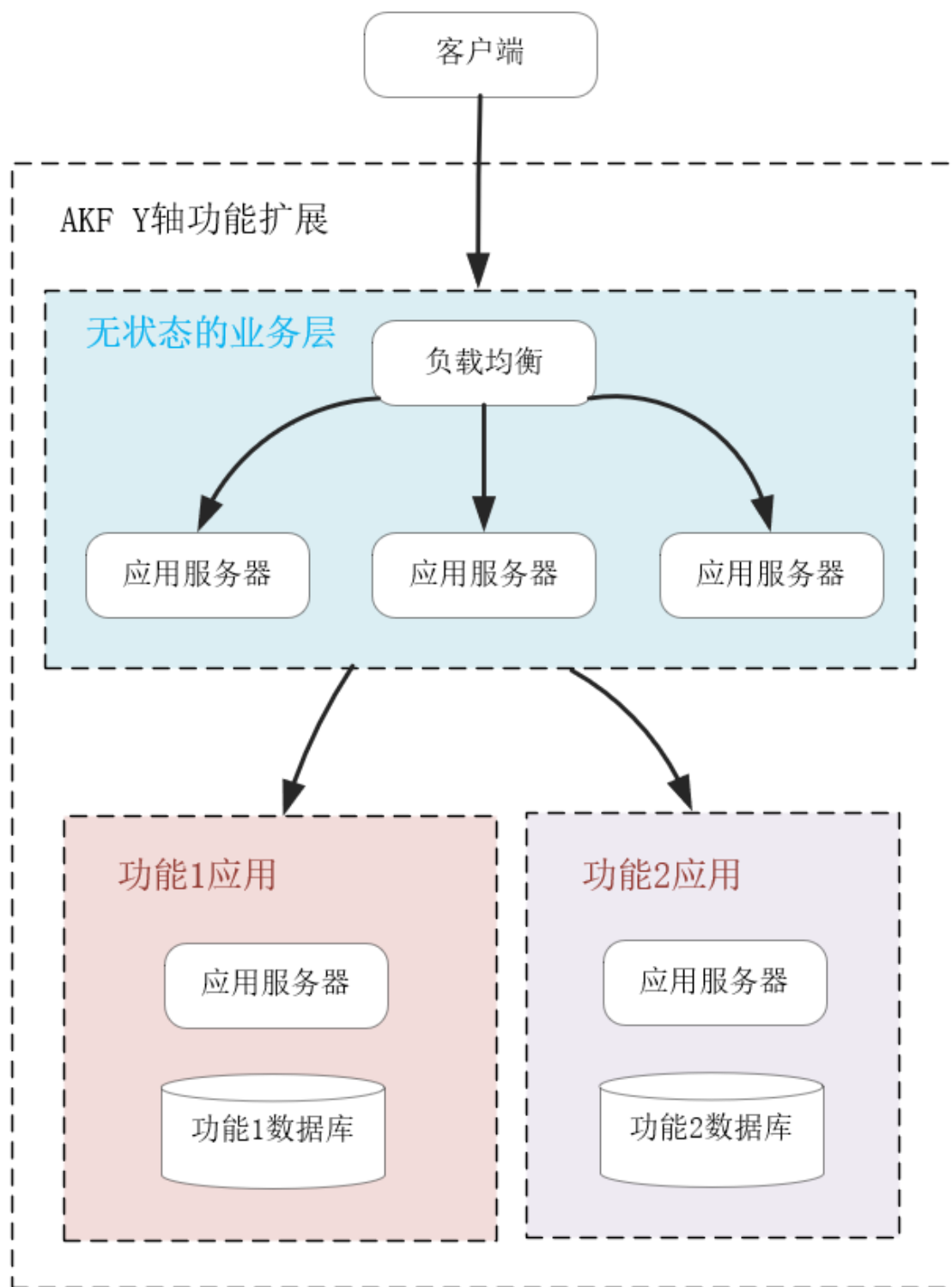
在现代经济中，更细分、更专业的产业化、供应链分工，可以给社会带来更高的效率，而 AKF Y 轴与之相似，当遇到上述性能瓶颈后，拆分系统功能，使得各组件的职责、分工更细，也可以提升系统的效率。比如，当我们将应用进程对数据库的读写操作拆分后，就可以扩展单机数据库为主备分布式系统，使得主库支持读写两种 SQL，而备库只支持读 SQL。这样，主库可以轻松地支持事务操作，且它将数据同步到备库中也并不复杂，如下图所示：



当然，上图中如果读性能达到了瓶颈，我们可以继续沿着 AKF X 轴，用复制的方式扩展多个备库，提升读 SQL 的性能，可见，AKF 多个轴完全可以搭配着协同使用。

拆分功能是需要重构代码的，它的实施成本比沿 X 轴简单复制扩展要高得多。在上图中，通常关系数据库的客户端 SDK 已经支持读写分离，所以实施成本由中间件承担了，这对我们理解 Y 轴的实施代价意义不大，所以我们再来看从业务上拆分功能的例子。

当这个博客平台访问量越来越大时，一台主库是无法扛住所有写流量的。因此，基于业务特性拆分功能，就是必须要做的工作。比如，把用户的个人信息、身份验证等功能拆分出一个子系统，再把文章、留言发布等功能拆分到另一个子系统，由无状态的业务层代码分开调用，并通过事务组合在一起，如下图所示：



这样，每个后端的子应用更加聚焦于细分的功能，它的数据库规模会变小，也更容易优化性能。比如，针对用户登录功能，你可以再次基于 Y 轴将身份验证功能拆分，用 Redis 等服务搭建一个基于 LRU 算法淘汰的缓存系统，快速验证用户身份。

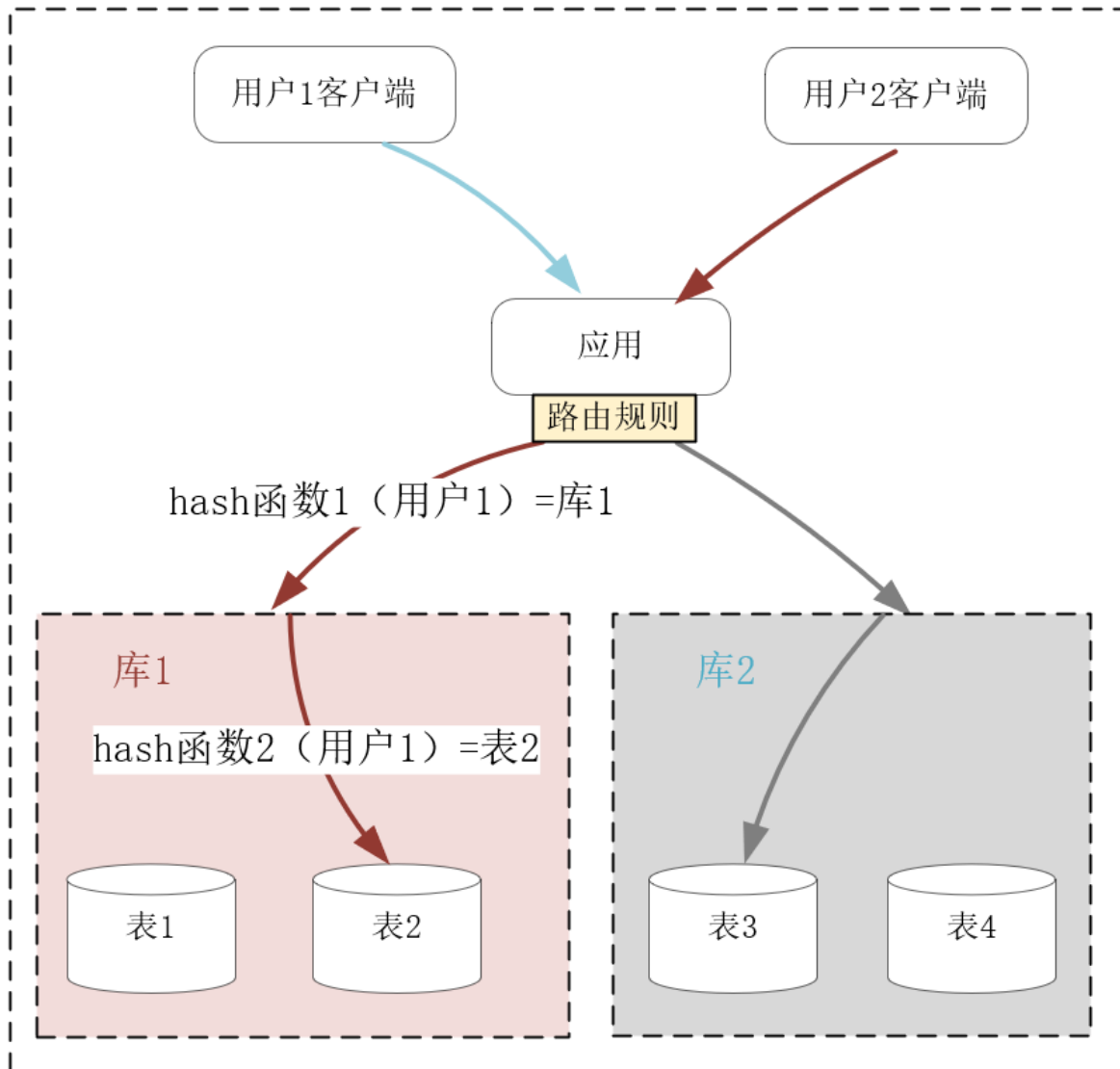
然而，沿 Y 轴做功能拆分，实施成本非常高，需要重构代码并做大量测试工作，上线部署也很复杂。比如上例中要对数据模型做拆分（如同一个库中的表拆分到多个库中，或者表中的字段拆到多张表中），设计组件之间的 API 交互协议，重构无状态应用进程中的代码，为了完成升级还要做数据迁移，等等。解决数据增长引发的性能下降问题，除了成本较高的 AKF Y 轴扩展方式外，沿 Z 轴扩展系统也很有效，它的实施成本更低一些，下面我们具体看一下。

如何基于 AKF Z 轴扩展系统？

不同于站在服务角度扩展系统的 X 轴和 Y 轴，AKF Z 轴则从用户维度拆分系统，它不仅可以提升数据持续增长降低的性能，还能基于用户的地理位置获得额外收益。

仍然以上面虚拟的博客平台为例，当注册用户数量上亿后，无论你怎么基于 Y 轴的功能去拆分表（即“垂直”地拆分表中的字段），都无法使得关系数据库单个表的行数在千万级以下，这样表字段的 B 树索引非常庞大，难以完全放在内存中，最后大量的磁盘 IO 操作会拖慢 SQL 语句的执行。

这个时候，关系数据库最常用的分库分表操作就登场了，它正是 AKF 沿 Z 轴拆分系统的实践。比如已经含有上亿行数据的 User 用户信息表，可以分成 10 个库，每个库再分成 10 张表，利用固定的哈希函数，就可以把每个用户的数据映射到某个库的某张表中。这样，单张表的数据量就可以降低到 1 百万行左右，如果每个库部署在不同的服务器上（具体的部署方式视访问吞吐量以及服务器的配置而定），它们处理的数据量减少了很多，却可以独占服务器的硬件资源，性能自然就有了提升。如下图所示：



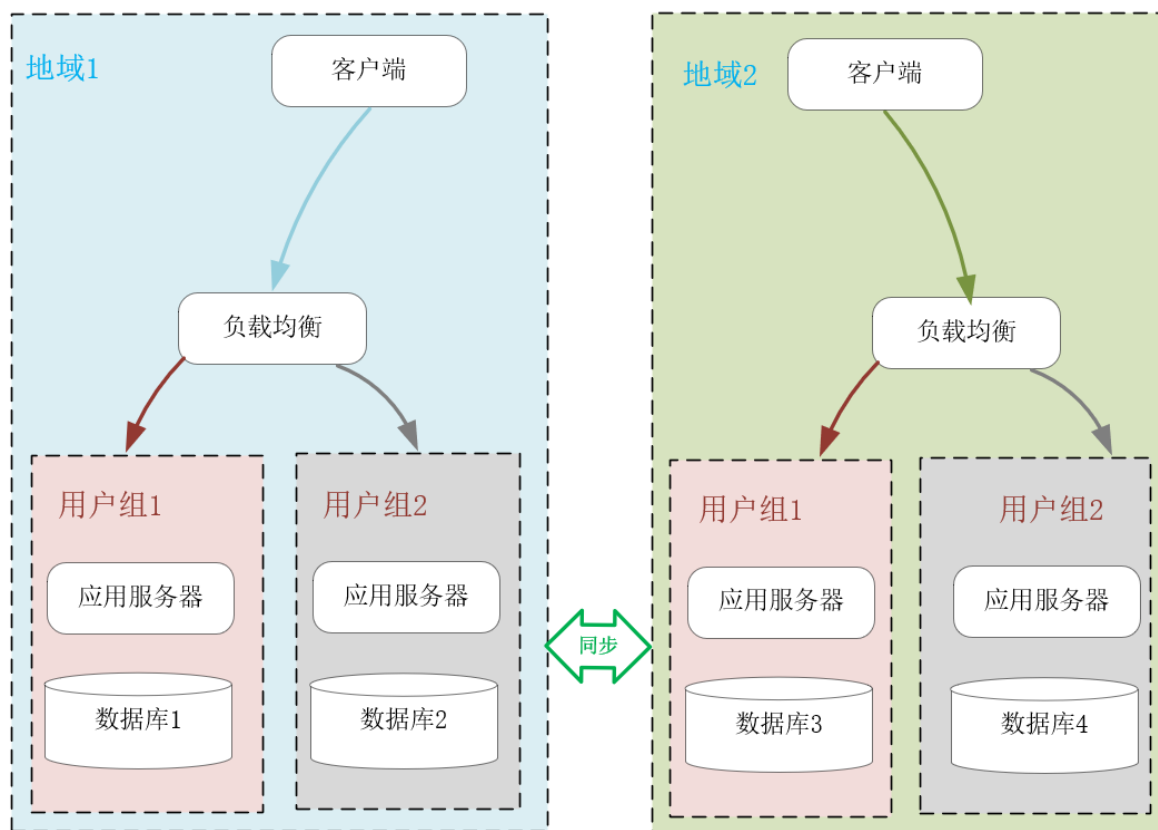
分库分表是关系数据库中解决数据增长压力的最有效办法，但分库分表同时也导致跨表的查询语句复杂许多，而跨库的事务几乎难以实现，因此这种扩展的代价非常高。当然，如果你使用的是类似 MySQL 这些成熟的关系数据库，整个生态中会有厂商提供相应的中间件层，使用它们可以降低 Z 轴扩展的代价。

再比如，最开始我们采用 X 轴复制扩展的服务，它们的负载均衡策略很简单，只需要选择负载最小的上游服务器即可，比如 RoundRobin 或者最小连接算法都可以达到目的。但若上游服务器通过 Y 轴扩展，开启了缓存功能，那么考虑到缓存的命中率，就必须改用 Z 轴扩展的方式，基于用户信息做哈希规则下的新路由，尽量将同一个用户的请求命中相同的上游服务器，才能充分提高缓存命中率。

Z 轴扩展还有一个好处，就是可以充分利用 IDC 与用户间的网速差，选择更快的 IDC 为用户提供高性能服务。网络是基于光速传播的，当 IDC 跨城市、国家甚至大洲时，用户访问不同 IDC 的网速就会有很大差异。当然，同一地域内不同的网络运营商之间，也会有很大的网速差。

例如你在全球都有 IDC 或者公有云服务器时，就可以通过域名为当地用户就近提供服务，这样性能会高很多。事实上，CDN 技术就基于 IP 地址的位置信息，就近为用户提供静态资源的高速访问。

下图中，我使用了 2 种 Z 轴扩展系统的方式。首先是基于客户端的地理位置，选择不同的 IDC 就近提供服务。其次是将不同的用户分组，比如免费用户组与付费用户组，这样在业务上分离用户群体后，还可以有针对性地提供不同水准的服务。



沿 AKF Z 轴扩展系统可以解决数据增长带来的性能瓶颈，也可以基于数据的空间位置提升系统性能，然而它的实施成本比较高，尤其是在系统宕机、扩容时，一旦路由规则发生变化，会带来很大的数据迁移成本，[第 24 讲] 我将要介绍的一致性哈希算法，其实就是用来解决这一问题的。

总之

X 轴扩展系统时实施成本最低，只需要将程序复制到不同的服务器上运行，再用下游的负载均衡分配流量即可。

X 轴只能应用在无状态进程上，**故无法解决数据增长引入的性能瓶颈。**

Y 轴扩展系统时实施成本最高，通常涉及到部分代码的重构，但它通过拆分功能，使系统中的组件分工更细，因此可以解决数据增长带来的性能压力，也可以提升系统的总体效率。

比如关系数据库的读写分离、表字段的垂直拆分，**或者引入缓存，都属于沿 Y 轴扩展系统。**

Z 轴扩展系统时实施成本也比较高，但它基于用户信息拆分数据后，可以在解决数据增长问题的同时，基于地理位置就近提供服务，进而大幅度降低请求的时延，比如常见的 CDN 就是这么提升用户体验的。

但 Z 轴扩展系统后，一旦发生路由规则的变动导致数据迁移时，运维成本就会比较高。

当然，X、Y、Z 轴的扩展并不是孤立的，我们可以同时应用这 3 个维度扩展系统。分布式系统非常复杂，AKF 给我们提供了一种自上而下的方法论，让我们能够针对不同场景下的性能瓶颈，以最低的成本提升性能。

聊聊：MicroService核心需要解决的问题？以及如何解决？

1. 服务注册与发现-注册中心
2. 微服务的统一入口-API网管
3. 服务容错限流
4. 服务的集中式配置中心
5. 服务调用链追踪
6. 服务部署
7. 服务监控

MicroService常见的解决方案

1. Springcloud Netflix
2. Springcloud alibaba
3. Dubbo
4. Kubernetes-k8s
5. Service mesh-istio
6. 其他

入门级 Spring Cloud试题

为什么需要学习Spring Cloud

不论是商业应用还是用户应用，在业务初期都很简单，我们通常会把它实现为单体结构的应用。但是，随着业务逐渐发展，产品思想会变得越来越复杂，单体结构的应用也会越来越复杂。这就会给应用带来如下的几个问题：

代码结构混乱：业务复杂，导致代码量很大，管理会越来越困难。同时，这也会给业务的快速迭代带来巨大挑战；开发效率变低：开发人员同时开发一套代码，很难避免代码冲突。开发过程会伴随着不断解决冲突的过程，这会严重的影响开发效率；排查解决问题成本高：线上业务发现 bug，修复 bug 的过程可能很简单。但是，由于只有一套代码，需要重新编译、打包、上线，成本很高。由于单体结构的应用随着系统复杂度的增高，会暴露出各种各样的问题。近些年来，微服务架构逐渐取代了单体架构，且这种趋势将会越来越流行。Spring Cloud是目前最常用的微服务开发框架，已经在企业级开发中大量的应用。

什么是Spring Cloud

Spring Cloud是一系列框架的有序集合。它利用Spring Boot的开发便利性巧妙地简化了分布式系统基础设施的开发，如服务发现注册、配置中心、智能路由、消息总线、负载均衡、断路器、数据监控等，都可以用Spring Boot的开发风格做到一键启动和部署。Spring Cloud并没有重复制造轮子，它只是将各家公司开发的比较成熟、经得起实际考验的服务框架组合起来，通过Spring Boot风格进行再封装屏蔽掉了复杂的配置和实现原理，最终给开发者留出了一套简单易懂、易部署和易维护的分布式系统开发工具包。

Spring Cloud设计目标与优缺点

设计目标

协调各个微服务，简化分布式系统开发。

优缺点

微服务的框架那么多比如：dubbo、Kubernetes，为什么就要使用Spring Cloud的呢？

优点：

产出于Spring大家族，Spring在企业级开发框架中无人能敌，来头很大，可以保证后续的更新、完善组件丰富，功能齐全。Spring Cloud 为微服务架构提供了非常完整的支持。例如、配置管理、服务发现、断路器、微服务网关等；Spring Cloud 社区活跃度很高，教程很丰富，遇到问题很容易找到解决方案服务拆分粒度更细，耦合度比较低，有利于资源重复利用，有利于提高开发效率可以更精准的制定优化服务方案，提高系统的可维护性减轻团队的成本，可以并行开发，不用关注其他人怎么开发，先关注自己的开发微服务可以是跨平台的，可以用任何一种语言开发适于互联网时代，产品迭代周期更短缺点：

微服务过多，治理成本高，不利于维护系统分布式系统开发的成本高（容错，分布式事务等）对团队挑战大总的来说优点大过于缺点，目前看来Spring Cloud是一套非常完善的分布式框架，目前很多企业开始用微服务、Spring Cloud的优势是显而易见的。因此对于想研究微服务架构的同学来说，学习Spring Cloud是一个不错的选择。

聊聊：使用 Spring Cloud 有什么优势？

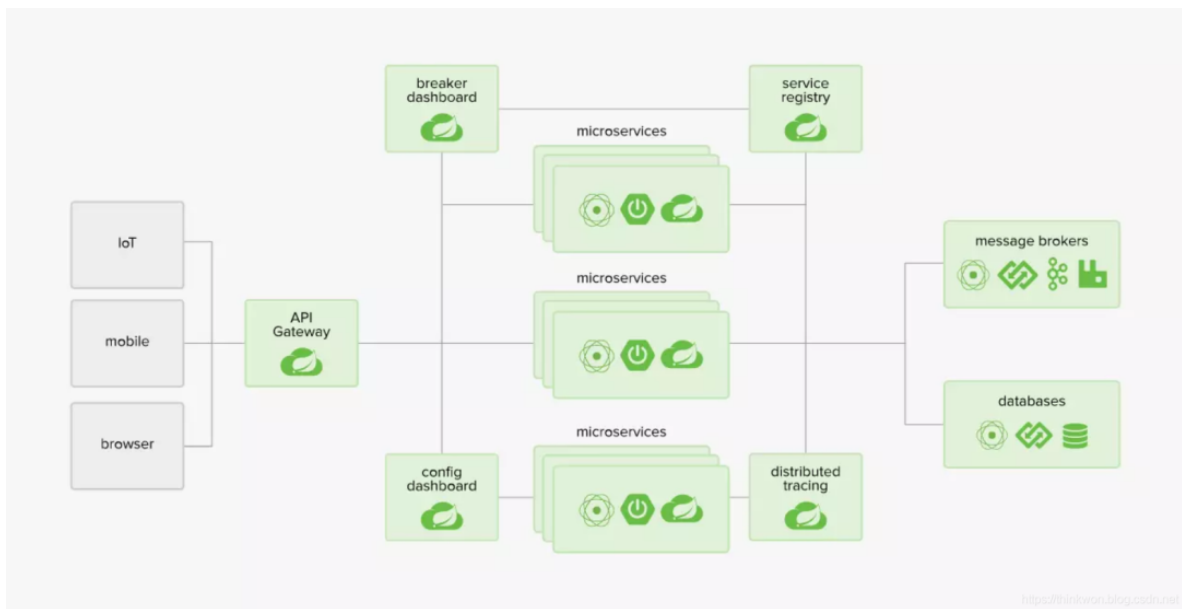
使用 Spring Boot 开发分布式微服务时，我们面临以下问题

- (1) 与分布式系统相关的复杂性-这种开销包括网络问题，延迟开销，带宽问题，安全问题。
- (2) 服务发现-服务发现工具管理群集中的流程和服务如何查找和互相交谈。它涉及一个服务目录，在该目录中注册服务，然后能够查找并连接到该目录中的服务。
- (3) 冗余-分布式系统中的冗余问题。
- (4) 负载均衡 --负载均衡改善跨多个计算资源的工作负荷，诸如计算机，计算机集群，网络链路，中央处理单元，或磁盘驱动器的分布。
- (5) 性能-问题 于各种运营开销导致的性能问题。
- (6) 部署复杂性 devops 技能的要求。

Spring Cloud发展前景

Spring Cloud对于中小型互联网公司来说是一种福音，因为这类公司往往没有实力或者没有足够的资金投入去开发自己的分布式系统基础设施，使用Spring Cloud一站式解决方案能在从容应对业务发展的同时大大减少开发成本。同时，随着近几年微服务架构和Docker容器概念的火爆，也会让Spring Cloud在未來越来越“云”化的软件开发风格中立有一席之地，尤其是在五花八门的分布式解决方案中提供了标准化的、全站式的技术方案，意义可能会堪比当年Servlet规范的诞生，有效推进服务端软件系统技术水平的进步。

聊聊：SpringCloud整体架构



Spring Cloud的子项目，大致可分成两类，一类是对现有成熟框架"Spring Boot化"的封装和抽象，也是数量最多的项目；第二类是开发了一部分分布式系统的基础设施的实现，如Spring Cloud Stream扮演的就是kafka, ActiveMQ这样的角色。

Spring Cloud Config

集中配置管理工具，分布式系统中统一的外部配置管理，默认使用Git来存储配置，可以支持客户端配置的刷新及加密、解密操作。

Spring Cloud Netflix

Netflix OSS 开源组件集成，包括Eureka、Hystrix、Ribbon、Feign、Zuul等核心组件。

- Eureka：服务治理组件，包括服务端的注册中心和客户端的服务发现机制；
- Ribbon：负载均衡的服务调用组件，具有多种负载均衡调用策略；
- Hystrix：服务容错组件，实现了断路器模式，为依赖服务的出错和延迟提供了容错能力；
- Feign：基于Ribbon和Hystrix的声明式服务调用组件；
- Zuul：API网关组件，对请求提供路由及过滤功能。

Spring Cloud Bus

用于传播集群状态变化的消息总线，使用轻量级消息代理链接分布式系统中的节点，可以用来动态刷新集群中的服务配置。

Spring Cloud Consul

基于Hashicorp Consul的服务治理组件。

Spring Cloud Security

安全工具包，对Zuul代理中的负载均衡OAuth2客户端及登录认证进行支持。

Spring Cloud Sleuth

Spring Cloud应用程序的分布式请求链路跟踪，支持使用Zipkin、HTrace和基于日志（例如ELK）的跟踪。

Spring Cloud Stream

轻量级事件驱动微服务框架，可以使用简单的声明式模型来发送及接收消息，主要实现为Apache Kafka及RabbitMQ。

Spring Cloud Task

用于快速构建短暂、有限数据处理任务的微服务框架，用于向应用中添加功能性和非功能性的特性。

Spring Cloud Zookeeper

基于Apache Zookeeper的服务治理组件。

Spring Cloud Gateway

API网关组件，对请求提供路由及过滤功能。

Spring Cloud OpenFeign

基于Ribbon和Hystrix的声明式服务调用组件，可以动态创建基于Spring MVC注解的接口实现用于服务调用，在Spring Cloud 2.0中已经取代Feign成为了一等公民。

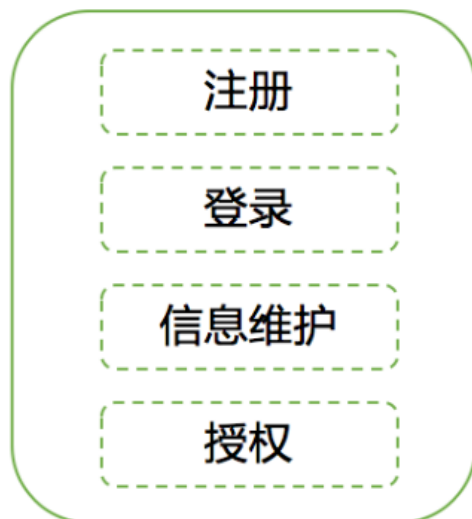
聊聊： MicroService是什么？ 优势是什么？

微服务架构是一种架构风格，一个大型复杂软件应由一个或多个微服务组成。

系统中的各个微服务可被独立技术选型,独立开发,独立部署,独立运维，各个微服务之间是松耦合的。

每个微服务仅关注于完成一件任务并很好地完成该任务。在所有情况下，每个任务代表着一个小的业务能力。

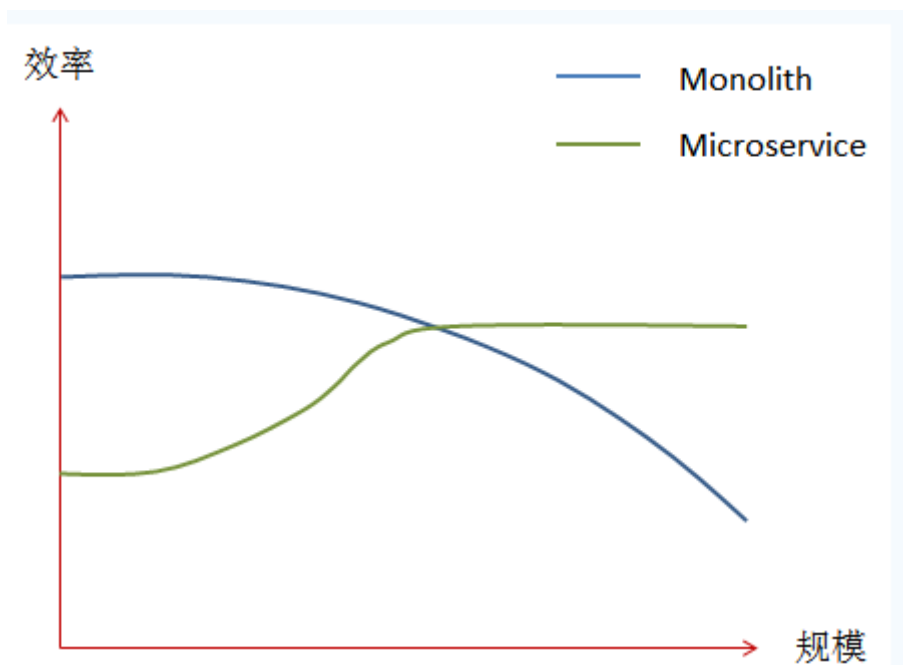
单体架构



微服务架构



Monolith&MicroService选型



从上图中可以看到，在刚开始的阶段，使用Microservice架构模式开发应用的效率明显低于Monolith。但是随着应用规模的增大，基于Microservice架构模式的开发效率将明显上升，而基于Monolith模式开发的效率将逐步下降。

单体应用架构：中小型项目(功能相对较少),公司官网，管理系统等

微服务架构：大型项目（功能比较多） 商城 erp, 人力资源, 宠物乐园等-互联网项目

聊聊: Spring Cloud主要的子项目有哪些

Spring Cloud的子项目，大致可分成两类，一类是对现有成熟框架"Spring Boot化"的封装和抽象，也是数量最多的项目；第二类是开发了一部分分布式系统的基础设施的实现，如Spring Cloud Stream扮演的就是kafka, ActiveMQ这样的角色。

Spring Cloud Config

集中配置管理工具，分布式系统中统一的外部配置管理，默认使用Git来存储配置，可以支持客户端配置的刷新及加密、解密操作。

Spring Cloud Netflix

Netflix OSS 开源组件集成，包括Eureka、Hystrix、Ribbon、Feign、Zuul等核心组件。

Eureka：服务治理组件，包括服务端的注册中心和客户端的服务发现机制；Ribbon：负载均衡的服务调用组件，具有多种负载均衡调用策略；Hystrix：服务容错组件，实现了断路器模式，为依赖服务的出错和延迟提供了容错能力；Feign：基于Ribbon和Hystrix的声明式服务调用组件；Zuul：API网关组件，对请求提供路由及过滤功能。

Spring Cloud Bus

用于传播集群状态变化的消息总线，使用轻量级消息代理链接分布式系统中的节点，可以用来动态刷新集群中的服务配置。

Spring Cloud Consul

基于Hashicorp Consul的服务治理组件。

Spring Cloud Security

安全工具包，对Zuul代理中的负载均衡OAuth2客户端及登录认证进行支持。

Spring Cloud Sleuth

Spring Cloud应用程序的分布式请求链路跟踪，支持使用Zipkin、HTrace和基于日志（例如ELK）的跟踪。

Spring Cloud Stream

轻量级事件驱动微服务框架，可以使用简单的声明式模型来发送及接收消息，主要实现为Apache Kafka及RabbitMQ。

Spring Cloud Task

用于快速构建短暂、有限数据处理任务的微服务框架，用于向应用中添加功能性和非功能性的特性。

Spring Cloud Zookeeper

基于Apache Zookeeper的服务治理组件。

Spring Cloud Gateway

API网关组件，对请求提供路由及过滤功能。

Spring Cloud OpenFeign

基于Ribbon和Hystrix的声明式服务调用组件，可以动态创建基于Spring MVC注解的接口实现用于服务调用，在Spring Cloud 2.0中已经取代Feign成为了一等公民。

聊聊：Spring Cloud的常用组件

- 服务注册与发现：Eureka、Nacos
- 配置中心：Config、Apollo、Nacos
- 网关：Zuul、Config
- 服务调用与负载均衡：Feign、Ribbon
- 断路器：Hystrix、Sentinel
- 链路追踪：Sleuth、Zipkin

聊聊：Spring Cloud的版本关系

Spring Cloud是一个由许多子项目组成的综合项目，各子项目有不同的发布节奏。为了管理Spring Cloud与各子项目的版本依赖关系，发布了一个清单，其中包括了某个Spring Cloud版本对应的子项目版本。为了避免Spring Cloud版本号与子项目版本号混淆，Spring Cloud版本采用了名称而非版本号的命名，这些版本的名字采用了伦敦地铁站的名字，根据字母表的顺序来对应版本时间顺序，例如Angel

是第一个版本，Brixton是第二个版本。当Spring Cloud的发布内容积累到临界点或者一个重大BUG被解决后，会发布一个"service releases"版本，简称SRX版本，比如Greenwich.SR2就是Spring Cloud发布的Greenwich版本的第2个SRX版本。目前Spring Cloud的最新版本是Hoxton。

聊聊：Spring Cloud和SpringBoot版本对应关系

Spring Cloud Version	SpringBoot Version
Hoxton	2.2.x
Greenwich	2.1.x
Finchley	2.0.x
Edgware	1.5.x
Dalston	1.5.x

聊聊：Spring Cloud和各子项目版本对应关系

Component	Edgware.SR6	Greenwich.SR2
spring-cloud-bus	1.3.4.RELEASE	2.1.2.RELEASE
spring-cloud-commons	1.3.6.RELEASE	2.1.2.RELEASE
spring-cloud-config	1.4.7.RELEASE	2.1.3.RELEASE
spring-cloud-netflix	1.4.7.RELEASE	2.1.2.RELEASE
spring-cloud-security	1.2.4.RELEASE	2.1.3.RELEASE
spring-cloud-consul	1.3.6.RELEASE	2.1.2.RELEASE
spring-cloud-sleuth	1.3.6.RELEASE	2.1.1.RELEASE
spring-cloud-stream	Ditmars.SR5	Fishtown.SR3
spring-cloud-zookeeper	1.2.3.RELEASE	2.1.2.RELEASE
spring-boot	1.5.21.RELEASE	2.1.5.RELEASE
spring-cloud-task	1.2.4.RELEASE	2.1.2.RELEASE
spring-cloud-gateway	1.0.3.RELEASE	2.1.2.RELEASE
spring-cloud-openfeign	暂无	2.1.2.RELEASE

注意：Hoxton版本是基于SpringBoot 2.2.x版本构建的，不适用于1.5.x版本。

随着2019年8月SpringBoot 1.5.x版本停止维护，Edgware版本也将停止维护。

聊聊：SpringBoot和SpringCloud的区别？

SpringBoot专注于快速方便的开发单个个体微服务。

SpringCloud是关注全局的微服务协调整理治理框架，它将SpringBoot开发的一个个单体微服务整合并管理起来，

为各个微服务之间提供，配置管理、服务发现、断路器、路由、微代理、事件总线、全局锁、决策竞选、分布式会话等等集成服务

SpringBoot可以离开SpringCloud独立使用开发项目，但是SpringCloud离不开SpringBoot，属于依赖的关系

SpringBoot专注于快速、方便的开发单个微服务个体，SpringCloud关注全局的服务治理框架。

使用 Spring Boot 开发分布式微服务时，我们面临以下问题

- (1) 与分布式系统相关的复杂性 - 这种开销包括网络问题，延迟开销，带宽问题，安全问题。
- (2) 服务发现 - 服务发现工具管理群集中的流程和服务如何查找和互相交谈。它涉及一个服务目录，在该目录中注册服务，然后能够查找并连接到该目录中的服务。
- (3) 冗余 - 分布式系统中的冗余问题。
- (4) 负载均衡 - 负载均衡改善跨多个计算资源的工作负荷，诸如计算机，计算机集群，网络链路，中央处理单元，或磁盘驱动器的分布。
- (5) 性能问题 - 由于各种运营开销导致的性能问题。
- (6) 部署复杂性 - DevOps的要求。

服务注册和发现是什么意思？Spring Cloud 如何实现？

当我们开始一个项目时，我们通常在属性文件中进行所有的配置。随着越来越多的服务开发和部署，添加和修改这些属性变得更加复杂。有些服务可能会下降，而某些位置可能会发生变化。手动更改属性可能会产生问题。Eureka 服务注册和发现可以在这种情况下提供帮助。由于所有服务都在 Eureka 服务器上注册并通过调用 Eureka 服务器完成查找，因此无需处理服务地点的任何更改和处理。

聊聊：Spring Cloud 和dubbo区别？

- (1) 服务调用方式：dubbo是RPC，Spring Cloud是Rest Api。
- (2) 注册中心：dubbo 是zookeeper，Spring Cloud可以是zookeeper或其他。
- (3) 服务网关：dubbo本身没有实现，只能通过其他第三方技术整合，springcloud有Zuul路由网关，作为路由服务器，进行消费者的请求分发,springcloud支持断路器，与git完美集成配置文件支持版本控制，事物总线实现配置文件的更新与服务自动装配等等一系列的微服务架构要素。
- (4) 架构完整度：Spring Cloud包含诸多微服务组件要素，完整度上比Dubbo高。

简单说一下Springcloud Netflix eureka

Eureka是netflix的一个子模块，也是核心模块之一。

Eureka是一个服务注册与发现组件,简单说就是用来统一管理微服务的通信地址，它包含了EurekaServer服务端(也叫注册中心)和EurekaClient客户端两部分组成，EurekaServer是独立的服务，而EurekaClient需要继承到每个微服务中。

微服务(EurekaClient)在启动的时候会向EurekaServer提交自己的通信地址清单如:服务名,ip,端口，在EurekaServer会形成一个微服务的通信地址列表 --- 这叫服务注册

微服务(EurekaClient)会定期的从EurekaServer拉取一份微服务通信地址列表缓存到本地。一个微服务在向另一个微服务发起调用的时候会根据目标服务的服务名找到其通信地址清单，然后基于HTTP协议向目标服务发起请求。---这叫服务发现

另外，微服务(EurekaClient)采用“心跳”机制向EurekaServer发请求进行服务续约，其实就是定时向EurekaServer发请求报告自己的健康状况，告诉EurekaServer自己还活着，不要把自己从服务地址清单中掉，那么当微服务(EurekaClient)宕机未向EurekaServer续约，或者续约请求超时，注册中心就会从地址清单中剔除该续约失败的服务

简单说一下Springcloud Netflix ribbon/feign

Ribbon是Netflix发布的云中间层服务开源项目，主要功能是提供客户端负载均衡算法。

Ribbon是一个客户端负载均衡器,它可以按照一定规则来完成多台服务器负载均衡调用,这些规则还支持自定义。

Feign是一个声明式的Web Service客户端，它的目的就是让Web Service调用更加简单。Feign提供了HTTP请求的接口模板（上面标的有访问地址），通过编写简单的接口和插入注解，就可以定义好HTTP请求的参数、格式、地址等信息。而Feign则会完全代理HTTP请求，我们只需要像调用方法一样调用它就可以完成服务请求及相关处理。Feign整合了Ribbon和Hystrix(关于Hystrix我们后面再讲)，可以让我们不再需要显式地使用这两个组件。

简单说一下Springcloud Netflix hystrix

Hystrix是国外知名的视频网站Netflix所开源的非常流行的高可用架构框架。Hystrix能够完美的解决分布式系统架构中打造高可用服务面临的一系列技术难题。

Hystrix “豪猪”，具有自我保护的能力。hystrix 通过如下机制来解决雪崩效应问题。

- 1 资源隔离（限流）：包括线程池隔离和信号量隔离，限制调用分布式服务的资源使用，某一个调用的服务出现问题不会影响其他服务调用。
- 2 熔断：当失败率达到阈值自动触发降级(如因网络故障/超时造成的失败率高)，熔断器触发的快速失败会进行快速恢复。
- 3 降级机制：超时降级、资源不足时(线程或信号量)降级，降级后可以配合降级接口返回托底数据。
- 4 缓存：提供了请求缓存、请求合并实现。

简单说一下Springcloud Netflix zuul

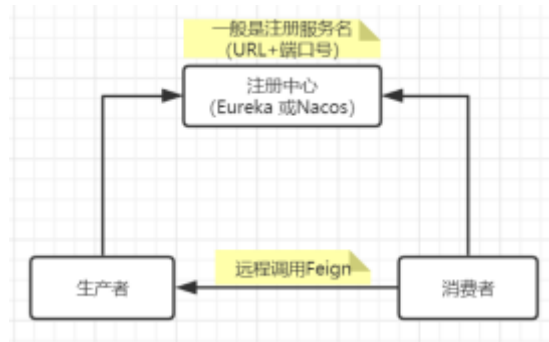
Zuul 是netflix开源的一个API Gateway 服务器。主要功能如下：

1. 动态路由 通过不同的地址区分调用哪个具体的服务
2. 过滤器 通过过滤器可以实现登录拦截，权限判断等
3. 负载均衡 底层通过ribbon实现负载均衡调用
4. 熔断降级 底层通过hystrix实现熔断降级等服务保障措施
5. ...

简单说一下Springcloud Netflix Config server

在分布式系统中，由于服务数量巨多，为了方便服务配置文件统一管理，实时更新，所以需要分布式配置中心组件。在Spring Cloud中，有分布式配置中心组件spring cloud config，它支持配置服务放在配置服务的内存中（即本地），也支持放在远程Git仓库中。在spring cloud config 组件中，分两个角色，一是config server，二是config client。

服务注册和发现是什么意思？Spring Cloud 如何实现？



当我们开始一个项目时，我们通常在属性文件中进行所有的配置。随着越来越多的服务开发和部署，添加和修改这些属性变得更加复杂。有些服务可能会下降，而某些位置可能会发生变化。手动更改属性可能会产生问题。

Eureka 服务注册和发现可以在这种情况下提供帮助。由于所有服务都在 Eureka 服务器上注册并通过调用 Eureka 服务器完成查找，因此无需处理服务地点的

Spring Cloud 提高 试题

聊聊：负载均衡的意义什么？

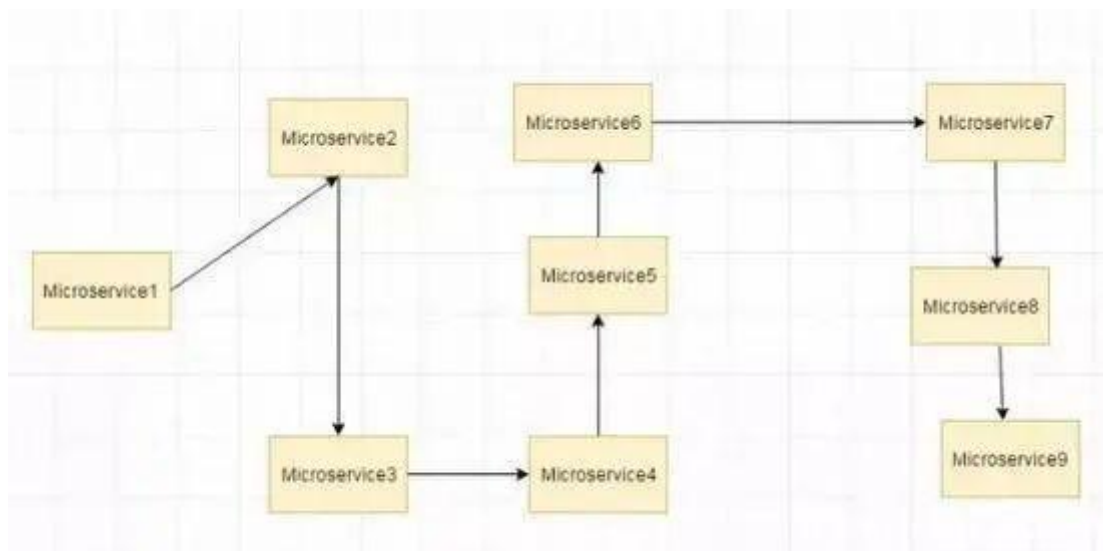
在计算中，负载均衡可以改善跨计算机，计算机集群，网络链接，中央处理单元或磁盘驱动器等多种计算资源的工作负载分布。负载均衡旨在优化资源使用，最大化吞吐量，最小化响应时间并避免任何单一资源的过载。使用多个组件进行负载均衡而不是单个组件可能会通过冗余来提高可靠性和可用性。负载均衡通常涉及专用软件或硬件，例如多层交换机或域名系统服务器进程。

聊聊：什么是 Hystrix？它如何实现容错？

Hystrix 是一个延迟和容错库，旨在隔离远程系统，服务和第三方库的访问点，当出现故障是不可避免的故障时，停止级联故障并在复杂的分布式系统中实现弹性。

通常对于使用微服务架构开发的系统，涉及到许多微服务。这些微服务彼此协作。

思考以下微服务



假设如果上图中的微服务 9 失败了，那么使用传统方法我们将传播一个异常。但这仍然会导致整个系统崩溃。

随着微服务数量的增加，这个问题变得更加复杂。微服务的数量可以高达 1000.这是 hystrix 出现的地方 我们将使用 Hystrix 在这种情况下的 Fallback 方法功能。我们有两个服务 employee-consumer 使用 employee-consumer 公开的服务。

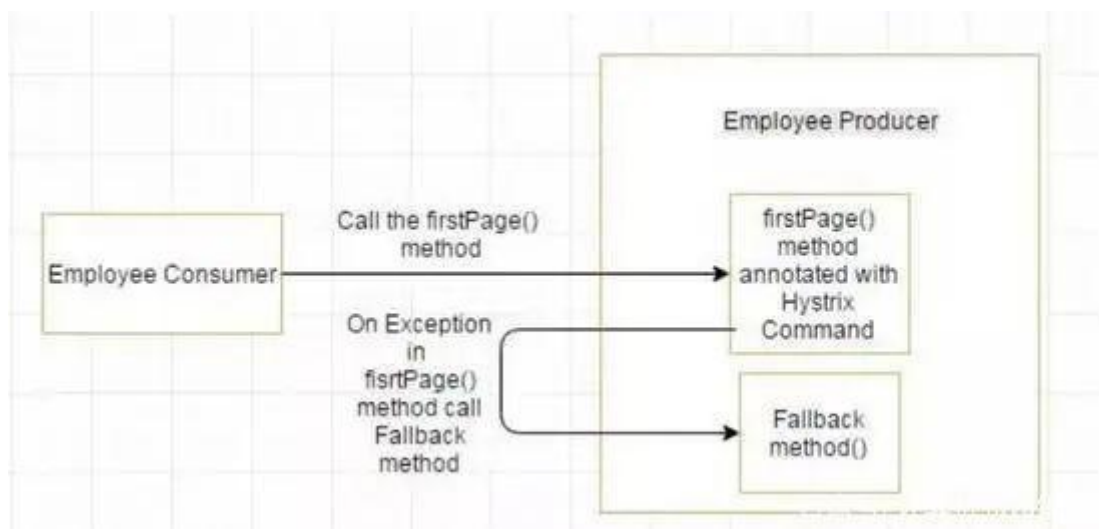
简化图如下所示



现在假设由于某种原因，employee-producer 公开的服务会抛出异常。我们在这种情况下使用 Hystrix 定义了一个回退方法。这种后备方法应该具有与公开服务相同的返回类型。如果暴露服务中出现异常，则回退方法将返回一些值。

聊聊：什么是 Hystrix 断路器？我们需要它吗？

由于某些原因，employee-consumer 公开服务会引发异常。在这种情况下使用Hystrix 我们定义了一个回退方法。如果在公开服务中发生异常，则回退方法返回一些默认值。



如果 firstPage method() 中的异常继续发生，则 Hystrix 电路将中断，并且employee使用者将一起跳过 firstPage 方法，并直接调用回退方法。断路器的目的是给firstPage方法或firstPage方法可能调用的其他方法留出时间，并导致异常恢复。可能发生的情况是，在负载较小的状况下，导致异常的问题有更好的恢复机会。

聊聊：什么是 Netflix Feign？它的优点是什么？

Feign 是受到 Retrofit, JAXRS-2.0 和 WebSocket 启发的 java 客户端联编程序。

Feign 的第一个目标是将约束分母的复杂性统一到 http apis，而不考虑其稳定性。

在 employee-consumer 的例子中，我们使用了 employee-producer 使用 REST模板公开的 REST 服务。

但是我们必须编写大量代码才能执行以下步骤

- (1) 使用功能区进行负载均衡。
- (2) 获取服务实例，然后获取基本 URL。
- (3) 利用 REST 模板来使用服务。前面的代码如下

```
1
2 @Controller
3 public class ConsumerControllerClient {
4     @Autowired
5     private LoadBalancerClient loadBalancer;
6     public void getEmployee() throws RestClientException, IOException {
7         ServiceInstance serviceInstance=loadBalancer.choose("employee-producer");
8         System.out.println(serviceInstance.getUri());
9         String baseUrl=serviceInstance.getUri().toString();
10        baseUrl=baseUrl+"/employee";
11        RestTemplate restTemplate = new RestTemplate();
12        ResponseEntity<String> response=null;
13        try{
14            response=restTemplate.exchange(baseUrl,
15                HttpMethod.GET, getHeaders(),String.class);
16        }
17        catch (Exception ex)
18        {
19            System.out.println(ex);
20        }
21        System.out.println(response.getBody());
22    }
23 }
```

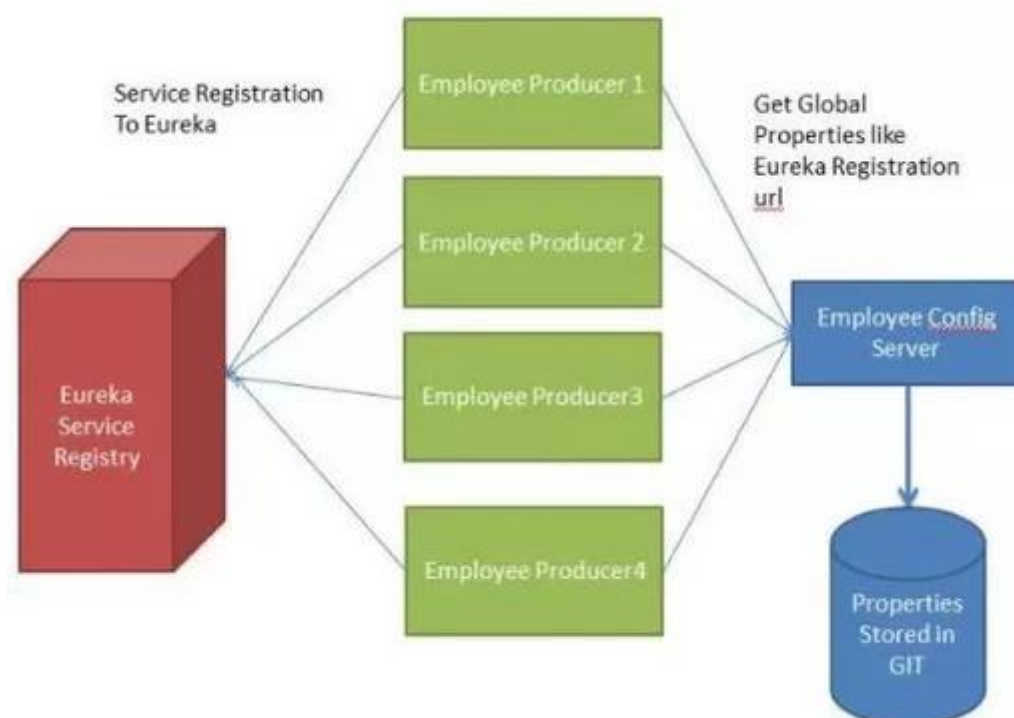
百家号/计算机java编程

之前的代码，有像 NullPointerException 的概率，并不是最优的。我们将看到如何使用 Netflix Feign 使调用变得更加简洁。而且如果当 Netflix Ribbon 依赖关系也在类路径中，那么 Feign 默认也会负责负载均衡。

聊聊：什么是 Spring Cloud Bus? 我们需要它吗?

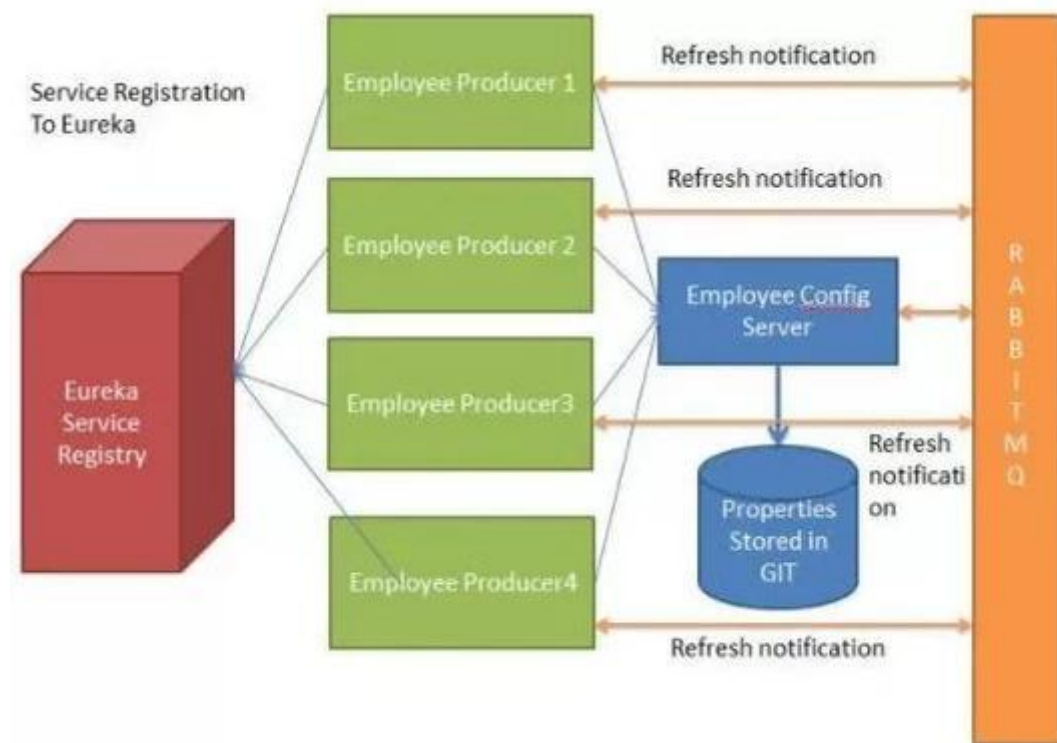
考虑以下情况：我们多个应用程序使用 Spring Cloud Config 读取属性，而 Spring Cloud Config 从 GIT 读取这些属性。

下面的例子中多个员工生产者模块从 Employee Config Module 获取 Eureka 注册的财产。



如果假设 GIT 中的 Eureka 注册属性更改为指向另一台 Eureka 服务器，会发生什么情况。在这种情况下，我们将不得不重新启动服务以获取更新的属性。

还有另一种使用执行器端点/刷新方式。但是我们将不得不为每个模块单独调用这个 url。例如，如果 Employee Producer1 部署在端口 8080 上，则调用 http://localhost:8080/refresh。同样对于 Employee Producer2 http://localhost:8081/refresh 等等。这又很麻烦。这就是 Spring Cloud Bus 发挥作用的地方。



Spring Cloud Bus 提供了跨多个实例刷新配置的功能。因此，在上面的示例中，如果我们刷新 Employee Producer1，则会自动刷新所有其他必需的模块。如果我们有多个微服务启动并运行，这特别有用。这是通过将所有微服务连接到单个消息代理来实现的。无论何时刷新实例，此事件都会订阅到侦听此代理的所有微服务，并且它们也会刷新。可以通过使用端点/总线/刷新来实现对任何单个实例的刷新。

聊聊：Spring Cloud断路器的作用

当一个服务调用另一个服务由于网络原因或自身原因出现问题，调用者就会等待被调用者的响应 当更多的服务请求到这些资源导致更多的请求等待，发生连锁效应（雪崩效应）

断路器有完全打开状态:一段时间内 达到一定的次数无法调用 并且多次监测没有恢复的迹象 断路器完全打开 那么下次请求就不会请求到该服务

半开:短时间内 有恢复迹象 断路器会将部分请求发给该服务，正常调用时 断路器关闭

关闭：当服务一直处于正常状态 能正常调用

聊聊：什么是Spring Cloud Config?

在分布式系统中，由于服务数量巨多，为了方便服务配置文件统一管理，实时更新，所以需要分布式配置中心组件。在Spring Cloud中，有分布式配置中心组件spring cloud config，它支持配置服务放在配置服务的内存中（即本地），也支持放在远程Git仓库中。在spring cloud config 组件中，分两个角色，一是config server，二是config client。

使用一般也是三步：

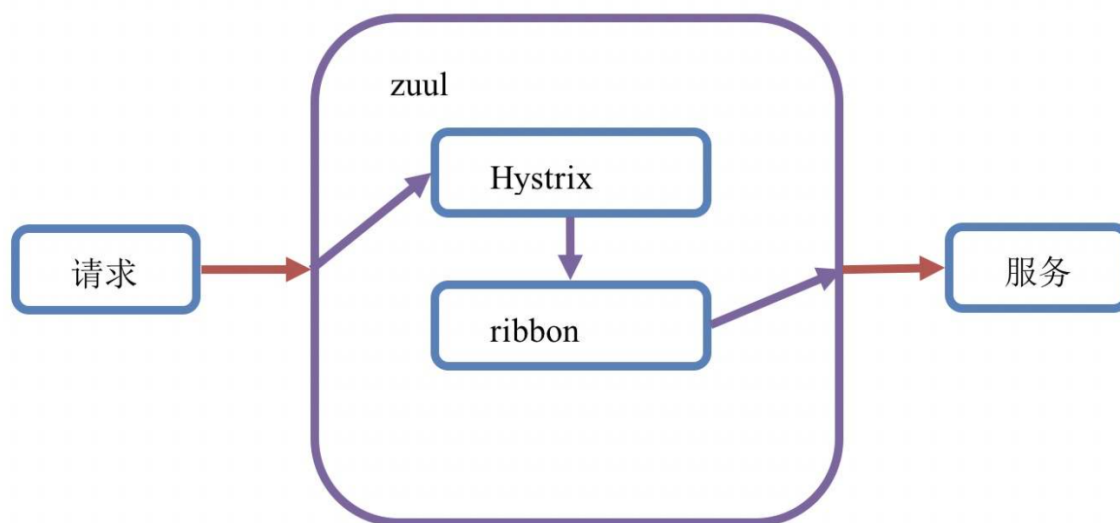
- (1) 添加pom依赖
- (2) 配置文件添加相关配置
- (3) 启动类添加注解@EnableConfigServer

聊聊：什么是Zuul?

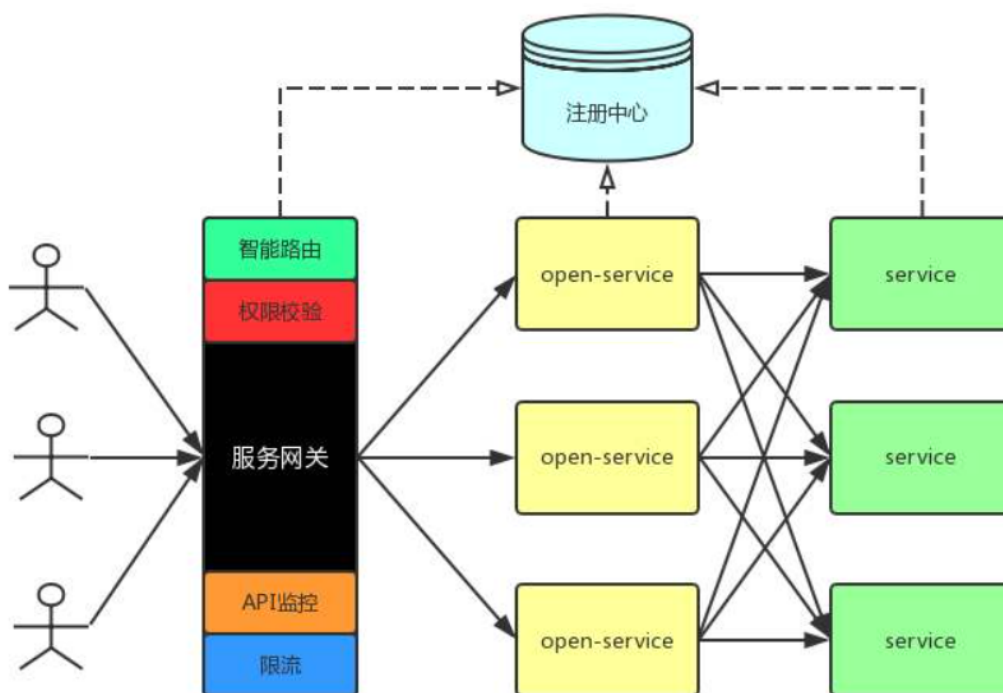
zuul 是netflix开源的一个API Gateway 服务器, 本质上是一个web servlet应用。

Zuul 在云平台上提供动态路由, 监控, 弹性, 安全等边缘服务的框架。

Zuul 相当于是设备和 Netflix 流应用的 Web 网站后端所有请求的前门。



zuul的例子可以参考 netflix 在github上的 simple webapp, 可以按照netflix 在github wiki 上文档说明来进行使用。



Zuul 的主要功能是路由转发和过滤器。

路由功能是微服务的一部分, 比如 /api/user 转发到到 User 服务, /api/shop 转发到到 Shop 服务。

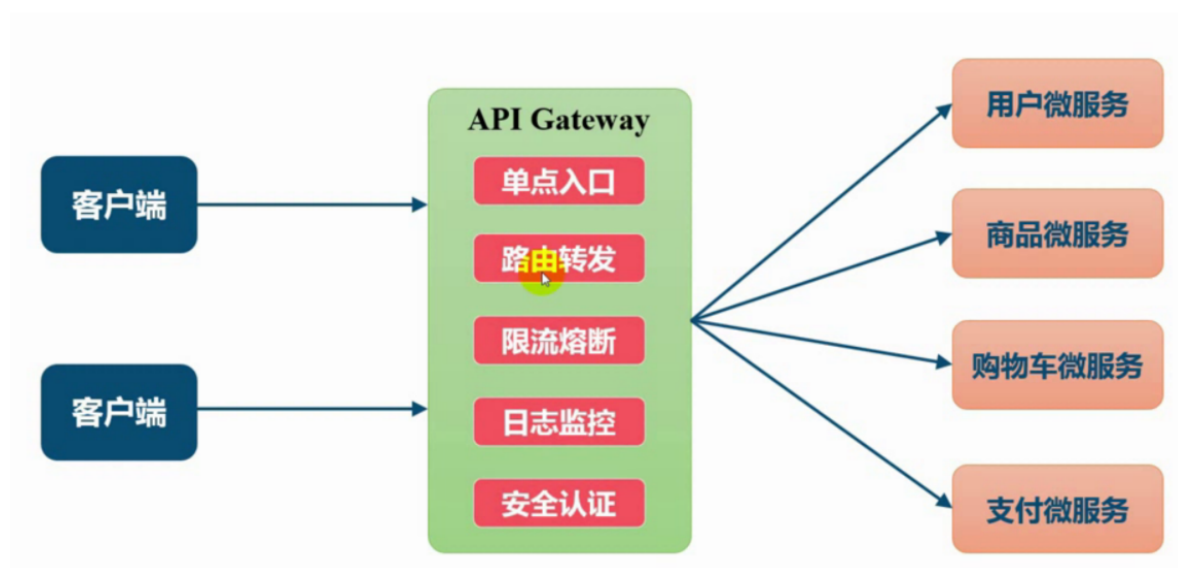
Zuul 默认和 Ribbon 结合实现了负载均衡的功能。

zuul承担的一个重要的角色：就是 api网关

聊聊：API网关是什么

API网关可以提供单独且统一的API入口用于访问内部一个或多个API。

简单来说嘛就是一个统一入口，比如现在的支付宝或者微信的相关api服务一样，都有一个统一的api地址，统一的请求参数，统一的鉴权。



这个图应该比较容易理解，其实就是一个公共的入口，通过这个入口可以访问到你想要的服务。

聊聊：SpringCloud中Zuul网关原理及其配置

Zuul是spring cloud中的微服务网关。网关：是一个网络整体系统中的前置门户入口。请求首先通过网关，进行路径的路由，定位到具体的服务节点上。

Zuul是一个微服务网关，首先是一个微服务。也是会在Eureka[注册中心](#)中进行服务的注册和发现。也是一个网关，请求应该通过Zuul来进行路由。

Zuul网关不是必要的。是推荐使用的。

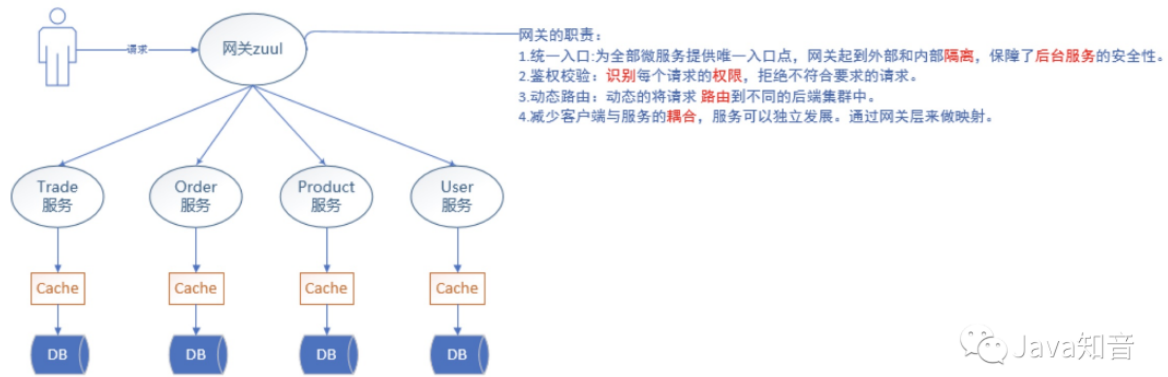
使用Zuul，一般在微服务数量较多（多于10个）的时候推荐使用，对服务的管理有严格要求的时候推荐使用，当微服务权限要求严格的时候推荐使用。

一、Zuul网关的作用

网关有以下几个作用：

- 统一入口：未全部为服务提供一个唯一的入口，网关起到外部和内部隔离的作用，保障了后台服务的安全性。
- 鉴权校验：识别每个请求的权限，拒绝不符合要求的请求。

- 动态路由：动态的将请求路由到不同的后端集群中。
- 减少客户端与服务端的耦合：服务可以独立发展，通过网关层来做映射。



Java知音

二、Zuul网关的应用

1、网关访问方式

通过zuul访问服务的，URL地址默认格式为：<http://zuulHostIp:port/要访问的服务名称/服务中的URL>

服务名称：properties配置文件中的spring.application.name。

服务的URL：就是对应的服务对外提供的URL路径监听。

2、网关依赖注入

```
<!-- spring cloud Eureka Client 启动器 -->
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-eureka</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-zuul</artifactId>
</dependency>
<!-- zuul网关的重试机制，不是使用ribbon内置的重试机制
    是借助spring-retry组件实现的重试
    开启zuul网关重试机制需要增加下述依赖
-->
<dependency>
    <groupId>org.springframework.retry</groupId>
    <artifactId>spring-retry</artifactId>
</dependency>
```

3、网关启动器

```

/**
 * @EnableZuulProxy - 开启Zuul网关。
 * 当前应用是一个Zuul微服务网关。会在Eureka注册中心中注册当前服务。并发现其他的服务。
 * Zuul需要的必要依赖是spring-cloud-starter-zuul。
 */
@SpringBootApplication
@EnableZuulProxy
public class ZuulApplication {
    public static void main(String[] args) {
        SpringApplication.run(ZuulApplication.class, args);
    }
}

```

4、网关全局变量配置

4.1 URL路径匹配

```

# URL pattern
# 使用路径方式匹配路由规则。
# 参数key结构: zuul.routes.customName.path=xxx
# 用于配置路径匹配规则。
# 其中customName自定义。通常使用要调用的服务名称，方便后期管理
# 可使用的通配符有: * ** ?
# ? 单个字符
# * 任意多个字符，不包含多级路径
# ** 任意多个字符，包含多级路径
zuul.routes.eureka-application-service.path=/api/**
# 参数key结构: zuul.routes.customName.url=xxx
# url用于配置符合path的请求路径路由到的服务地址。
zuul.routes.eureka-application-service.url=http://127.0.0.1:8080/

```

4.2 服务名称匹配

```

# service id pattern 通过服务名称路由
# key结构 : zuul.routes.customName.path=xxx
# 路径匹配规则
zuul.routes.eureka-application-service.path=/api/**
# key结构 : zuul.routes.customName.serviceId=xxx
# serviceId用于配置符合path的请求路径路由到的服务名称。
zuul.routes.eureka-application-service.serviceId=eureka-application-service

```

服务名称匹配也可以使用简化的配置:

```

# simple service id pattern 简化配置方案
# 如果只配置path，不配置serviceId。则customName相当于服务名称。
# 符合path的请求路径直接路由到customName对应的服务上。
zuul.routes.eureka-application-service.path=/api/**

```

4.3 路由排除配置

```
# ignored service id pattern
# 配置不被zuul管理的服列表。多个服务名称使用逗号','分隔。
# 配置的服务将不被zuul代理。
zuul.ignored-services=eureka-application-service
# 此方式相当于给所有新发现的服务默认排除zuul网关访问方式，只有配置了路由网关的服务才可以通过zuul网关访问
# 通配方式配置排除列表。
zuul.ignored-services=*
# 使用服务名称匹配规则配置路由列表，相当于只对已配置的服务提供网关代理。
zuul.routes.eureka-application-service.path=/api/**

# 通配方式配置排除网关代理路径。所有符合ignored-patterns的请求路径都不被zuul网关代理。
zuul.ignored-patterns=/**/test/**
zuul.routes.eureka-application-service.path=/api/**
```

4.4 路由前缀配置

```
# prefix URL pattern 前缀路由匹配
# 配置请求路径前缀，所有基于此前缀的请求都由zuul网关提供代理。
zuul.prefix=/api
# 使用服务名称匹配方式配置请求路径规则。
# 这里的配置将为：http://ip:port/api/appservice/**的请求提供zuul网关代理，可以将要访问服务进行前缀分类。
# 并将请求路由到服务eureka-application-service中。
zuul.routes.eureka-application-service.path=/appservice/**
```

5 Zuul网关配置总结

网关配置方式有多种，默认、URL、服务名称、排除|忽略、前缀。

网关配置没有优劣好坏，应该在不同的情况下选择合适的配置方案。扩展：大公司为什么都有[API网关](#)？聊聊API网关的作用

zuul网关其底层使用ribbon来实现请求的路由，并内置Hystrix，可选择性提供网关fallback逻辑。使用zuul的时候，并不推荐使用Feign作为application client端的开发实现。毕竟Feign技术是对ribbon的再封装，使用Feign本身会提高通讯消耗，降低通讯效率，只在服务相互调用的时候使用Feign来简化代码开发就够了。而且商业开发中，使用Ribbon+RestTemplate来开发的比例更高。

三、Zuul网关过滤器

Zuul中提供了过滤器定义，可以用来过滤代理请求，提供额外功能逻辑。如：权限验证，日志记录等。

Zuul提供的过滤器是一个父类。父类是ZuulFilter。通过父类中定义的抽象方法filterType，来决定当前的Filter种类是什么。有前置过滤、路由后过滤、后置过滤、异常过滤。

- 前置过滤：是请求进入Zuul之后，立刻执行的过滤逻辑。
- 路由后过滤：是请求进入Zuul之后，并Zuul实现了请求路由后执行的过滤逻辑，路由后过滤，是在远程服务调用之前过滤的逻辑。
- 后置过滤：远程服务调用结束后执行的过滤逻辑。
- 异常过滤：是任意一个过滤器发生异常或远程服务调用无结果反馈的时候执行的过滤逻辑。无结果反馈，就是远程服务调用超时。

3.1 过滤器实现方式

继承父类ZuulFilter。在父类中提供了4个抽象方法，分别是：filterType, filterOrder, shouldFilter, run。其功能分别是：

filterType：方法返回字符串数据，代表当前过滤器的类型。可选值有-pre, route, post, error。

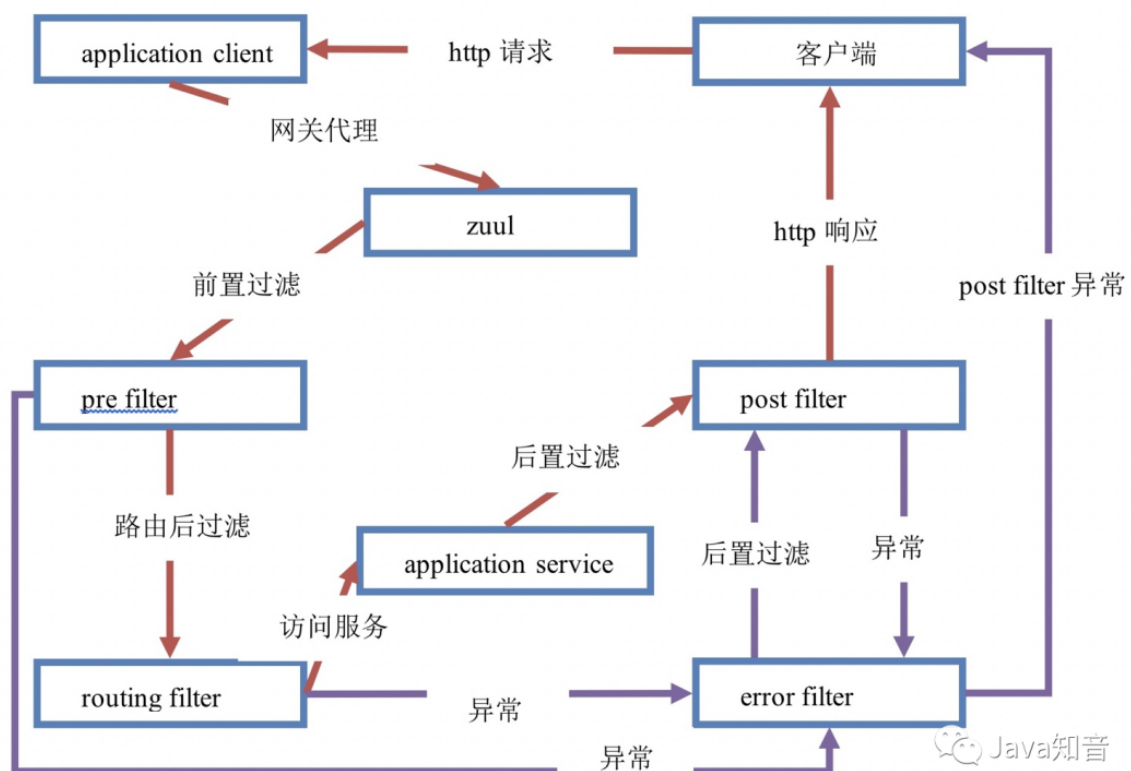
- pre - 前置过滤器，在请求被路由前执行，通常用于处理身份认证，日志记录等；
- route - 在路由执行后，服务调用前被调用；
- error - 任意一个filter发生异常的时候执行或远程服务调用没有反馈的时候执行（超时），通常用于处理异常；
- post - 在route或error执行后被调用，一般用于收集服务信息，统计服务性能指标等，也可以对response结果做特殊处理。

filterOrder：返回int数据，用于为同filterType的多个过滤器定制执行顺序，返回值越小，执行顺序越优先。

shouldFilter：返回boolean数据，代表当前filter是否生效。

run：具体的过滤执行逻辑。如pre类型的过滤器，可以通过对请求的验证来决定是否将请求路由到服务上；如post类型的过滤器，可以对服务响应结果做加工处理（如为每个响应增加footer数据）。

3.2 过滤器的生命周期



3.3 代码示例

```
/**
 * Zuul过滤器，必须继承ZuulFilter父类。
 * 当前类型的对象必须交由Spring容器管理。使用@Component注解描述。
 * 继承父类后，必须实现父类中定义的4个抽象方法。
 * shouldFilter、run、filterType、filterOrder
 */
@Component
public class LoggerFilter extends ZuulFilter {
```



```

private static final Logger logger =
LoggerFactory.getLogger(LoggerFilter.class);

/**
 * 返回boolean类型。代表当前filter是否生效。
 * 默认值为false。
 * 返回true代表开启filter。
 */
@Override
public boolean shouldFilter() {
    return true;
}

/**
 * run方法就是过滤器的具体逻辑。
 * return 可以返回任意的对象，当前实现忽略。（spring-cloud-zuul官方解释）
 * 直接返回null即可。
 */
@Override
public Object run() throws ZuulException {
    // 通过zuul，获取请求上下文
    RequestContext rc = RequestContext.getCurrentContext();
    HttpServletRequest request = rc.getRequest();

    logger.info("LogFilter1.....method={},url={}",
        request.getMethod(), request.getRequestURL().toString());
    // 可以记录日志、鉴权，给维护人员记录提供定位协助、统计性能
    return null;
}

/**
 * 过滤器的类型。可选值有：
 * pre - 前置过滤
 * route - 路由后过滤
 * error - 异常过滤
 * post - 远程服务调用后过滤
 */
@Override
public String filterType() {
    return "pre";
}

/**
 * 同种类的过滤器的执行顺序。
 * 按照返回值的自然升序执行。
 */
@Override
public int filterOrder() {
    return 0;
}
}

```

四、Zuul网关的容错（与Hystrix的无缝结合）

在spring cloud中，Zuul启动器中包含了Hystrix相关依赖，在Zuul网关工程中，默认是提供了Hystrix Dashboard服务监控数据的(hystrix.stream)，但是不会提供监控面板的界面展示。可以说，在spring cloud中，zuul和Hystrix是无缝结合的。

4.1 Zuul中的服务降级处理

在Edgware版本之前，Zuul提供了接口ZuulFallbackProvider用于实现fallback处理。从Edgware版本开始，Zuul提供了ZuulFallbackProvider的子接口FallbackProvider来提供fallback处理。

Zuul的fallback容错处理逻辑，只针对timeout异常处理，当请求被Zuul路由后，只要服务有返回（包括异常），都不会触发Zuul的fallback容错逻辑。

因为对于Zuul网关来说，做请求路由分发的时候，结果由远程服务运算的。那么远程服务反馈了异常信息，Zuul网关不会处理异常，因为无法确定这个错误是否是应用真实想要反馈给客户端的。

4.2 代码示例

```
/**
 * 如果需要在Zuul网关服务中增加容错处理fallback，需要实现接口ZuulFallbackProvider
 * spring-cloud框架，在Edgware版本(包括)之后，声明接口ZuulFallbackProvider过期失效，
 * 提供了新的ZuulFallbackProvider的子接口 - FallbackProvider
 * 在老版本中提供的ZuulFallbackProvider中，定义了两个方法。
 * - String getRoute()
 * 当前的fallback容错处理逻辑处理的是哪一个服务。可以使用通配符'*'代表为全部的服务提供容错
 处理。
 * 如果只为某一个服务提供容错，返回对应服务的spring.application.name值。
 * - ClientHttpResponse fallbackResponse()
 * 当服务发生错误的时候，如何容错。
 * 新版本中提供的FallbackProvider提供了新的方法。
 * - ClientHttpResponse fallbackResponse(Throwable cause)
 * 如果使用新版本中定义的接口来做容错处理，容错处理逻辑，只运行子接口中定义的新方法。也就是
 有参方法。
 * 是为远程服务发生异常的时候，通过异常的类型来运行不同的容错逻辑。
 */
@Component
public class TestFallbackProvider implements FallbackProvider {

    /**
     * return - 返回fallback处理哪一个服务。返回的是服务的名称
     * 推荐 - 为指定的服务定义特性化的fallback逻辑。
     * 推荐 - 提供一个处理所有服务的fallback逻辑。
     * 好处 - 服务某个服务发生超时，那么指定的fallback逻辑执行。如果有新服务上线，未提供
     fallback逻辑，有一个通用的。
     */
    @Override
    public String getRoute() {
        return "eureka-application-service";
    }

    /**
     * fallback逻辑。在早期版本中使用。
     * Edgware版本之后，ZuulFallbackProvider接口过期，提供了新的子接口FallbackProvider
     * 子接口中提供了方法ClientHttpResponse fallbackResponse(Throwable cause)。
     * 优先调用子接口新定义的fallback处理逻辑。
     */
    @Override
    public ClientHttpResponse fallbackResponse() {
        System.out.println("ClientHttpResponse fallbackResponse()");
    }
}
```

```

List<Map<String, Object>> result = new ArrayList<>();
Map<String, Object> data = new HashMap<>();
data.put("message", "服务正忙, 请稍后重试");
result.add(data);

ObjectMapper mapper = new ObjectMapper();

String msg = "";
try {
    msg = mapper.writeValueAsString(result);
} catch (JsonProcessingException e) {
    msg = "";
}

return this.executeFallback(HttpStatus.OK, msg,
    "application", "json", "utf-8");
}

/**
 * fallback逻辑。优先调用。可以根据异常类型动态决定处理方式。
 */
@Override
public ClientHttpResponse fallbackResponse(Throwable cause) {
    System.out.println("ClientHttpResponse fallbackResponse(Throwable
cause)");
    if(cause instanceof NullPointerException){

        List<Map<String, Object>> result = new ArrayList<>();
        Map<String, Object> data = new HashMap<>();
        data.put("message", "网关超时, 请稍后重试");
        result.add(data);

        ObjectMapper mapper = new ObjectMapper();

        String msg = "";
        try {
            msg = mapper.writeValueAsString(result);
        } catch (JsonProcessingException e) {
            msg = "";
        }

        return this.executeFallback(HttpStatus.GATEWAY_TIMEOUT,
            msg, "application", "json", "utf-8");
    }else{
        return this.fallbackResponse();
    }
}

/**
 * 具体处理过程。
 * @param status 容错处理后的返回状态, 如200正常GET请求结果, 201正常POST请求结果, 404
资源找不到错误等。
 * 使用spring提供的枚举类型对象实现。HttpStatus
 * @param contentMsg 自定义的响应内容。就是反馈给客户端的数据。
 * @param mediaType 响应类型, 是响应的主类型, 如: application、text、media。
 * @param subMediaType 响应类型, 是响应的子类型, 如: json、stream、html、plain、
jpeg、png等。

```

```

    * @param charsetName 响应结果的字符集。这里只传递字符集名称，如：utf-8、gbk、big5
    等。

    * @return ClientHttpResponse 就是响应的具体内容。
    * 相当于一个HttpServletResponse。
    */
    private final ClientHttpResponse executeFallback(final HttpStatus status,
        String contentMsg, String mediaType, String subMediaType, String
        charsetName) {
        return new ClientHttpResponse() {

            /**
             * 设置响应的头信息
             */
            @Override
            public HttpHeaders getHeaders() {
                HttpHeaders header = new HttpHeaders();
                MediaType mt = new MediaType(mediaType, subMediaType,
                    Charset.forName(charsetName));
                header.setContentType(mt);
                return header;
            }

            /**
             * 设置响应体
             * zuul会将本方法返回的输入流数据读取，并通过HttpServletResponse的输出流输出
            到客户端。
             */
            @Override
            public InputStream getBody() throws IOException {
                String content = contentMsg;
                return new ByteArrayInputStream(content.getBytes());
            }

            /**
             * ClientHttpResponse的fallback的状态码 返回String
             */
            @Override
            public String getStatusText() throws IOException {
                return this.getStatusCode().getReasonPhrase();
            }

            /**
             * ClientHttpResponse的fallback的状态码 返回HttpStatus
             */
            @Override
            public HttpStatus getStatusCode() throws IOException {
                return status;
            }

            /**
             * ClientHttpResponse的fallback的状态码 返回int
             */
            @Override
            public int getRawStatusCode() throws IOException {
                return this.getStatusCode().value();
            }

            /**

```

```

        * 回收资源方法
        * 用于回收当前fallback逻辑开启的资源对象的。
        * 不要关闭getBody方法返回的那个输入流对象。
        */
        @Override
        public void close() {
        }

    };
}
}

```

五、Zuul网关的限流保护

Zuul网关组件也提供了限流保护。当请求并发达到阈值，自动触发限流保护，返回错误结果。只要提供error错误处理机制即可。

Zuul的限流保护需要额外依赖spring-cloud-zuul-ratelimit组件。

```

<dependency>
  <groupId>com.marcosbarbero.cloud</groupId>
  <artifactId>spring-cloud-zuul-ratelimit</artifactId>
  <version>1.3.4.RELEASE</version>
</dependency>

```

5.1 全局限流配置

使用全局限流配置，zuul会对代理的所有服务提供限流保护。

```

# 开启限流保护
zuul.ratelimit.enabled=true
# 60s内请求超过3次，服务端就抛出异常，60s后可以恢复正常请求
zuul.ratelimit.default-policy.limit=3
zuul.ratelimit.default-policy.refresh-interval=60
# 针对IP进行限流，不影响其他IP
zuul.ratelimit.default-policy.type=origin

```

5.2 局部限流配置

使用局部限流配置，zuul仅针对配置的服务提供限流保护。

```

# 开启限流保护
zuul.ratelimit.enabled=true
# hystrix-application-client服务60s内请求超过3次，服务抛出异常。
zuul.ratelimit.policies.hystrix-application-client.limit=3
zuul.ratelimit.policies.hystrix-application-client.refresh-interval=60
# 针对IP限流。
zuul.ratelimit.policies.hystrix-application-client.type=origin

```

5.3 限流参数简介

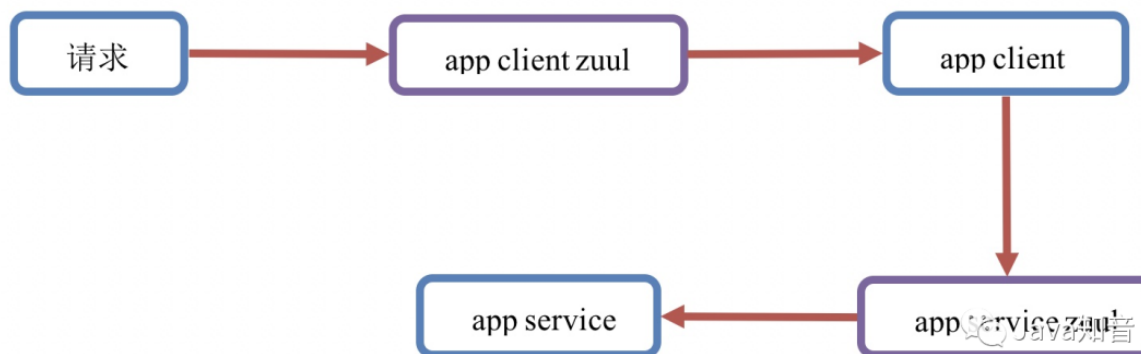
参数	默认值	备注	
zuul.ratelimit.enabled	false	是否启用限流保护	
zuul.ratelimit.repository	IN_MEMORY	限流保护的存储位置。 可选值： IN_MEMORY -内存存储，存储在 ConcurrentHashMap 集合中。 CONSUL -consul 存储 REDIS -redis 缓存存储 JPA -数据库存储 如果没有特殊要求，建议使用 IN_MEMORY，效率最高。	
zuul.ratelimit.policies.serviceId.refresh-interval	60	统计时长，单位秒。	如： refresh-interval=60,limit=10,quota=30。 代表 60 秒内，允许 10 次请求，且 10 次请求总计访问时长不超过 30 秒。
zuul.ratelimit.policies.serviceId.limit		每个统计时长内的限制请求数	
zuul.ratelimit.policies.serviceId.quota		每个统计时长内，允许访问的总时间	
zuul.ratelimit.policies.serviceId.type		限流类型。针对什么请求进行限流。 常用值： ORIGIN - 对访问 IP 限流 URL - 对访问 URL 限流	

Java知音

Java知音

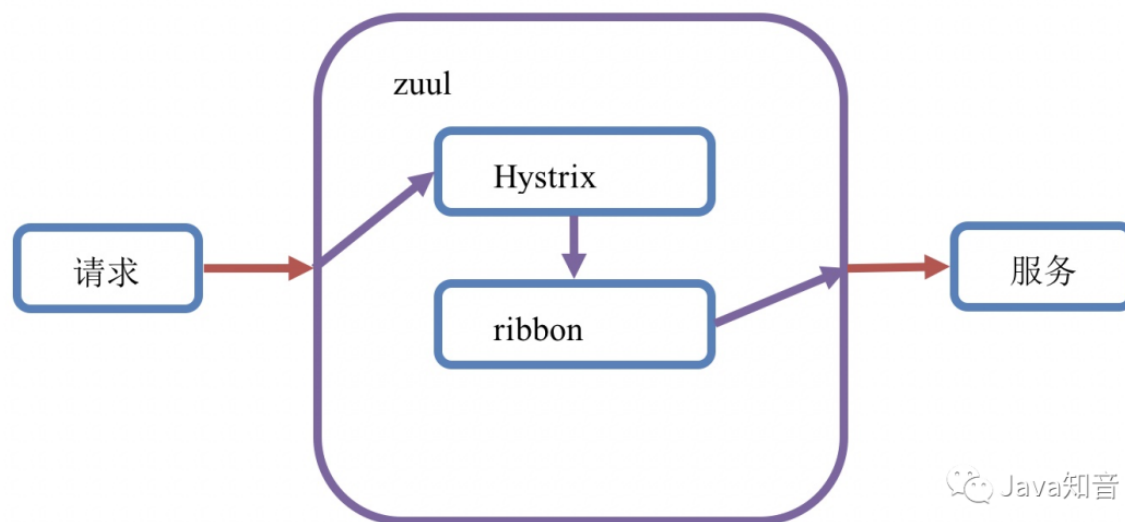
六、Zuul网关性能调优：网关的两层超时调优

使用Zuul的spring cloud微服务结构图：



从上图中可以看出。整体请求逻辑还是比较复杂的，在没有zuul网关的情况下，app client请求app service的时候，也有请求超时的可能。那么当增加了zuul网关的时候，请求超时的可能就更明显了。

当请求通过zuul网关路由到服务，并等待服务返回响应，这个过程中zuul也有超时控制。zuul的底层使用的是Hystrix+ribbon来实现请求路由。结构如下：



zuul中的Hystrix内部使用线程池隔离机制提供请求路由实现，其默认的超时时长为1000毫秒。ribbon底层默认超时时长为5000毫秒。如果Hystrix超时，直接返回超时异常。如果ribbon超时，同时Hystrix未超时，ribbon会自动进行服务集群轮询重试，直到Hystrix超时为止。如果Hystrix超时时长小于ribbon超时时长，ribbon不会进行服务集群轮询重试。

那么在zuul中可配置的超时时长就有两个位置：Hystrix和ribbon。具体配置如下：

```
# 开启zuul网关重试
zuul.retryable=true
# hystrix超时时间设置
# 线程池隔离，默认超时时间1000ms
hystrix.command.default.execution.isolation.thread.timeoutInMilliseconds=8000

# ribbon超时时间设置：建议设置比Hystrix小
# 请求连接的超时时间：默认5000ms
ribbon.ConnectTimeout=5000
# 请求处理的超时时间：默认5000ms
ribbon.ReadTimeout=5000
# 重试次数：MaxAutoRetries表示访问服务集群下原节点（同路径访问）；
# MaxAutoRetriesNextServer表示访问服务集群下其余节点（换台服务器）
ribbon.MaxAutoRetries=1
ribbon.MaxAutoRetriesNextServer=1
# 开启重试
ribbon.OkToRetryOnAllOperations=true
```

Spring-cloud中的zuul网关重试机制是使用spring-retry实现的。工程必须依赖下述资源：

```
<dependency>
  <groupId>org.springframework.retry</groupId>
  <artifactId>spring-retry</artifactId>
</dependency>
```

聊聊：什么是Spring Cloud Gateway?

Spring Cloud Gateway是Spring Cloud官方推出的第二代网关框架，取代Zuul网关。网关作为流量的控制，在微服务系统中有着非常作用，网关常见的功能有路由转发、权限校验、限流控制等作用。

它使用了一个RouteLocatorBuilder的bean去创建路由，除了创建路由，RouteLocatorBuilder也可以让你添加各种predicates和filters，predicates断言的意思，顾名思义就是根据具体的请求的规则，由具体的route去处理，filters是各种过滤器，用来对请求做各种判断和修改。

Spring Cloud 高级 试题

1. 聊聊：微服务之间是如何独立通讯的

1.同步进行远程过程调用（Remote Procedure Invocation）：

也就是我们常说的服务的注册与发现

直接通过远程过程调用来访问别的service。

优点：

简单，常见,因为没有中间件代理，系统更简单

缺点：

只支持请求/响应的模式，不支持别的，比如通知、请求/异步响应、发布/订阅、发布/异步响应

降低了可用性，因为客户端和服务端在请求过程中必须都是可用的

2.异步消息通讯：

使用异步消息来做服务间通信。服务间通过消息管道来交换消息，从而通信。

优点：

把客户端和服务端解耦，更松耦合

提高可用性，因为消息中间件缓存了消息，直到消费者可以消费

支持很多通信机制比如通知、请求/异步响应、发布/订阅、发布/异步响应

缺点：

消息中间件有额外的复杂

聊聊：微服务之间是如何独立通讯的（同类型题）

微服务通信机制

系统中的各个微服务可被独立部署，各个微服务之间是松耦合的。每个微服务仅关注于完成一件任务并很好地完成该任务。

围绕业务能力组织服务、自动化部署、智能端点、对语言及数据的去集中化控制。

- 将组件定义为可被独立替换和升级的软件单元。
- 以业务能力为出发点组织服务的策略。
- 倡导谁开发，谁运营的开发运维一体化方法。
- RESTful HTTP协议是微服务架构中最常用的通讯机制。
- 每个微服务可以考虑选用最佳工具完成(如不同的编程语言)。

- 允许不同微服务采用不同的数据持久化技术。
- 微服务非常重视建立架构及业务相关指标的实时监控和日志机制，必须考虑每个服务的失败容错机制。
- 注重快速更新，因此系统会随时间不断变化及演进。可替代性模块化设计。

微服务通信方式：

同步：RPC，REST等

异步：消息队列。要考虑消息可靠传输、高性能，以及编程模型的变化等。

消息队列中间件如何选型

- 1.协议：AMQP、STOMP、MQTT、私有协议等。
- 2.消息是否需要持久化。
- 3.吞吐量。
- 4.高可用支持，是否单点。
- 5.分布式扩展能力。
- 6.消息堆积能力和重放能力。
- 7.开发便捷，易于维护。
- 8.社区成熟度。

RabbitMQ是一个实现了AMQP(高级消息队列协议)协议的消息队列中间件。RabbitMQ支持其中的最多一次和最少一次两种。网易蜂巢平台的服务[架构](#)，服务间通过RabbitMQ实现通信。

2.聊聊：微服务的优缺点分别是什么?说下你在项目开发中碰到的坑

优点

- 每一个服务足够内聚,代码容易理解
- 开发效率提高,一个服务只做一件事
- 微服务能够被小团队单独开发
- 微服务是松耦合的,是有功能意义的服务
- 可以用不同的语言开发,面向接口编程
- 易于与第三方集成
- 微服务只是业务逻辑的代码,不会和HTML,CSS或者其他界面组合

开发中,两种开发模式
前后端分离
全栈工程师

- 可以灵活搭配,连接公共库/连接独立库

缺点

- 分布式系统的负责性
- 多服务运维难度,随着服务的增加,运维的压力也在增大
- 系统部署依赖
- 服务间通信成本
- 数据一致性
- 系统集成测试
- 性能监控

3 聊聊：Eureka和ZooKeeper都可以提供服务注册与发现的功能,请说说两个的区别

1.ZooKeeper保证的是CP,Eureka保证的是AP

ZooKeeper在选举期间注册服务瘫痪,虽然服务最终会恢复,但是选举期间不可用的

Eureka各个节点是平等关系,只要有一台Eureka就可以保证服务可用,而查询到的数据并不是最新的
自我保护机制会导致

Eureka不再从注册列表移除因长时间没收到心跳而应该过期的服务

Eureka仍然能够接受新服务的注册和查询请求,但是不会被同步到其他节点(高可用)

当网络稳定时,当前实例新的注册信息会被同步到其他节点中(最终一致性)

Eureka可以很好的应对因网络故障导致部分节点失去联系的情况,而不会像ZooKeeper一样使得整个注册系统瘫痪

2.ZooKeeper有Leader和Follower角色,Eureka各个节点平等

3.ZooKeeper采用过半数存活原则,Eureka采用自我保护机制解决分区问题

4.Eureka本质上是一个工程,而ZooKeeper只是一个进程

问：Eureka和ZooKeeper都可以提供服务注册与发现的功能,请说说两个的区别（同类型的题目）

1.ZooKeeper保证的是CP,Eureka保证的是AP

ZooKeeper在选举期间注册服务瘫痪,虽然服务最终会恢复,但是选举期间不可用的

Eureka各个节点是平等关系,只要有一台Eureka就可以保证服务可用,而查询到的数据并不是最新的

自我保护机制会导致

Eureka不再从注册列表移除因长时间没收到心跳而应该过期的服务

Eureka仍然能够接受新服务的注册和查询请求,但是不会被同步到其他节点(高可用)

当网络稳定时,当前实例新的注册信息会被同步到其他节点中(最终一致性)

Eureka可以很好的应对因网络故障导致部分节点失去联系的情况,而不会像ZooKeeper一样使得整个注册系统瘫痪

2.ZooKeeper有Leader和Follower角色,Eureka各个节点平等3.ZooKeeper采用过半数存活原则,Eureka采用自我保护机制解决分区问题4.Eureka本质上是一个工程,而ZooKeeper只是一个进程

4. 聊聊：eureka自我保护机制是什么？

当Eureka Server 节点在短时间内丢失了过多实例的连接时（比如网络故障或频繁启动关闭客户端）节点会进入自我保护模式，保护注册信息，不再删除注册数据，故障恢复时，自动退出自我保护模式。

5: 聊聊: 什么是Netflix Feign? 它的优点是什么?

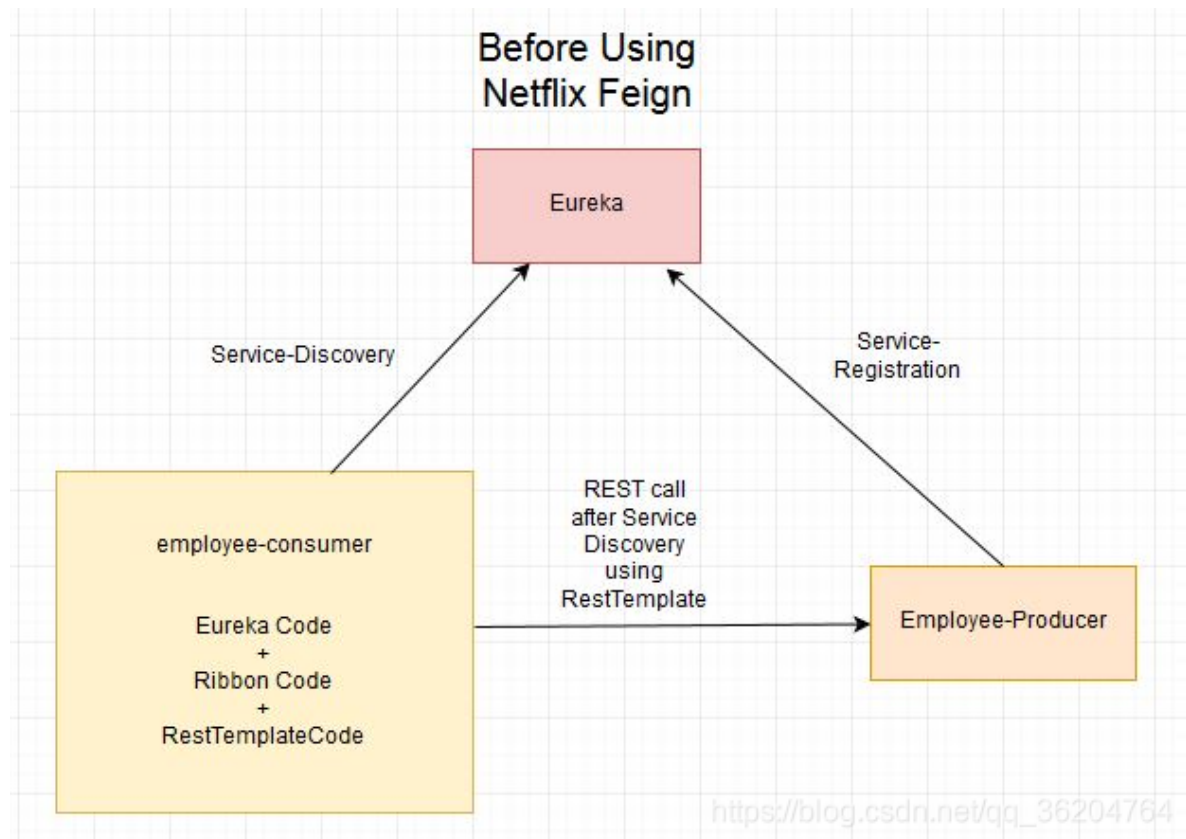
答:

Feign是一个受到Retrofit, JAXRS-2.0和WebSocket启发的java到http客户端绑定器。

Feign的第一个目标是降低将Denominator统一绑定到http apis的复杂性, 无论其是否安宁。

员工 - 消费者中的先前示例我们使用**REST模板**

使用员工生产者公开的REST服务



但是我们必须编写大量代码来执行以下操作 -

- 使用功能区进行负载均衡。
- 获取服务实例, 然后获取基本URL。
- 使用REST模板来消费服务

聊聊: Feign的底层工作原理

Feign是一个声明式的伪Http客户端, 它使得写Http客户端变得更简单。

使用Feign, 只需要创建一个接口并注解。

Feign是一个HTTP请求调用的轻量级框架, 可以以JAVA接口注解的方式调用HTTP请求, 而不用像Java中通过封装HTTP请求报文的方式直接调用。

Feign通过处理注解, 将请求模板化, 当实际调用的时候, 传入参数, 根据参数再应用到请求上, 进而转化成真正的请求。

它具有可插拔的注解特性, 可使用Feign 注解和JAX-RS注解。Feign支持可插拔的编码器和解码器。

Feign默认集成了Ribbon进行负载均衡

1. 封装了HTTP调用流程，更适合面向接口化开发
2. 代码少，使用简单
3. 几乎完全可以从服务提供方的Controller中依靠复制操作，来构建出相应的服务接口客户端，或是通过Swagger生成的API文档来编写出客户端，亦或是通过Swagger的代码生成器来生成客户端绑定（复制粘贴党的福音）

6、聊聊：Eureka的原理和工作机制

- 概念

Eureka负责管理、记录服务提供者的信息，服务调用者把自己的需求告诉Eureka，然后 Eureka会把符合你需求的服务告诉你，它实现了服务的自动注册、发现、状态监控。

简单来说，就是服务提供者将服务放到Eureka里，Eureka再把相应的服务（也就是服务调用者需要的服务）给服务调用者

Eureka：服务注册中心（可以是一个集群），对外暴露自己的地址提供者：启动后向Eureka注册自己信息（地址，提供什么服务）

消费者：向Eureka订阅服务，Eureka会将对应服务的所有提供者地址列表发送给消费者，并且定期更新

心跳(续约)：提供者定期通过http方式向Eureka刷新自己的状态

- 工作流程

服务启动后向Eureka注册，Eureka Server会将注册信息向其他Eureka Server进行同步，当服务消费者要调用服务提供者，则向服务注册中心获取服务提供者地址，然后会将服务提供者地址缓存在本地，下次再调用时，则直接从本地缓存中取，完成一次调用

1. 服务提供者

服务提供者要向EurekaServer注册服务，并且完成服务续约等工作。

- 服务注册

服务提供者在启动时，会检测配置属性中的eureka.client.register-with-erueka，若它为ture，代表将自己的信息注册到EurekaServer。

则会向EurekaServer 发起一个Rest请求，并携带自己的元数据信息，Eureka Server会把这些信息保存到一个双层Map结构中。第一层Map的Key就是服务名称，第二层Map的key是服务的实例id

- 服务续约(心跳机制)

在注册服务完成以后，服务提供者会维持一个心跳（定时向EurekaServer发起Rest 请求），告诉EurekaServer：“我还活着”。这个我们称为服务的续约（renew），client 心跳线程http调用server的renew，增加心跳次数，修改注册表中对应InstanceInfo的最近心跳时间

2. 服务消费者

当服务消费者启动时，会检测eureka.client.fetch-registry参数的值，如果为true，代表拉取其它服务的信息，则会从Eureka Server服务的列表只读备份，然后缓存在本地。默认是每隔30秒会重新获取并更新数据。也可以自己设置时间

3. 失效剔除

有些时候，我们的服务提供方并不一定会正常下线，可能因为内存溢出、网络故障等原因导致服务无法正常工作。Eureka Server需要将这样的服务剔除出服务列表。因此它会开启一个定时任务，每隔60秒对所有失效的服务（超过90秒未响应）进行剔除。

可以通过eureka.server.eviction-interval-timer-in-ms参数对其进行修改，单位是毫秒

4. 自我保护机制

当一个服务未按时进行心跳续约时，Eureka会统计最近15分钟心跳失败的服务实例的比例是否超过了85%。在生产环境下，因为网络延迟等原因，心跳失败实例的比例很有可能超标，但是此时就把服务剔除列表并不妥当，因为服务可能没有宕机。Eureka就会把当前实例的注册信息保护起

来，不予剔除。生产环境下这很有效，保证了大多数服务依然可用。
自我保护模式可以自己设置，可设为关闭

```
eureka: server:
enable-self-preservation: false # 关闭自我保护模式（缺省为打开）
eviction-interval-timer-in-ms: 1000 # 扫描失效服务的间隔时间（缺省为
60*1000ms）
```

5. 多级缓存机制

在eureka client拉取注册表的时候，就会用到所谓的多级缓存机制，多级缓存机制中有两个缓存，一个叫只读缓存 `ReadOnlyCacheMap`，一个叫读写缓存 `ReadWriteCacheMap`。

eureka client 拉取注册表的时候，会先从 `ReadOnlyCacheMap` 中去获取注册表数据，如果获取不到的话再去 `ReadWriteCacheMap` 中找，如果还是找不到的话，那就只能重新从注册表中 `registry` 拉取了

`ReadOnlyCacheMap`就是一个普通的`ConcurrentHashMap`，而`ReadWriteCacheMap`是guava cache，如果`ReadWriteCacheMap`读不到数据，就会通过`ClassLoader`的 `load`方法直接从注册表获取数据再返回

多级缓存机制有多种过期策略：

- 主动过期：当服务实例发生注册、下线、故障的时候，`ReadWriteCacheMap`中所有的缓存过期掉
 - 定时过期：`readWriteCacheMap`在构建的时候，指定了一个自动过期的时间，默认值就是180秒，所以你往`readWriteCacheMap`中放入一个数据，180秒过后，就将这个数据给他过期了
 - 被动过期：默认是每隔30秒，执行一个定时调度的线程任务，对`readOnlyCacheMap`和`readWriteCacheMap`中的数据进行一个比对，如果两块数据是不一致的，那么就将`readWriteCacheMap`中的数据放到`readOnlyCacheMap`中来
- 通过过期的机制，可以发现一个问题，就是如果`ReadWriteCacheMap`发生了主动过期或定时过期，此时里面的缓存就被清空或部分被过期了，但是在此之前`readOnlyCacheMap`刚执行了被动过期，发现两个缓存是一致的，就会接着使用里面的缓存数据
- 所以可能会存在30秒的时间，`readOnlyCacheMap`和`ReadWriteCacheMap`的数据不一致
- Eureka和zookeeper的区别
- CAP理论指出，一个分布式系统不可能同时满足C(一致性)、A(可用性)和P(分区容错性)。由于分区容错性P是在分布式系统中必须要保证的，因此我们只能在A和C之间进行权衡。

- Zookeeper保证CP

Zookeeper 为主从结构，有leader节点和follow节点。当leader节点down掉之后，剩余节点会重新进行选举。选举过程中会导致服务不可用，丢掉了可用行

- Eureka保证AP

Eureka各个节点都是平等的，几个节点挂掉不会影响正常节点的工作，剩余的节点依然可以提供注册和查询服务。而Eureka的客户端在向某个Eureka注册或时如果发现连接失败，则会自动切换至其它节点，只要有一台Eureka还在，就能保证注册服务可用(保证可用性)，只不过查到的信息可能不是最新的(不保证强一致性)

7. 聊聊：网关Gateway的工作原理

Spring Cloud Gateway是Spring官方基于Spring5.0，Spring Boot2.0和Project Reactor等技术开发的网关，

Spring Cloud Gateway旨在为微服务架构提供简单，有效且统一的API路由管理方式。Spring Cloud Gateway 作

为Spring Cloud 生态系统中的网关，目标是替代Netflix Zuul，其不仅提供统一的路由方式，并且还基于Filter链的方式提供了网关基本的功能，例如：安全，监控、埋点，限流等。

相比阻塞I/O的zuul，gateway使用了webflux中的reactor-netty响应式编程组件，底层使用了netty通讯框架

Spring Cloud Gateway 的核心处理流程：

- Gateway 的客户端回向 Spring Cloud Gateway 发起请求，请求首先会被 `HttpWebHandlerAdapter` 进行提取组装成网关的上下文，然后网关的上下文会传递到 `DispatcherHandler`
- `DispatcherHandler` 是所有请求的分发处理器，`DispatcherHandler` 主要负责分发请求对应的处理器，比如将请求分发到对应 `RoutePredicateHandlerMapping` (路由断言处理器映射器)
- 路由断言处理映射器主要用于路由的查找，以及找到路由后返回对应的 `FilteringWebHandler`
- `FilteringWebHandler` 主要负责组装 `Filter` 链表并调用 `Filter` 执行一系列 `Filter` 处理，然后把请求转到后端对应的代理服务处理，处理完毕后，将 `Response` 返回到 Gateway 客户端。

在 `Filter` 链中，过滤器可以在转发请求之前处理或者接收到被代理服务的返回结果之后处理。所有的 `Pre` 类型的 `Filter` 执行完毕之后，才会转发请求到被代理的服务处理。被代理的服务把所有请求完毕之后，才会执行 `Post` 类型的过滤器

8. 聊聊：Feign工作原理

Feign是一个声明式的伪Http客户端，它使得写Http客户端变得更简单。使用Feign，只需要创建一个接口并注解。

Feign是一个HTTP请求调用的轻量级框架，可以以JAVA接口注解的方式调用HTTP请求，而不用像Java中通过封装HTTP请求报文的方式直接调用。Feign通过处理注解，将请求模板化，当实际调用的时候，传入参数，根据参数再应用到请求上，进而转化成真正的请求。

它具有可插拔的注解特性，可使用Feign注解和JAX-RS注解。Feign支持可插拔的编码器和解码器。

Feign默认集成了Ribbon

1. 封装了HTTP调用流程，更适合面向接口化开发
2. 代码少，使用简单
3. 几乎完全可以从服务提供方的Controller中依靠复制操作，来构建出相应的服务接口客户端，或是通过Swagger生成的API文档来编写出客户端，亦或是通过Swagger的代码生成器来生成客户端绑定（复制粘贴党的福音）

9. 聊聊：Ribbon工作原理

- 概述：
SpringCloud Ribbon是一个基于HTTP和TCP的客户端的负载均衡工具，Ribbon和微服务是同级别的，融合到微服务的一些基础设施（如Feign），不需要独立部署。
- Ribbon会将微服务之间的Rest请求转为客户端的负载均衡的RPC调用
- Ribbon默认的负载均衡策略是轮询，但不止轮询一种，可以自定义配置
- 工作流程
 1. 微服务之间通过Feign调用，最后通过LoadBalancerFeignClient发送请求
 2. LoadBalancerFeignClient端从client端服务的上下文环境中找到负载均衡器，并把提取到的服务名称交给负载均衡器
 3. 负载均衡器提到选到server实例，将client端的请求包装成调用请求LoadBalancerCommand
 4. 根据封装的信息，发送远程调用到具体的服务实例
 5. 和Feign的集成模式：
在使用Feign作为客户端时，最终请求会转发成 `http://<服务名称>/` 的格式，通过LoadBalancerFeignClient，提取出服务标识 `<服务名称>`，然后根据服务名称在上下文中查

找对应服务的负载均衡器FeignLoadBalancer，负载均衡器负责根据既有的服务实例的统计信息，挑选出最合适的服务实例

10. 聊聊：Hystrix的工作原理

- 容错限流的需求

在复杂的分布式系统中通常有很多依赖，如果一个应用不能对来自依赖故障进行隔离，那么应用本身就处于被拖垮的风险中。在一个高流量的网站中，某一个单一后端一旦发生延迟，将会在数秒内导致所有的应用资源被耗尽，这也就是我们常说的雪崩效应。

比如在电商系统的下单业务中，在订单服务创建订单后同步调用库存服务进行库存的扣减，假如库存服务出现了故障，那么会导致下单请求线程会被阻塞，当有大量的下单请求时，则会占满应用连接数从而导致订单服务无法对外提供服务。

- 容错限流的原理

对于基本的容错限流模式，主要有以下几点需要考量：

- 主动超时：在调用依赖时尽快的超时，可以设置比较短的超时时间，比如2s，防止长时间的等待
- 限流：限制最大并发数
- 熔断：错误数达到阈值时，类似于保险丝熔断隔离：隔离不同的依赖调用
- 服务降级：资源不足时进行服务降级

- 容错模式

- 断路器模式

实现流程为：当断路器的开关为关闭时，每次请求进来都是成功的，当后端服务出现问题，请求出现的错误数达到一定的阈值，则会触发断路器为打开状态；在断路器为打开状态时，进来的所有请求都会被拒绝，当然也不是一直会拒绝请求，而是弹性的，过了特定的时间后，断路器会进入半打开状态，这是会让一部分请求通过进行尝试，如果尝试还是有问题，则继续进入打开状态，如果尝试没有问题了，则会进入关闭状态

- 舱壁隔离模式

舱壁隔离模式可以对资源进行隔离，类似于船的船舱都是被隔离开来的，当其中一个或者几个船舱出现问题，比如漏水，是不会影响到其他的船舱的，从而实现一种资源隔离的效果

- 容错理念

- 凡是依赖都有可能会失败
- 凡是资源都有限制，比如CPU、Memory、Threads、Queue
- 网络并不可靠，可能存在网络抖动等其他问题
- 延迟是应用稳定的杀手，延迟会占据大量的资源

- Hystrix工作流程

1. 对于一次依赖调用，会被封装在一个HystrixCommand对象中，调用的执行有两种方式，一种是调用execute()方法同步调用，另一种是调用queue()方法进行异步调用。
2. 执行时会判断断路器开关是否打开，如果断路器打开，则进入getFallback()降级逻辑；如果断路器关闭，则判断线程池/信号量资源是否已满，如果资源满了，则进入 getFallback()降级逻辑；如果没满，则执行run()方法。再判断执行run()方法是否超时，超时则进入getFallback()降级逻辑，run()方法执行失败，则进入getFallback()降级逻辑，执行成功则报告Metrics。Metrics中的数据包括执行成功、超时、失败等情况的数据，Hystrix会计算一个断路器的健康值，也就是失败率，当失败率超过阈值后则会触发断路器开关打开。
3. getFallback()逻辑为：如果没有实现fallback()方法，则直接抛出异常，另外fallback降级也是需要资源的，在fallback时需要获取一个针对fallback的信号量，只有获取成功才能

fallback, 获取信号量失败, 则抛出异常, 获取信号量成功, 才会执行fallback方法并且会响应fallback方法中的内容

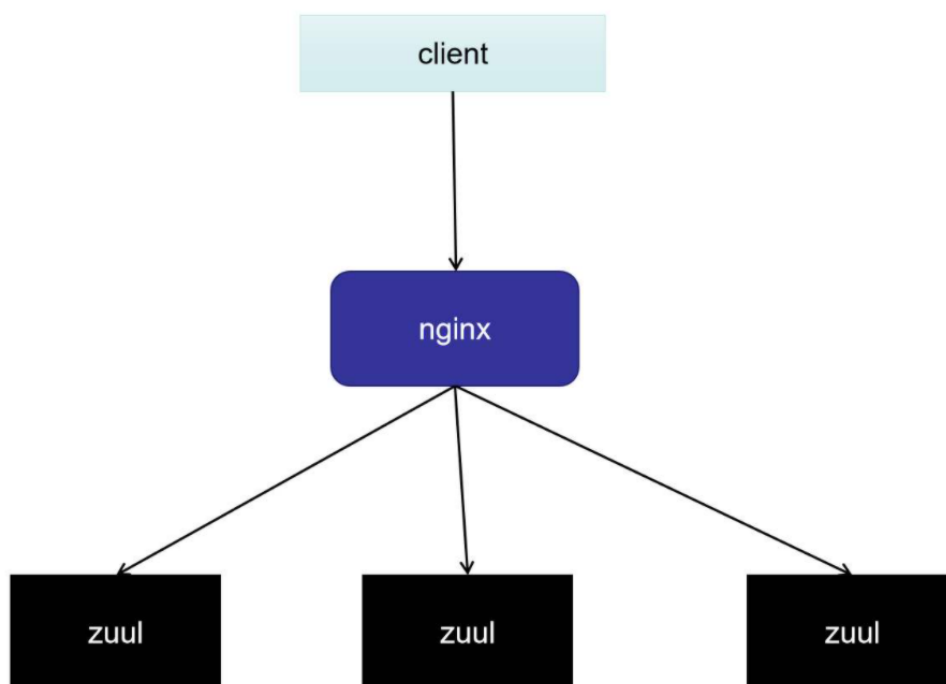
13、聊聊： Nginx 和 Zuul 的区别和共同点

Nginx 和 Zuul 的区别

- 1、Nginx是C语言开发,而 Zuul 是Java语言开发
- 2、其次,Nginx负载均衡实现,采用服务器实现负载均衡,而Zuul负载均衡的实现是采用 Ribbon + Eureka 来实现本地负载均衡.
- 3、Nginx适合于服务器端负载均衡,Zuul适合微服务中实现网关
- 4、Nginx相比Zuul功能会更加强大,因为Nginx整合一些脚本语言(Nginx + lua)
- 5、Nginx 是一个高性能的HTTP 和反向代理服务器, 也是一个 IMAP / POP3 /SMTP 服务器. Zuul是 Spring Cloud Netflix 中的开源的一个API Gateway 服务器,本质上是一个web servlet 应用, 提供动态路由,监控,弹性,安全等边缘服务的框架. Zuul 相当于是设备和Netflix 流应用的Web 网站后端所有请求的前门

Nginx 和 Zuul 的共同点

- 1、可以实现负载均衡 (Zuul使用的是Ribbon实现负载均衡)
- 2、可以实现反向代理 (即隐藏真实ip地址)
- 3、可以过滤请求,实现网关的效果



央企真题：Feign Ribbon Hystrix 三者关系（重点题目）

特别注意此题：在小伙伴面试 央企的时候，问到了，

可惜：他没有回答上了哈。

Feign介绍

Feign是一款Java语言编写的HttpClient绑定器，在Spring Cloud微服务中用于实现微服务之间的声明式调用。Feign 可以定义请求到其他服务的接口，用于微服务间的调用，不用自己再写http请求，在客户端实现，调用此接口就像远程调用其他服务一样，当请求出错时可以调用接口的实现类来返回

Feign是一个声明式的web service客户端，它使得编写web service客户端更为容易。创建接口，为接口添加注解，即可使用Feign。Feign可以使用Feign注解或者JAX-RS注解，还支持热插拔的编码器和解码器。Spring Cloud为Feign添加了Spring MVC的注解支持，并整合了Ribbon和Eureka来为使用Feign时提供负载均衡。

feign源码的github地址：

<https://github.com/OpenFeign/feign>

Ribbon介绍

Ribbon 作为负载均衡，在客户端实现，服务端可以启动两个端口不同但servername一样的服务

Ribbon是Netflix发布的开源项目，主要功能是提供客户端的软件负载均衡算法，将Netflix的中间层服务连接在一起。Ribbon客户端组件提供一系列完善的配置项如连接超时，重试等。简单的说，就是在配置文件中列出Load Balancer后面所有的机器，Ribbon会自动的帮助你基于某种规则（如简单轮询，随机连接等）去连接这些机器。我们也很容易使用Ribbon实现自定义的负载均衡算法。简单地说，Ribbon是一个客户端负载均衡器。

Ribbon工作时分为两步：第一步先选择 Eureka Server, 它优先选择在同一个Zone且负载较少的Server；第二步再根据用户指定的策略，在从Server取到的服务注册列表选择一个地址。其中Ribbon提供了多种策略，例如轮询、随机、根据响应时间加权等。

ribbon源码的github地址：

<https://github.com/Netflix/ribbon>

Hystrix介绍

Hystrix作为熔断流量控制，在客户端实现，在方法上注解，当请求出错时可以调用注解中的方法返回

Hystrix熔断器，容错管理工具，旨在通过熔断机制控制服务和第三方库的节点,从而对延迟和故障提供更强大的容错能力。在Spring Cloud Hystrix中实现了线程隔离、断路器等一系列的服务保护功能。它也是基于Netflix的开源框架 Hystrix实现的，该框架目标在于通过控制那些访问远程系统、服务和第三方库的节点，从而对延迟和故障提供更强大的容错能力。Hystrix具备了服务降级、服务熔断、线程隔离、请求缓存、请求合并以及服务监控等强大功能。

Hystrix源码的github地址：

<https://github.com/Netflix/hystrix>

重点：三者之间的关系图

如果微服务项目加上了spring-cloud-starter-netflix-hystrix依赖，那么，feign会通过代理模式，自动将所有的方法用 hystrix 进行包装。

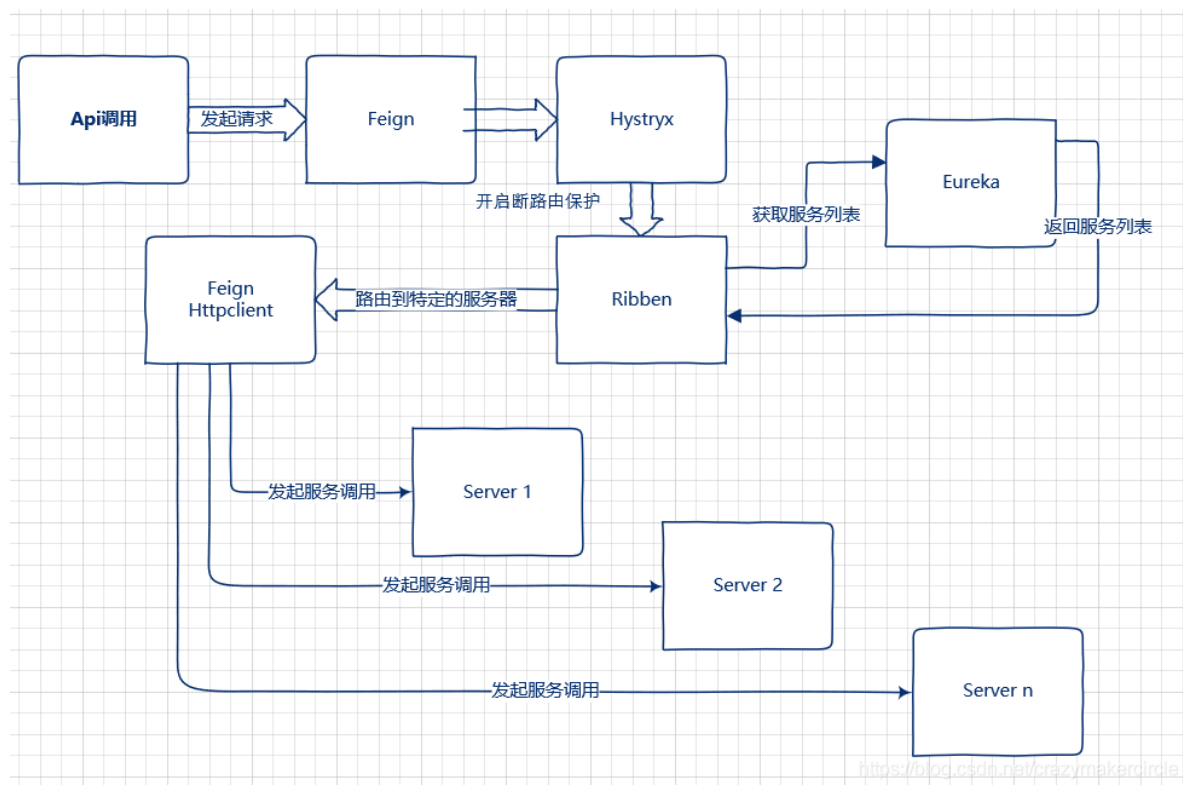
在Spring Cloud微服务体系下，微服务之间的互相调用可以通过Feign进行声明式调用，在这个服务调用过程中Feign会通过Ribbon从服务注册中心获取目标微服务的服务器地址列表，之后在网络请求的过程中Ribbon就会将请求以负载均衡的方式打到微服务的不同实例上，从而实现Spring Cloud微服务架构中最为关键的功能即服务发现及客户端负载均衡调用。

另一方面微服务在互相调用的过程中，为了防止某个微服务的故障消耗掉整个系统所有微服务的连接资源，所以在实施微服务调用的过程中我们会要求在调用方实施针对被调用微服务的熔断逻辑。

而要实现这个逻辑场景在Spring Cloud微服务框架下我们是通过Hystrix这个框架来实现的。

调用方会针对被调用微服务设置调用超时时间，一旦超时就会进入熔断逻辑，而这个故障指标信息也会返回给Hystrix组件，Hystrix组件会根据熔断情况判断被调微服务的故障情况从而打开熔断器，之后所有针对该微服务的请求就会直接进入熔断逻辑，直到被调微服务故障恢复，Hystrix断路器关闭为止。

三者之间的关系图，大致如下：



Feign典型配置说明

Feign自身可以支持多种HttpClient工具包，例如OkHttp及Apache HttpClient，针对Apache HttpClient的典型配置如下：

```

feign:
  #替换掉JDK默认URLConnection实现的 Http Client
  httpclient:
    enabled: true
  hystrix:
    enabled: true
  client:
    config:
      default:
        #连接超时时间
        connectTimeout: 5000
        #读取超时时间
        readTimeout: 5000

```

Hystrix配置说明

在Spring Cloud微服务体系中Hystrix主要被用于实现实现微服务之间网络调用故障的熔断、过载保护及资源隔离等功能。

```

hystrix:
  propagate:
    request-attribute:
      enabled: true
  command:
    #全局默认配置
    default:
      #线程隔离相关
      execution:
        timeout:
          #是否给方法执行设置超时时间，默认为true。一般我们不要改。
          enabled: true
      isolation:
        #配置请求隔离的方式，这里是默认的线程池方式。还有一种信号量的方式semaphore，使用比
        较少。
        strategy: threadPool
        thread:
          #方式执行的超时时间，默认为1000毫秒，在实际场景中需要根据情况设置
          timeoutInMilliseconds: 10000
          #发生超时时是否中断方法的执行，默认值为true。不要改。
          interruptOnTimeout: true
          #是否在方法执行被取消时中断方法，默认值为false。没有实际意义，默认就好！
          interruptOnCancel: false
      circuitBreaker: #熔断器相关配置
        enabled: true #是否启动熔断器，默认为true，false表示不要引入Hystrix。
        requestVolumeThreshold: 20 #启用熔断器功能窗口时间内的最小请求数，假设我们设置的
        窗口时间为10秒，
        sleepWindowInMilliseconds: 5000 #所以此配置的作用是指定熔断器打开后多长时间内允许
        一次请求尝试执行，官方默认配置为5秒。
        errorThresholdPercentage: 50 #窗口时间内超过50%的请求失败后就会打开熔断器将后续请
        求快速失败掉，默认配置为50

```

Ribbon配置说明

Ribbon在Spring Cloud中对于支持微服务之间的通信发挥着非常关键的作用，其主要功能包括客户端负载均衡器及用于中间层通信的客户端。在基于Feign的微服务通信中无论是否开启Hystrix，Ribbon都是必不可少的，Ribbon的配置参数主要如下：

```
ribbon:
  eager-load:
    enabled: true
  #说明：同一台实例的最大自动重试次数，默认为1次，不包括首次
  MaxAutoRetries: 1
  #说明：要重试的下一个实例的最大数量，默认为1，不包括第一次被调用的实例
  MaxAutoRetriesNextServer: 1
  #说明：是否所有的操作都重试，默认为true
  OkToRetryOnAllOperations: true
  #说明：从注册中心刷新服务器列表信息的时间间隔，默认为2000毫秒，即2秒
  ServerListRefreshInterval: 2000
  #说明：使用Apache HttpClient连接超时时间，单位为毫秒
  ConnectTimeout: 3000
  #说明：使用Apache HttpClient读取的超时时间，单位为毫秒
  ReadTimeout: 3000
```

如上图所示，在Spring Cloud中使用Feign进行微服务调用分为两层：Hystrix的调用和Ribbon的调用，Feign自身的配置会被覆盖。

而如果开启了Hystrix，那么Ribbon的超时时间配置与Hystrix的超时时间配置则存在依赖关系，因为涉及到Ribbon的重试机制，所以一般情况下都是Ribbon的超时时间小于Hystrix的超时时间，否则会出现以下错误：

```
2019-10-12 21:56:20,208 111231 [http-nio-8084-exec-2] WARN
o.s.c.n.z.f.r.s.AbstractRibbonCommand - The Hystrix timeout of 10000ms for the
command operation is set lower than the combination of the Ribbon read and
connect timeout, 24000ms.
```

Ribbon和Hystrix的超时时间配置的关系

那么Ribbon和Hystrix的超时时间配置的关系具体是什么呢？如下：

```
Hystrix的超时时间=Ribbon的重试次数(包含首次) * (ribbon.ReadTimeout +
ribbon.ConnectTimeout)
```

而Ribbon的重试次数的计算方式为：

```
Ribbon重试次数(包含首次)= 1 + ribbon.MaxAutoRetries +
ribbon.MaxAutoRetriesNextServer + (ribbon.MaxAutoRetries *
ribbon.MaxAutoRetriesNextServer)
```

以上图中的Ribbon配置为例子，Ribbon的重试次数=1+(1+1+1)=4，所以Hystrix的超时配置应该 $\geq 4 * (3000+3000)=24000$ 毫秒。在Ribbon超时但Hystrix没有超时的情况下，Ribbon便会采取重试机制；而重试期间如果时间超过了Hystrix的超时配置则会立即被熔断（fallback）。

如果不配置Ribbon的重试次数，则Ribbon默认会重试一次，加上第一次调用Ribbon，总的重试次数为2次，以上述配置参数为例，Hystrix超时时间配置为 $2 \times 6000 = 12000$ ，由于很多情况下，大家一般不会主动配置Ribbon的重试次数，所以这里需要注意下！强调下，以上超时配置的值只是示范，超时配置有点大不太合适实际的线上场景，大家根据实际情况设置即可！

说明下，如果不启用Hystrix，Feign的超时时间则是Ribbon的超时时间，Feign自身的配置也会被覆盖。

聊聊：OpenFeign 进行RPC调优的几个方面？

OpenFeign 是 Spring 官方推出的一种声明式服务调用和负载均衡组件。

它的出现就是为了替代已经进入停更维护状态的 Feign（Netflix Feign），同时它也是 Spring 官方的顶级开源项目。

我们在日常的开发中使用它的频率也很高，而 OpenFeign 有一些实用的小技巧，配置之后可以让 OpenFeign 更好的运行。

1.超时优化

OpenFeign 底层内置了 Ribbon 框架，并且使用了 Ribbon 的请求连接超时时间和请求处理超时时间作为其超时时间，而 Ribbon 默认的请求连接超时时间和请求处理超时时间都是 1s，如下源码所示：

```
@EnableConfigurationProperties
@Import({HttpClientConfiguration.class, OkHttpRibbonConfiguration.class,
public class RibbonClientConfiguration {
    public static final int DEFAULT_CONNECT_TIMEOUT = 1000;
    public static final int DEFAULT_READ_TIMEOUT = 1000;
    public static final boolean DEFAULT_GZIP_PAYLOAD = true;
    @RibbonClientName
    private String name = "client";
    @Autowired
    private PropertiesFactory propertiesFactory;
```

请求连接超时时间

请求处理超时时间

所有当我们使用 OpenFeign 调用了服务接口超过 1s，就会出现以下错误：

```
Run: OpenFeignConsumerApplication
Console
Endpoints
java.net.SocketTimeoutException: Read timed out
    at java.net.SocketInputStream.socketRead0(Native Method) ~[na:1.8.0_251]
    at java.net.SocketInputStream.socketRead(SocketInputStream.java:116) ~[na:1.8.0_251]
    at java.net.SocketInputStream.read(SocketInputStream.java:171) ~[na:1.8.0_251]
    at java.net.SocketInputStream.read(SocketInputStream.java:141) ~[na:1.8.0_251]
    at java.io.BufferedInputStream.fill(BufferedInputStream.java:246) ~[na:1.8.0_251]
    at java.io.BufferedInputStream.read1(BufferedInputStream.java:286) ~[na:1.8.0_251]
    at java.io.BufferedInputStream.read(BufferedInputStream.java:345) ~[na:1.8.0_251]
    at sun.net.www.http.HttpClient.parseHTTPHeader(HttpClient.java:735) ~[na:1.8.0_251]
    at sun.net.www.http.HttpClient.parseHTTP(HttpClient.java:678) ~[na:1.8.0_251]
    at sun.net.www.protocol.http.HttpURLConnection.getInputStream0(HttpURLConnection.java:1593) ~[na:1.8.0_251]
    at sun.net.www.protocol.http.HttpURLConnection.getInputStream(HttpURLConnection.java:1498) ~[na:1.8.0_251]
```

因为 1s 确实太短了，因此我们需要手动设置 OpenFeign 的超时时间以保证它能正确的处理业务。

OpenFeign 的超时时间有以下两种更改方法：

1. 通过修改 Ribbon 的超时时间，被动的修改 OpenFeign 的超时时间。
2. 直接修改 OpenFeign 的超时时间（推荐使用）。

1.1 设置Ribbon超时时间

在项目配置文件 application.yml 中添加以下配置：

```
ribbon:
  ReadTimeout: 5000 # 请求连接的超时时间
  ConnectionTimeout: 10000 # 请求处理的超时时间
```

1.2 设置OpenFeign超时时间

在项目配置文件 application.yml 中添加以下配置：

```
feign:
  client:
    config:
      default: # 设置的全局超时时间
        connectTimeout: 2000 # 请求连接的超时时间
        readTimeout: 5000 # 请求处理的超时时间
```

推荐使用此方式来设置 OpenFeign 的超时时间，因为这样的（配置）语义更明确。

2.请求连接优化

OpenFeign 底层通信组件默认使用 JDK 自带的 URLConnection 对象进行 HTTP 请求的，因为没有使用连接池，所以性能不是很好。我们可以将 OpenFeign 的通讯组件，手动替换成像 Apache HttpClient 或 OKHttp 这样的专用通信组件，这些的**专用通信组件自带连接池可以更好地对 HTTP 连接对象进行重用与管理，同时也能大大的提升 HTTP 请求的效率**。接下来我以 Apache HttpClient 为例，演示一下专用通讯组件的使用。

2.1 引入Apache HttpClient依赖

在项目的依赖管理文件 pom.xml 中添加以下配置：

```
<!-- 添加 openfeign 框架依赖 -->
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
<!-- 添加 httpClient 框架依赖 -->
<dependency>
  <groupId>io.github.openfeign</groupId>
  <artifactId>feign-httpclient</artifactId>
</dependency>
```

2.2 开启Apache HttpClient使用

启动 Apache HttpClient 组件，在项目配置文件 application.yml 中添加以下配置，：

```
feign:
  client:
    httpClient: # 开启 HttpClient
      enabled: true
```

```

93
94     Response response;
95     try {
96         response = this.client.execute(request, options); client: LoadBalancerFeignClient@7074
97         response = response
98     } catch (IOException va
99         if (this.logLevel != null)
100             this.logger.log(Level.INFO, "Feign exception: {0}", e);
101     }
102
103     throw FeignException.errorStatus(response.status(), response.body(), e);
104 }
105

```

OpenFeign 默认不会开启数据压缩功能，但我们可以手动的开启它的 Gzip 压缩功能，这样可以极大的提高宽带利用率和加速数据的传输速度，在项目配置文件 application.yml 中添加以下配置：

PS: 如果服务消费端的 CPU 资源比较紧张的话, 建议不要开启数据的压缩功能, 因为数据压缩和解压都需要消耗 CPU 的资源, 这样反而会给 CPU 增加了额外的负担, 也会导致系统性能降低。

OpenFeign 底层使用的是 Ribbon 做负载均衡的，查看源码我们可以看到它默认的负载均衡策略是轮询策略，如下图所示：

```
package com.netflix.loadbalancer;

import ...

public class BaseLoadBalancer extends AbstractLoadBalancer implements PrimeConnectionListener, IClientConfigAware {
    private static Logger logger = LoggerFactory.getLogger(BaseLoadBalancer.class);
    private static final IRule DEFAULT_RULE = new RoundRobinRule(); // 轮询负载均衡策略
    private static final BaseLoadBalancer.SerialPingStrategy DEFAULT_PING_STRATEGY = new BaseLoadBalancer.SerialPing
    private static final String DEFAULT_NAME = "default";
    private static final String PREFIX = "LoadBalancer_";
    protected IRule rule;
    protected IPingStrategy pingStrategy;
    protected IPing ping;
}
```

1. **权重策略: WeightedResponseTimeRule**, 根据每个服务提供者的响应时间分配一个权重, 响应时间越长, 权重越小, 被选中的可能性也就越低。它的实现原理是, 刚开始使用轮询策略并开启一个计时器, 每一段时间收集一次所有服务提供者的平均响应时间, 然后再给每个服务提供者附上一个权重, 权重越高被选中的概率也越大。
2. **最小连接数策略: BestAvailableRule**, 也叫最小并发数策略, 它是遍历服务提供者列表, 选取连接数最小的一个服务实例。如果有相同的最小连接数, 那么会调用轮询策略进行选取。

3. **区域敏感策略：ZoneAvoidanceRule**，根据服务所在区域（zone）的性能和服务的可用性来选择服务实例，在没有区域的环境下，该策略和轮询策略类似。
4. 可用敏感性策略：AvailabilityFilteringRule，先过滤掉非健康的服务实例，然后再选择连接数较小的服务实例。
5. 随机策略：RandomRule，从服务提供者的列表中随机选择一个服务实例。
6. 重试策略：RetryRule，按照轮询策略来获取服务，如果获取的服务实例为 null 或已经失效，则在指定的时间之内不断地进行重试来获取服务，如果超过指定时间依然没获取到服务实例则返回 null。

出于性能方面的考虑，我们可以选择用权重策略或区域敏感策略来替代轮询策略，因为这样的执行效率最高。

5. 日志级别优化

OpenFeign 提供了日志增强功能，它的日志级别有以下几个：

- **NONE**：默认的，不显示任何日志。
- **BASIC**：仅记录请求方法、URL、响应状态码及执行时间。
- **HEADERS**：除了 BASIC 中定义的信息之外，还有请求和响应的头信息。
- **FULL**：除了 HEADERS 中定义的信息之外，还有请求和响应的正文及元数据。

我们可以通过配置文件来设置日志级别，配置信息如下：

```
logging:
  level:
    cn.myjszl.service: debug
```

其中 cn.myjszl.service 为 OpenFeign 接口所在的包名。

虽然 OpenFeign 默认是不输出任何日志，但在开发阶段可能会被修改，因此在生产环境中，我们应仔细检查并设置合理的日志级别，以提高 OpenFeign 的运行效率。

总结

OpenFeign 是 Spring 官方推出的一种声明式服务调用和负载均衡组件，在生产环境中我们可以通过以下配置来优化 OpenFeign 的运行：

1. 修改 OpenFeign 的超时时间，让 OpenFeign 能够正确的处理业务；
2. 通过配置专用的通信组件 Apache HttpClient 或 OKHttp，让 OpenFeign 可以更好地对 HTTP 连接对象进行重用和管理，以提高其性能；
3. 开启数据压缩功能，可以提高宽带利用率和加速数据传输速度；
4. 使用合适的负载均衡策略来替换默认的轮询负载均衡策略，已获得更好的执行效率；
5. 检查生成环境中 OpenFeign 的日志级别，选择合适的日志输出级别，防止无效的日志输出。

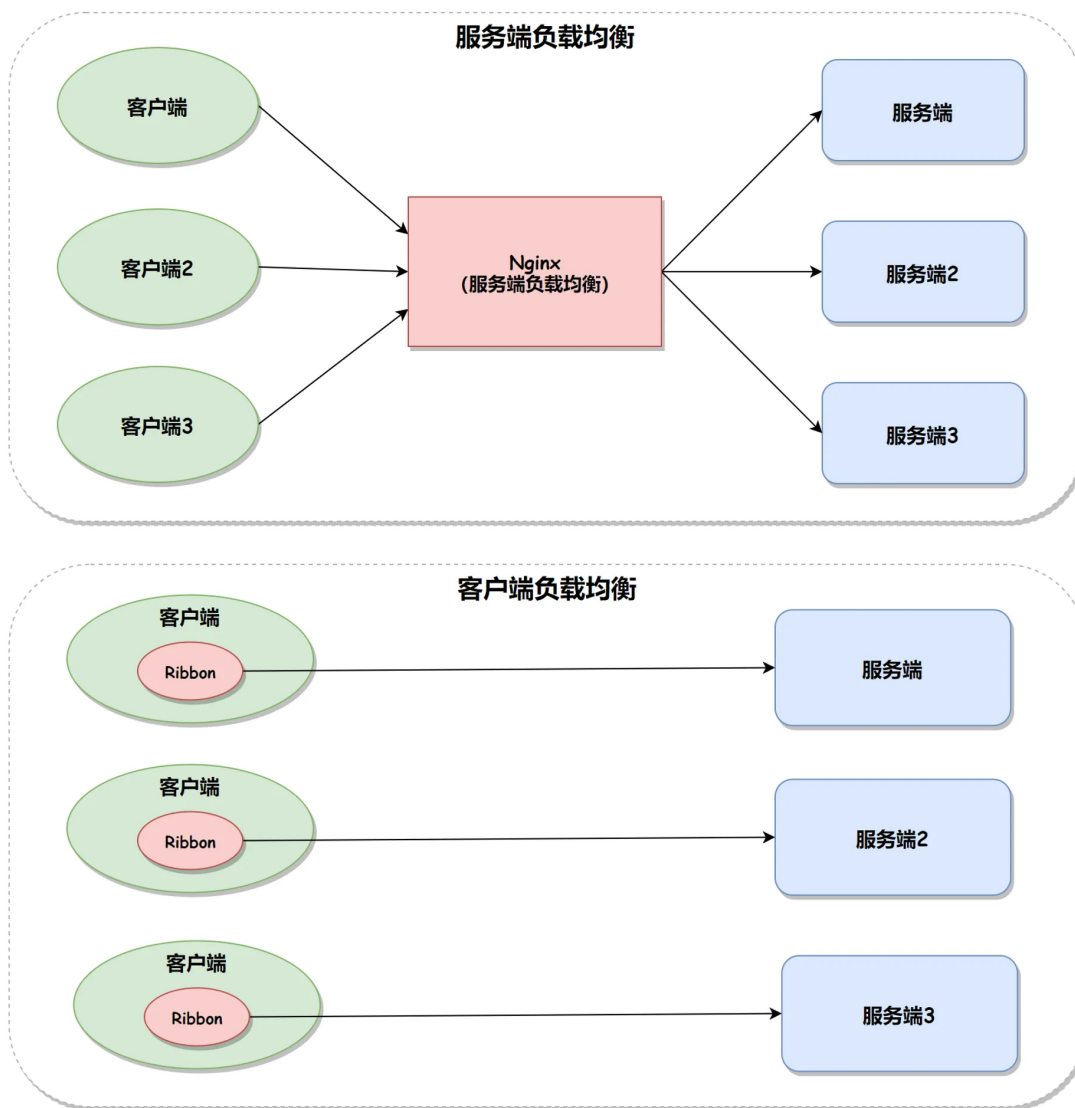
聊聊：Ribbon的7种负载均衡策略？

负载均衡通常有两种实现手段，一种是服务端负载均衡器，另一种是客户端负载均衡器，

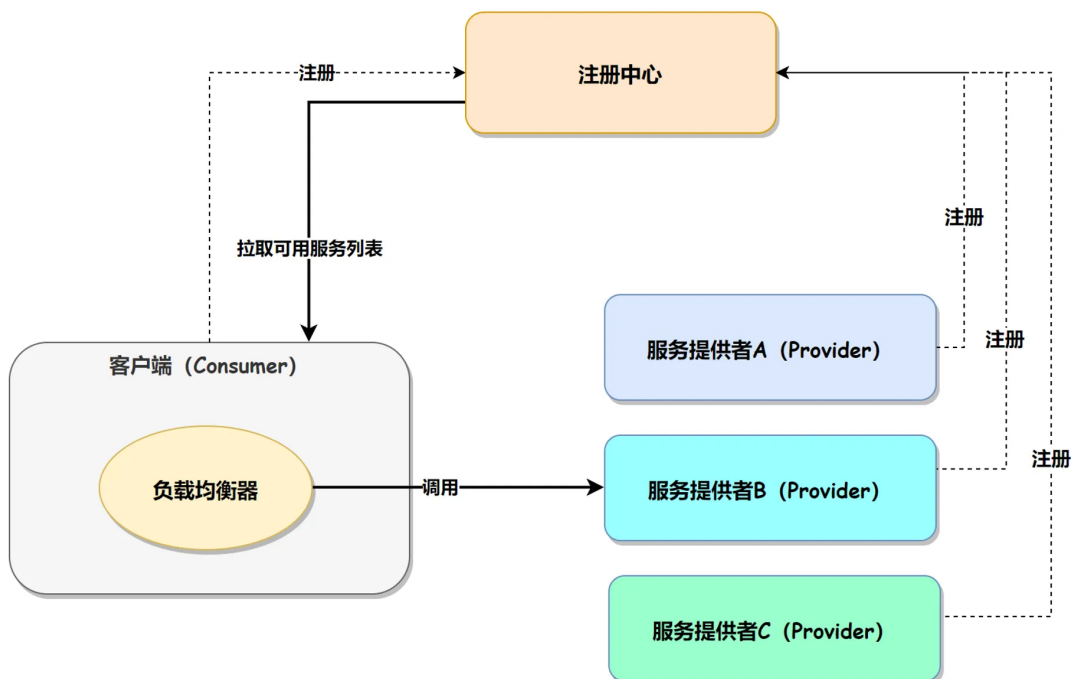
服务端负载均衡器的问题是，它提供了更强的流量控制权，但无法满足不同的消费者希望使用不同负载均衡策略的需求，而使用不同负载均衡策略的场景确实是存在的，所以客户端负载均衡就提供了这种灵活性。

然而客户端负载均衡也有其缺点，如果配置不当，可能会导致服务提供者出现热点，或者压根就拿不到任何服务的情况，所以我们本文就来了解一下这 7 种内置负载均衡策略的具体规则。

服务端负载均衡器和客户端负载均衡器的区别如下图所示：



客户端负载均衡器的实现原理是通过注册中心，如 Nacos，将可用的服务列表拉取到本地（客户端），再通过客户端负载均衡器（设置的负载均衡策略）获取到某个服务器的具体 ip 和端口，然后再通过 Http 框架请求服务并得到结果，其执行流程如下图所示：



而 Ribbon 就属于后者——客户端负载均衡器。

Ribbon 介绍

Ribbon 是 Spring Cloud 技术栈中非常重要的基础框架，它为 Spring Cloud 提供了负载均衡的能力，比如 Fegin 和 OpenFegin 都是基于 Ribbon 实现的，就连 Nacos 中的负载均衡也使用了 Ribbon 框架。

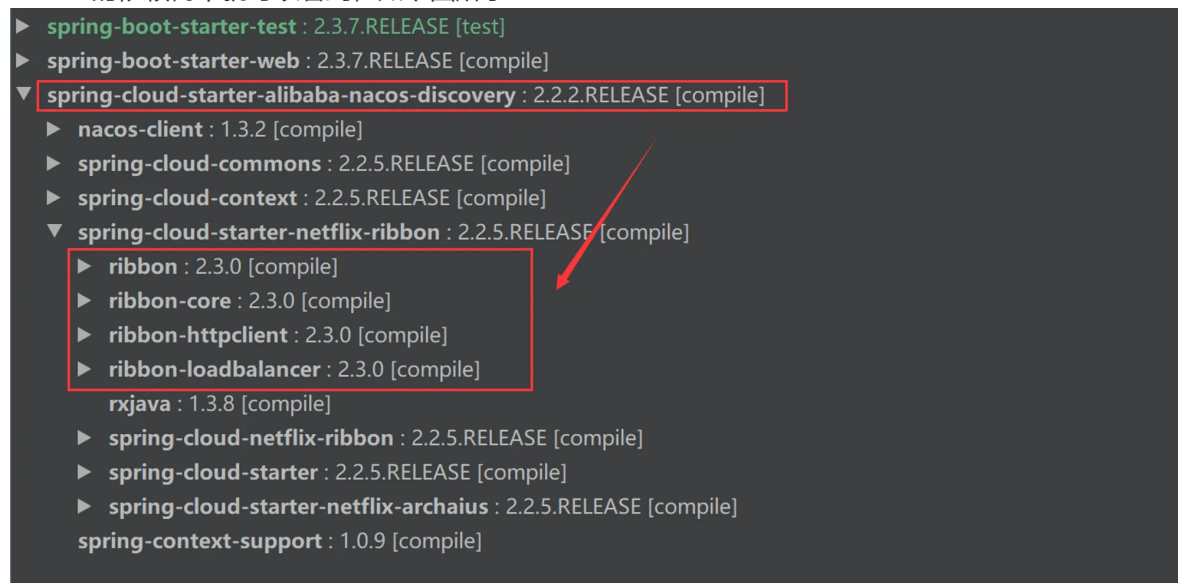
Ribbon 框架的强大之处在于，它不仅内置了 7 种负载均衡策略，同时还支持用户自定义负载均衡策略，所以其开放性和便利性也是它得以流行的主要原因。

负载均衡设置

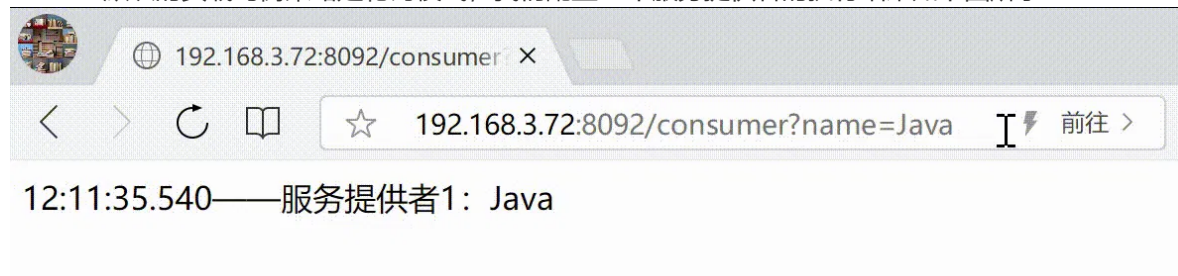
以 Nacos 中的 Ribbon 负载均衡设置为例，在配置文件 application.yml 中设置如下配置即可：

```
springcloud-nacos-provider: # nacos中的服务id
  ribbon:
    NFLoadBalancerRuleClassName: com.netflix.loadbalancer.RoundRobinRule #设置负载均衡策略
```

因为 Nacos 中已经内置了 Ribbon，所以在实际项目开发中无需再添加 Ribbon 依赖了，这一点我们在 Nacos 的依赖树中就可以看到，如下图所示：



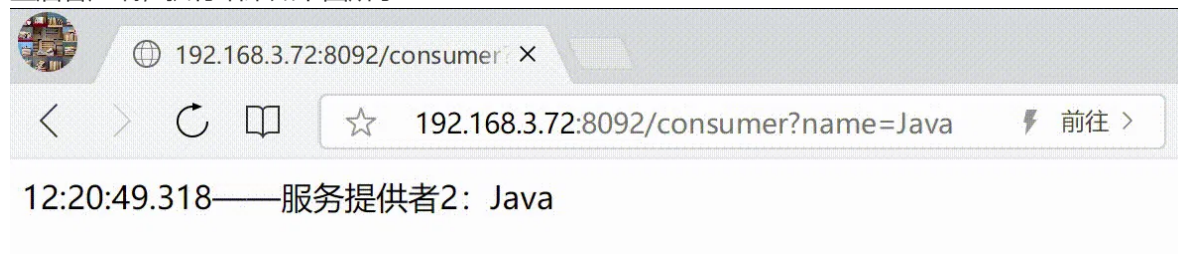
Ribbon 默认的负载均衡策略是轮询模式，我们配置 3 个服务提供者的执行结果如下图所示：



然后，我们再将 Ribbon 负载均衡策略设置为随机模式，配置内容如下：

```
springcloud-nacos-provider: # nacos中的服务id
  ribbon:
    NFLoadBalancerRuleClassName: com.netflix.loadbalancer.RandomRule #设置随机负载均衡
```

重启客户端，执行结果如下图所示：



7种负载均衡策略

1.轮询策略

轮询策略：RoundRobinRule，按照一定的顺序依次调用服务实例。比如一共有 3 个服务，第一次调用服务 1，第二次调用服务 2，第三次调用服务3，依次类推。此策略的配置设置如下：

```
springcloud-nacos-provider: # nacos中的服务id
ribbon:
  NFLoadBalancerRuleClassName: com.netflix.loadbalancer.RoundRobinRule #设置负载均衡
```

2.权重策略

权重策略：WeightedResponseTimeRule，根据每个服务提供者的响应时间分配一个权重，响应时间越长，权重越小，被选中的可能性也就越低。它的实现原理是，刚开始使用轮询策略并开启一个计时器，每一段时间收集一次所有服务提供者的平均响应时间，然后再给每个服务提供者附上一个权重，权重越高被选中的概率也越大。此策略的配置设置如下：

```
springcloud-nacos-provider: # nacos中的服务id
ribbon:
  NFLoadBalancerRuleClassName:
com.netflix.loadbalancer.WeightedResponseTimeRule
```

3.随机策略

随机策略：RandomRule，从服务提供者的列表中随机选择一个服务实例。此策略的配置设置如下：

```
springcloud-nacos-provider: # nacos中的服务id
ribbon:
  NFLoadBalancerRuleClassName: com.netflix.loadbalancer.RandomRule #设置负载均衡
```

4.最小连接数策略

最小连接数策略：BestAvailableRule，也叫最小并发数策略，它是遍历服务提供者列表，选取连接数最小的一个服务实例。如果有相同的最小连接数，那么会调用轮询策略进行选取。此策略的配置设置如下：

```
springcloud-nacos-provider: # nacos中的服务id
ribbon:
  NFLoadBalancerRuleClassName: com.netflix.loadbalancer.BestAvailableRule #设置负载均衡
```

5.重试策略

重试策略：RetryRule，按照轮询策略来获取服务，如果获取的服务实例为 null 或已经失效，则在指定的时间之内不断地进行重试来获取服务，如果超过指定时间依然没获取到服务实例则返回 null。此策略的配置设置如下：

```
ribbon:
  ConnectTimeout: 2000 # 请求连接的超时时间
  ReadTimeout: 5000 # 请求处理的超时时间
springcloud-nacos-provider: # nacos 中的服务 id
  ribbon:
    NFLoadBalancerRuleClassName: com.netflix.loadbalancer.RandomRule #设置负载均衡
```

6.可用性敏感策略

可用敏感性策略：AvailabilityFilteringRule，先过滤掉非健康的服务实例，然后再选择连接数较小的服务实例。此策略的配置设置如下：

```
springcloud-nacos-provider: # nacos中的服务id
  ribbon:
    NFLoadBalancerRuleClassName:
      com.netflix.loadbalancer.AvailabilityFilteringRule
```

7.区域敏感策略

区域敏感策略：ZoneAvoidanceRule，根据服务所在区域（zone）的性能和服务的可用性来选择服务实例，在没有区域的环境下，该策略和轮询策略类似。此策略的配置设置如下：

```
springcloud-nacos-provider: # nacos中的服务id
  ribbon:
    NFLoadBalancerRuleClassName: com.netflix.loadbalancer.ZoneAvoidanceRule
```

总结

Ribbon 为客户端负载均衡器，相比于服务端负载均衡器的统一负载均衡策略来说，它提供了更多的灵活性。Ribbon 内置了 7 种负载均衡策略：轮询策略、权重策略、随机策略、最小连接数策略、重试策略、可用性敏感策略、区域性敏感策略，并且用户可以通过继承 RoundRobinRule 来实现自定义负载均衡策略。

聊聊：SpringCloud 和 Dubbo 有哪些区别

首先，他们都是分布式管理框架。

Dubbo 是二进制传输，占用带宽会少一点。SpringCloud是http 传输，带宽会多一点，同时使用http协议一般会使用JSON报文，消耗会更大。

Dubbo 开发难度较大，所依赖的 jar 包有很多问题大型工程无法解决。SpringCloud 对第三方的继承可以一键式生成，天然集成。SpringCloud 接口协议约定比较松散，需要强有力的行政措施来限制接口无序升级。

最大的区别：

Spring Cloud抛弃了Dubbo 的RPC通信，采用的是基于HTTP的REST方式。

严格来说，这两种方式各有优劣。虽然在一定程度上来说，后者牺牲了服务调用的性能，但也避免了上面提到的原生RPC带来的问题。

而且REST相比RPC更为灵活，服务提供方和调用方的依赖只依靠一纸契约，不存在代码级别的强依赖，这在强调快速演化的微服务环境下，显得更为合适。

微博一面：RPC怎么做零呼损？

说在前面

在40岁老架构师 尼恩的[读者交流群](#)(50+)中，最近有小伙伴拿到了一线互联网企业如微博、阿里、汽车之家、极兔、有赞、希音、百度、网易、滴滴的面试资格，遇到一几个很重要的面试题：

- RPC怎么做无损升级？
- 微服务发布的时候，RPC怎么做零呼损？

与之类似的、其他小伙伴遇到过的问题还有：

- 微服务升级时，RPC里怎么避免调用方业务受损呢？

所以，这里尼恩给大家做一下系统化、体系化的梳理，使得大家可以充分展示一下大家雄厚的“技术肌肉”，让面试官爱到“不能自己、口水直流”。

也一并把这个题目以及参考答案，收入咱们的《[尼恩Java面试宝典](#)》V108版本，供后面的小伙伴参考，提升大家的 3高 架构、设计、开发水平。

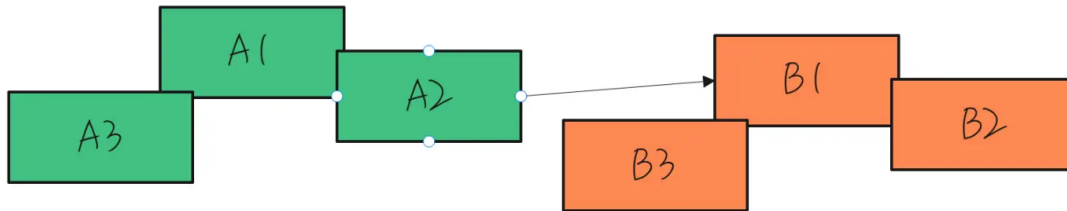
注：本文以 PDF 持续更新，最新尼恩 架构笔记、面试题 的PDF文件，请从公众号【技术自由圈】获取。

场景分析

微服务架构中，线上跑着几十个，甚至上百的微服务。

服务实例之间，有着错综复杂的RPC调用关系。

这些服务实例，归属到不同的小分队进行开发、设计、维护。



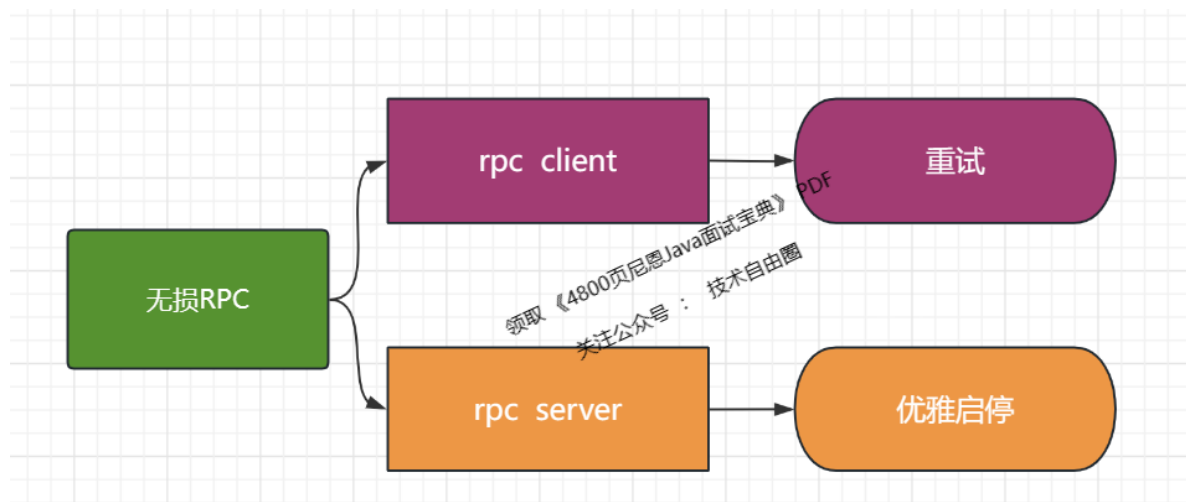
问题是：

在其中部分服务实例**重启、升级的过程中**，怎么做到让微服务RPC调用方系统不出问题呢？

确保RPC零呼损的维度

如果要确保RPC零呼损，至少可以从以下两个维度进行规避：

- 维度一：rpc client 维度
- 维度一：rpc server 维度



rpc client 维度零呼损的有效措施是：重试

rpc server 维度零呼损的有效措施是：优雅启停

rpc client 维度的零呼损：重试

rpc client 在发生错误的时候，可以进行 服务实例的 重试。

常见的rpc框架如 OpenFeign 框架就有重试几次、重试间隔这样的参数。

当然，如果希望通过重试的机制，让RPC零呼损，那么要保证升级的时候，还有健康的实例可用。

不能所有的微服务实例都同时升级，从而同时不可以使用，这样的话，重试也没有意义。

所以，重试措施最好配套对应的线上滚动升级/滚动发布、或灰度升级/灰度发布等类似的策略。

rpc server 维度的零呼损：优雅启停

优雅启停包括：

- 优雅启动
- 优雅停止/优雅下线

优雅启动

优雅启动，当微服务实例真正完成启动，甚至完成预热之后，真正具备处理 rpc 请求能力的时候，再将实例自己注册到注册中心。

为啥要优雅启动呢？核心原因是：避免了RPC请求发进来，没有完成启动流程的微服务实例却无法处理。

优雅启动的流程，还包含JVM预热的流程，有关JVM预热的方案，请查看尼恩的上一篇文章：

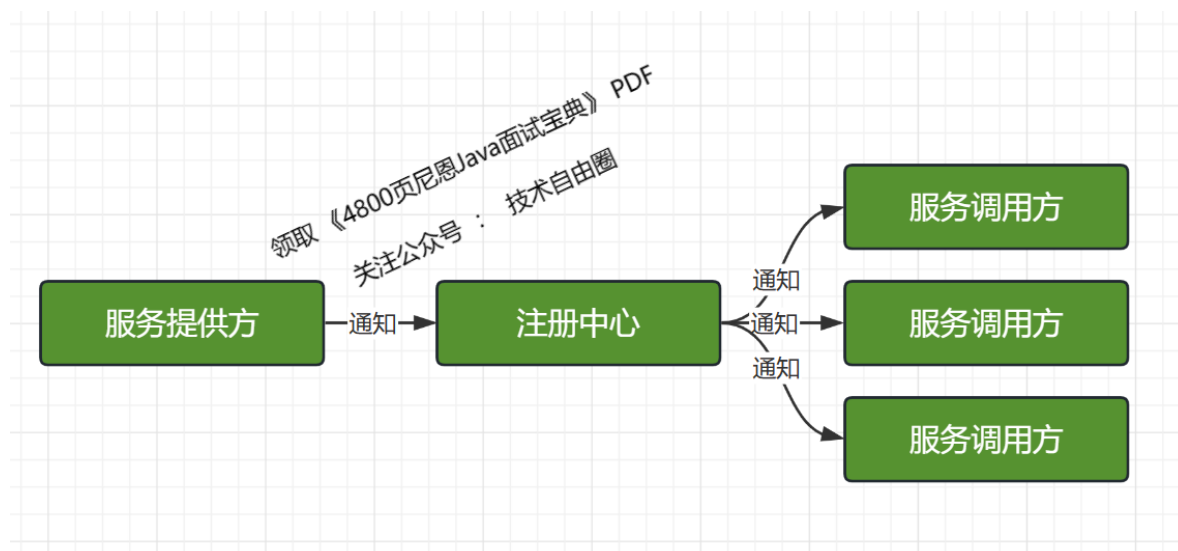
[微博一面：JVM预热，你的方案是啥？](#)

优雅停止/优雅下线

在服务下线前，先通过“某种方式”把要下线的实例从调用方维护的“健康列表”里面删除就可以了，这样负载均衡就选不到这个节点

这个操作，一般来说，都是依赖注册中心完成的。

当服务提供方关闭前，先通知注册中心进行下线，然后通过注册中心告诉调用方进行节点摘除。



如上图所示，整个关闭过程中依赖了两次 RPC 调用：

- 一次是服务提供方通知注册中心下线操作
- 一次是注册中心通知服务调用方下线节点操作。

需要注意的是：两次通知都是异步的，只保证最终一致性，并不保证强一致性。

这个中间，有一定的时间周期，所以，如果要做到应用无损升级，需要在发出通知之后，隔一段时间，再把服务实例正式关闭。

优雅停止/优雅下线的简单流程

1. 下线实例在注册中心进行注销，注销该实例元数据信息；
2. 注册中心节点元数据更新周期为15s，调用方在感知注册中心实例变更后，更新本地缓存服务地址，不再将流量路由到下线实例，期间保障业务无中断；
3. 下线实例等待30s（2个心跳周期）后，进行实际下线操作；

优雅停止的总结：优雅停止是当微服务快要下线的时候，先从注册中心进行去注销，然后把接收到的RPC调用消息，处理完毕后，再彻底关闭。通过优雅停机，可以有效地防止升级期间，发送到老节点的呼损。需要注意发送下线通知，到正式下线之间的时间间隔。

确保RPC零呼损的维度

如果要确保RPC零呼损，至少可以从以下两个维度进行规避：

- 维度一：rpc client 维度
- 维度一：rpc server 维度

rpc client 维度的核心策略是重试

rpc server 维度的核心策略是优雅启停

两个维度都不可或缺，都需要实现。

说在最后

RPC 相关面试题，是非常常见的面试题。

以上的内容，如果大家能对答如流，如数家珍，基本上面试官会被你震惊到、吸引到。

在面试之前，建议大家系统化的刷一波 5000页《尼恩Java面试宝典》，并且在刷题过程中，如果有啥问题，大家可以来找 40岁老架构师尼恩交流。

最终，让面试官爱到“不能自己、口水直流”。offer，也就来了。

参考文献

<https://blog.csdn.net/Weixiaohuai/article/details/125391957>

<https://www.cnblogs.com/daoqidelv/p/7043696.html>

清华大学出版社《Java高并发核心编程 卷2 加强版》

重磅推荐：起底式学习SpringCloud的书籍

疯狂创客圈 《SpringCloud Nginx 高并发核心编程》高并发 架构师 必备【[链接](#)】



SpringCloud、Nginx高并发核心编程【2020年11月新书】

说明：此书2019年出版，2021年新版更名为《Java 高并发核心编程（卷3）》，2022年升级为加强版

《SpringCloud、Nginx高并发核心编程》升级为《Java 高并发核心编程 卷3加强版》后，新版本 更加经典



《SpringCloud、Nginx高并发核心编程》
已经过时 >>> 请领取 最新版
《Java高并发核心编程 卷3加强版：亿级用户Web应用架构与实战》



免费领取：尼恩Java高并发三部曲，极致经典+入大厂必备+面试必备+高薪必备——建议猛刷三遍，打好底子

CSDN @40岁资深老架构师尼恩

参考文献：

<https://www.cnblogs.com/-wenli/p/13584796.html>

<https://www.cnblogs.com/crazymakercircle/p/11664812.html>

<https://www.cnblogs.com/blessing2022/p/16622105.html>

推荐阅读

- 《[百亿级访问量，如何做缓存架构设计](#)》
- 《[多级缓存 架构设计](#)》
- 《[消息推送 架构设计](#)》
- 《[阿里2面：你们部署多少节点？1000W并发，当如何部署？](#)》
- 《[美团2面：5个9高可用99.999%，如何实现？](#)》
- 《[网易一面：单节点2000Wtps，Kafka怎么做的？](#)》
- 《[字节一面：事务补偿和事务重试，关系是什么？](#)》
- 《[网易一面：25Wqps高吞吐写Mysql，100W数据4秒写完，如何实现？](#)》
- 《[亿级短视频，如何架构？](#)》
- 《[炸裂，靠“吹牛”过京东一面，月薪40K](#)》
- 《[太猛了，靠“吹牛”过顺丰一面，月薪30K](#)》
- 《[炸裂了...京东一面索命40问，过了就50W+](#)》
- 《[问麻了...阿里一面索命27问，过了就60W+](#)》
- 《[百度狂问3小时，大厂offer到手，小伙真狠！](#)》
- 《[饿了么太狠：面个高级Java，抖这多硬活、狠活](#)》
- 《[字节狂问一小时，小伙offer到手，太狠了！](#)》
- 《[收个滴滴Offer：从小伙三面经历，看看需要学点啥？](#)》

技术自由圈

实现架构转型，再无中年危机



关注**技术自由圈**公众号，获取每天技术干货
一起成为牛逼的**未来超级架构师**

几十篇架构笔记、5000页面试宝典、20个技术圣经

请加尼恩个人微信 免费拿走

暗号，请在 公众号后台 发送消息：**领电子书**

未来职业，如何突围：三栖架构师

未来职业，如何突围？

技术自由圈



——未来超级架构师社区

领路式指导

FSAC 三栖合一架构师

Future Super Architect Community

- 第一栖：Java 架构
- 第二栖：GO 架构
- 第三栖：大数据 架构

尼恩JAVA硬核架构班

会员制

提供技术方向指导，
职业生涯指导，少坑，少走弯路

简历指导

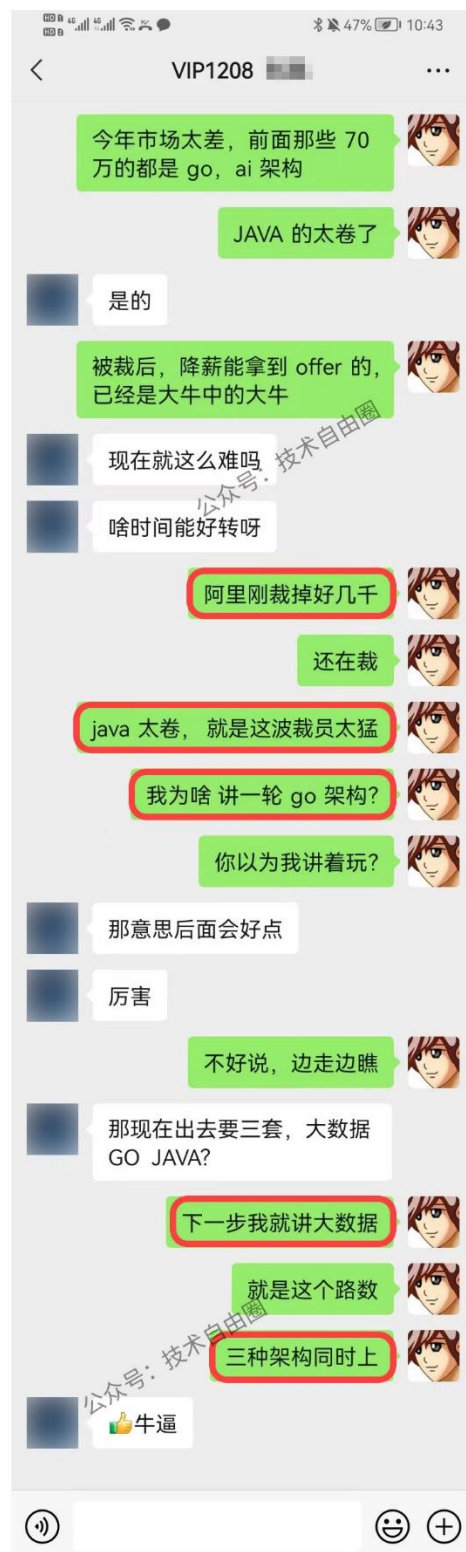
有助成功就业、跳槽大厂
挪窝涨薪必备

实操性

项目都是老架构师
在生产上实操过的项目

非水货

老架构师，不是水货架构师
《Java高并发三部曲》为证



成功案例：2年翻3倍，35岁卷王成功转型为架构师

详情：<http://topcoder.cloud/forum.php?mod=forumdisplay&fid=43&page=1>

最新 最后发表 热门 精华

成功案例：[1057号卷王] 3年小伙拿到外企offer，薪酬涨了200%

1 卷王1号 超级版主 前天 17:41

成功案例：[645号卷王] 4年经验卷王逆袭，被毕业后，反涨24W

1 卷王1号 超级版主 2022-9-21

成功案例：[878号卷王] 小伙8年经验，年薪60W

1 卷王1号 超级版主 2022-8-13

年薪70W案例：通过尼恩的指导，小伙伴年薪从40W涨到70W

1 卷王1号 超级版主 2022-2-11

成功案例：[493号卷王] 5年小伙拿满意offer，就业寒冬季逆涨30%

1 卷王1号 超级版主 前天 17:43

成功案例：[250号卷王] 就业极寒时代，收offer 涨25%

1 卷王1号 超级版主 前天 17:38

成功案例：[612号卷王] 就业极寒时代，从外包到自研

1 卷王1号 超级版主 前天 17:15

成功案例：[913号卷王] 热烈祝贺6年经验卷王，年薪40W

1 卷王1号 超级版主 2022-9-21

成功案例：[959号卷王] 4年经验卷王，喜获百度、Boss直聘等N个优质offer，最高涨100%

1 卷王1号 超级版主 2022-9-21

成功案例：[529号卷王] 5年经验卷王喜收2大offer，最高涨5K

1 卷王1号 超级版主 2022-9-21

成功案例：[811号卷王] 热烈祝贺7年经验卷王，薪酬涨30%

1 卷王1号 超级版主 2022-9-21

成功案例：[287号卷王] 不惧大寒潮，卷王逆市收4 offer，涨30%，可喜可贺

1 卷王1号 超级版主 2022-5-30

成功案例：[1002号卷王] 5月份“被毕业”，改简历后，斩获顶级央企Offer，涨薪7000+

1 卷王1号 超级版主 2022-7-5

成功案例: [7号卷王] 热烈祝贺小伙伴涨薪120%

1 卷王1号 超级版主 2022-8-13

成功案例: [134号卷王] 大三小伙卷1年, 斩获顶级央企Offer, 成功逆袭

1 卷王1号 超级版主 2022-7-6

成功案例: [1008号卷王] 5年经验卷王收42W offer, 月涨8000, 可喜可贺

1 卷王1号 超级版主 2022-5-30

成功案例: [453号卷王] 非全日制 6年卷王喜提3 offer, 年薪30W, 可喜可贺

1 卷王1号 超级版主 2022-5-21

成功案例: [924号卷王] 6年卷王喜提4 offer, 最高涨薪9000, 可喜可贺

1 卷王1号 超级版主 2022-5-21

成功案例: [15号卷王] 4年卷王入职 微软, 涨薪50%, 可喜可贺

1 卷王1号 超级版主 2022-5-12

成功案例: [527号卷王] 4年卷王喜提2 offer, 涨薪50%, 可喜可贺

1 卷王1号 超级版主 2022-5-13

成功案例: [788号卷王] 3年卷王喜提优质Offer, 涨薪60%

1 卷王1号 超级版主 2022-5-11

成功案例: 热烈祝贺: 非全日制卷王, 喜提2个心仪offer, 面3家过2家

1 卷王1号 超级版主 2022-4-21

成功案例: [693号卷王] 二线城市6年卷王喜提4大优质Offer, 含央企offer, 最高薪酬35W

1 卷王1号 超级版主 2022-4-16

成功案例: [85号卷王] 双非2本小伙, 春招大捷, 喜提9个offer, 最高薪酬近30万

1 卷王1号 超级版主 2022-4-14

成功案例: [741号卷王] 卷王逆袭! 6年小伙从很少面试机会到搞定35K*14薪Offer

1 卷王1号 超级版主 2022-4-12

成功案例: [642号卷王] 热烈祝贺, 6年卷王喜提优质国企offer

1 卷王1号 超级版主 2022-4-7

成功案例: [796号卷王] 热烈祝贺, 36岁卷王喜提52万优质offer

1 卷王1号 超级版主 2022-3-25

❑ 成功案例: [15号卷王] 小伙卷1年, 涨薪9K+, 喜收ebay等多个优质offer

① 卷王1号 超级版主 2022-3-24

❑ 成功案例: [821号卷王] 小伙狠卷3个月, 喜提10多个offer

① 卷王1号 超级版主 2022-3-21

❑ 成功案例: [736号卷王] 3年半经验收22k offer, 但是小伙志存高远, 冲击25k+

① 卷王1号 超级版主 2022-3-20

❑ 成功案例: 热烈祝贺1群小卷王offer拿到手软, 甚至拒了阿里offer

① 卷王1号 超级版主 2022-3-16

❑ 简历案例: 简历一改, 腾讯的邀请就来了! 热烈祝贺, 小伙收到一大堆面试邀请

① 卷王1号 超级版主 2022-3-10

❑ 成功案例: 祝贺我圈两大超级卷王, 一个过了阿里HR面, 一个过了阿里2面

① 卷王1号 超级版主 2022-3-10

❑ 成功案例: 小伙伴php转Java, 卷1.5年Java, 涨薪50%, 喜收多个优质offer

① 卷王1号 超级版主 2022-3-10

❑ 成功案例: 4年小伙狠卷半年, 拿到 移动、京东 两大顶级offer

 尼恩 超级版主 2022-3-5

❑ 成功案例: [267号卷王] 助力3年经验卷王, 拿到蜂巢的17k x 14薪的offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [143号卷王] 二本院校00后卷神, 毕业没到一年跳到字节, 年薪45W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [494号卷王] 尼恩分布式事务助力卷王拿到 中信银行offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [76号卷王] 2线城市卷王, 狠卷1.5年, 喜收22K offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [429号卷王] 小伙伴在社群卷5个月, 涨8k+

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [154号卷王] 双非学校毕业卷王, 连拿 京东到家&滴滴 两个大厂Offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [232号卷王] 涨薪10K, 继续卷向食物链顶端

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: 狠卷1年技术, 喜收 腾讯、阿里、微软三大Offer, 最高年薪56W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [449号卷王] 应届毕业卷王喜收 滴滴offer, 年薪33W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [551号卷王] 小伙伴学完后, 成功进入大厂, 并且推荐自己的朋友加VIP学习

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: [214号卷王] 助力2年经验卷王, 成功拿到17K月薪

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: [92号卷王] 课程实操助力社群小伙伴喜收 喜马拉雅Offer

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: 社群卷王小伙伴成功过了滴滴三面 获滴滴Offer

① 卷王1号 超级版主 2022-2-10

❑ [612号卷王]滴滴小伙伴, 蹲点考察半年, 觉得靠谱后加入 疯狂创客圈

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: [732号卷王] 尼恩助力3年经验卷王收获 京东offer, 年薪35W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [558号卷王] 2年经验卷王, 喜收 网易和阿里子公司两个优质offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [569号卷王] 双非应届生卷王, 喜收字节跳动实习offer

① 卷王1号 超级版主 2022-2-25

❑ 成功案例: [420号卷王] 狠卷1年, 卷王涨薪80%, 涨薪12000元!

① 卷王1号 超级版主 2022-2-25

❑ 成功案例: [76号卷王] 通过尼恩1年半的指导, 专科学历小伙伴从0.8K涨到22K

① 卷王1号 超级版主 2022-2-10

硬核推荐：尼恩Java硬核架构班

详情：<https://www.cnblogs.com/crazymakercircle/p/9904544.html>

尼恩Java 硬核架构班

已经发布

- ★ 《高性能RPC的基础实操之：从0到1开始IM撸一个IM》
- ★ 《分布式高性能RPC的基础实操之：千万级用户分布式IM实操- 含简历指导》
- ★ 《亿级用户超高并发秒杀实操- 含简历指导》
亮点：助力小伙伴搞定70W年薪，N个涨薪50%，**2023夏招面试涨薪神器**
- ★ 《横扫全网，工业级elasticsearch底层原理与高并发、高可用架构实操》
亮点：40岁老架构师细致解读，处处透着分布式、高性能中间件的原理和精髓
- ★ 《第1部曲：超级底层：葵花宝典（高性能秘籍）架构师视角解读OS操作系统》
亮点：大制作解读OS操作系统，并揭秘mmap、pagecache、zerocopy等底层的底层原理
2023夏招面试涨薪大神器
- ★ 《Rocketmq视频第2部曲：横扫全网工业级 rocketmq 高可用（HA）底层原理和实操》
亮点：起底式、绞杀式解读 rocketmq如何保障消息的可靠性？
- ★ 《Rocketmq视频第3部曲：超级内功篇、横扫全网 rocketmq 源码学习以及3高架构模式解读》
亮点：大制作解读 Rocketmq源码以及3高架构模式，助力大家内力猛增
- ★ 《Rocketmq视频第4部曲：10Wqps消息推送中台架构、设计、编码、测试实操》
亮点：Netty实操、分库分表实操、Rocketmq工业级使用实操
- ★ 《架构师内功篇：横扫全网 netty 高性能、高并发架构 底层原理、源码学习》
- ★ 《架构师实操篇：redis cluster 工业级高可用实操》
- ★ 《架构师实操篇：100W级别QPS日志平台实操》
- ★ 《彻底穿透：skywalking 源码(代表链路跟踪)+Java agent+bytebuddy 探针》
- ★ 《超高并发场景100Wqps三级缓存组件原理和实操》
- ★ 《全链路异步超底层原理和实操：手写hystrix熔断+webflux+Lettuce+Dubbo》
- ★ 《穿透云原生K8S+Jenkins+SpringCloud底层原理和实操》
- ★ 《Golang学习圣经，Go+Java混合 微服务架构 原理与实操》

规划中



左手大数据 (写入简历, 让简历 蓬荜生辉、金光闪闪)

HBASE + Flink + ElasticSearch 原理、架构、真刀实操



右手云原生 (写入简历, 让简历 蓬荜生辉、金光闪闪)

K8S + Devops + ServiceMesh 原理、架构、真刀实操

架构师实操篇: 基于netty 手写 rpc 框架- 参考 dubbo、seata rpc框架

架构师实操篇: 千万级任务调度平台 架构与实操- 基于尼恩17年的亿级搜索项目

架构师实操篇: 工业级 亿级文档搜索 平台 架构与实操- 基于尼恩17年的亿级搜索项目

尼恩JAVA硬核架构班 特色

会员制

提供技术方向指导,
职业生涯指导, 少躺坑, 少弯路

简历指导

有助成功就业、跳槽大厂
挪窝涨薪必备

实操性

项目都是老架构师
在生产上实操过的项目

非水货

老架构师, 不是水货架构师
《Java高并发三部曲》为证



手把手帮扶



让 少部分人 先走向 架构师岗位

2小时简历指导, 传20年内功



架构班（社群 VIP）的起源：

最初的视频，主要是给读者加餐。很多的读者，需要一些高质量的实操、理论视频，所以，我就围绕书，和底层，做了几个实操、理论视频，然后效果还不错，后面就做成迭代模式了。

架构班（社群 VIP）的功能：

提供高质量实操项目整刀真枪的架构指导、快速提升大家的：

- 开发水平
- 设计水平
- 架构水平

弥补业务中 CRUD 开发短板，帮助大家尽早脱离具备 3 高能力，掌握：

- 高性能
- 高并发
- 高可用

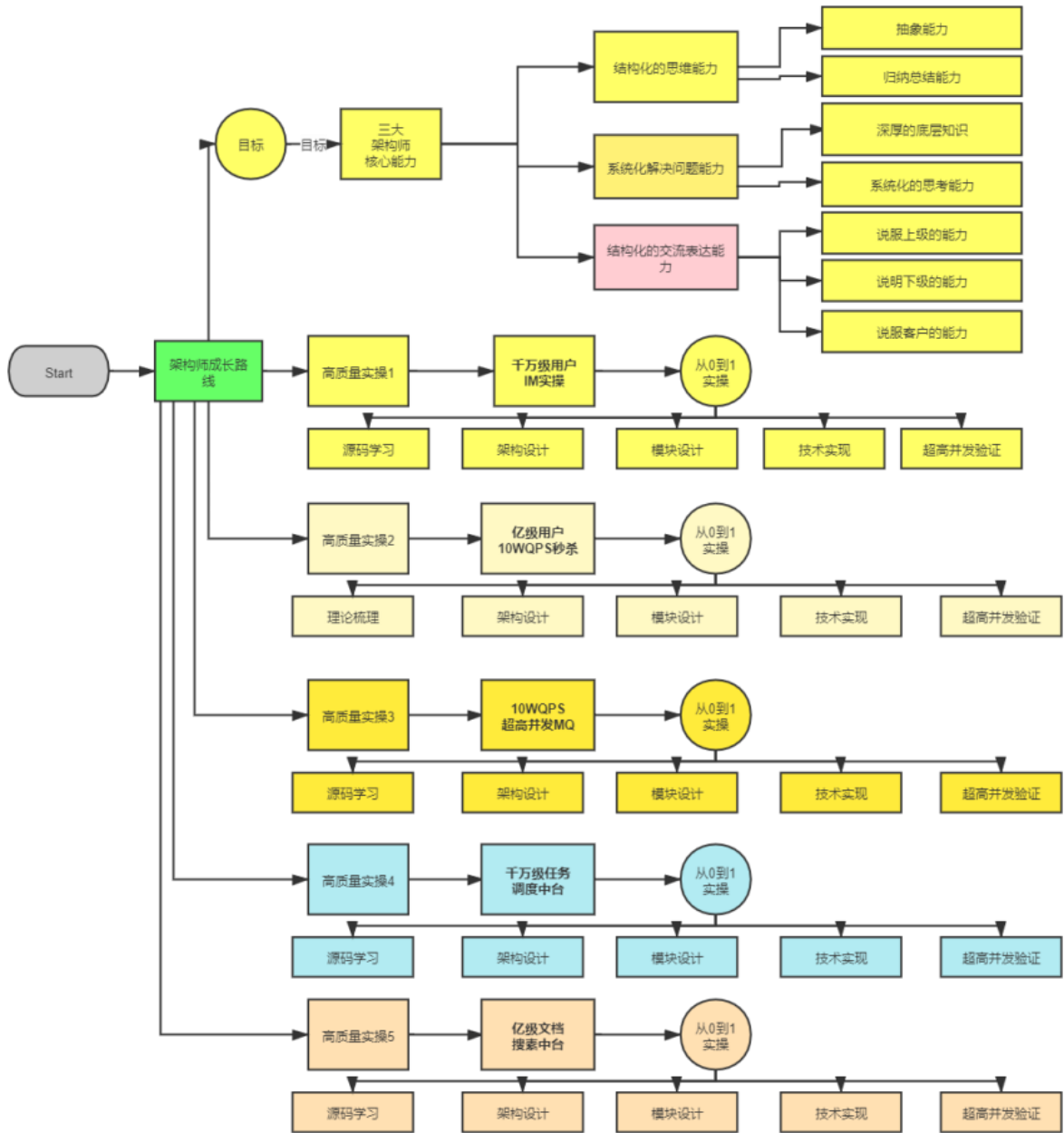
作为一个高质量的架构师成长、人脉社群，把所有的卷王聚焦起来，一起卷：

- 卷高并发实操
- 卷底层原理
- 卷架构理论、架构哲学
- 最终成为顶级架构师，实现人生理想，走向人生巅峰

架构班（社群 VIP）的目的：

- 高质量的实操，大大提升简历的含金量，吸引力，增强面试的召唤率
- 为大家提供九阳真经、葵花宝典，快速提升水平
- 进大厂、拿高薪
- 一路陪伴，提供助学视频和指导，辅导大家成为架构师
- 自学为主，和其他卷王一起，卷高并发实操，卷底层原理、卷大厂面试题，争取狠卷 3 月成高手，狠卷 3 年成为顶级架构师

N 个超高并发实操项目：简历压轴、个顶个精彩



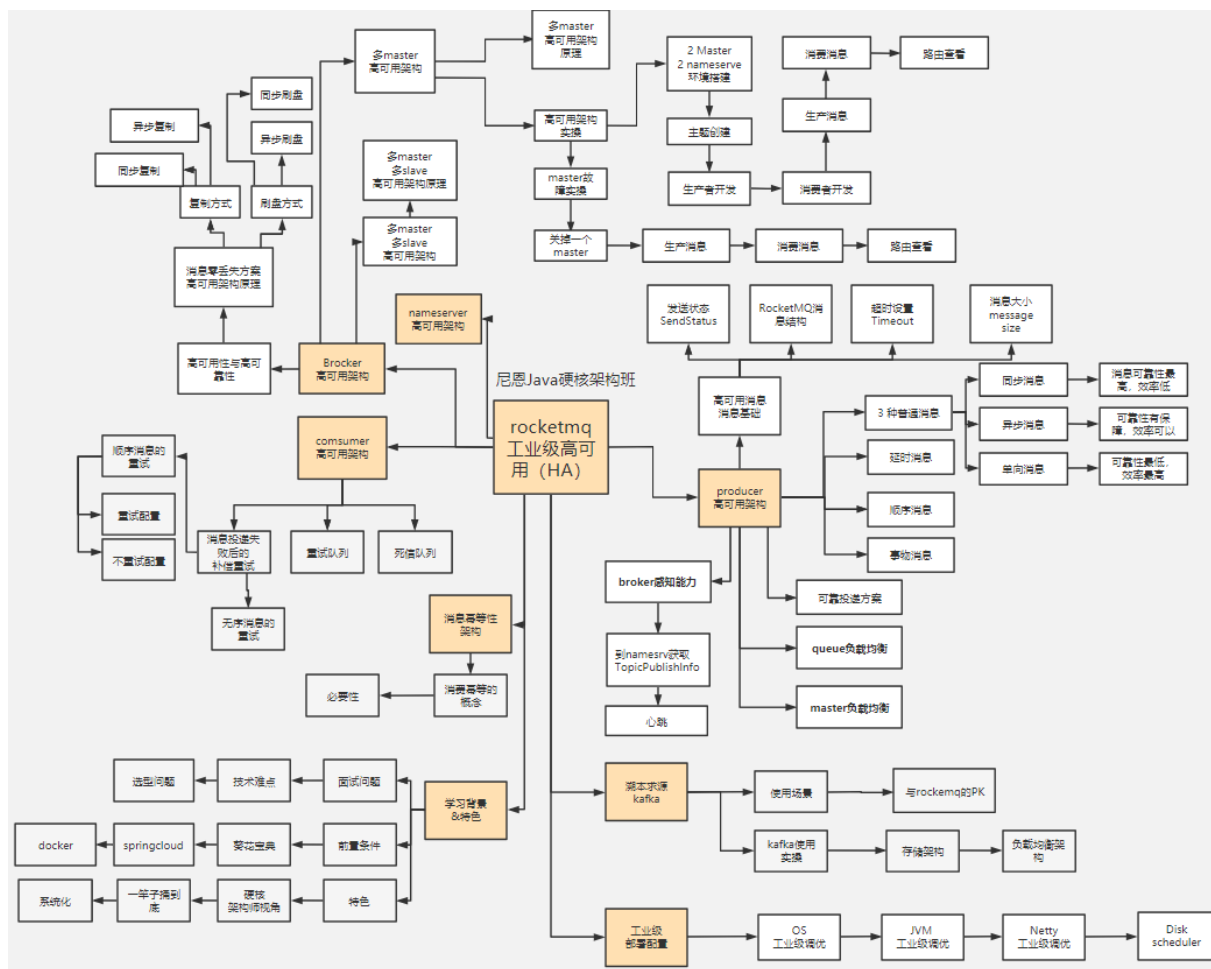
工业级 rocketmq 高可用底层原理和搭建实操，包含：高可用集群的搭建。

1、技术难题：RocketMQ 如何最大限度的保证消息不丢失的呢？RocketMQ 消息如何做到高可靠投递？

2、技术难题：基于消息的分布式事务，核心原理不理解

3、选型难题： kafka or rocketmq ，该娶谁？

下图链接: <https://www.processon.com/view/6178e8ae0e3e7416bde9da19>



简历优化后的成功涨薪案例（VIP含免费简历优化）

6年专科，2年翻4倍

2年从8K涨到35K

2021年从8K涨到22K

高并发 VIP76

老师，求助。

现在有两个满意的 offer，不知道怎么抉择。

一个是吉利，17k，大数据与 ai 部门。

另一个是一个平台，从零开始用 java 重写现在的项目，分布式架构，带团队，自己招人。22k，我觉得我说少了，我自己提的，然后今天发了 offer

呵呵，你太牛了

我也不好说

工资高的是个小公司，不到 50 人

感觉好事都被你占了

这一年半，真的谢谢您。

呵呵，相互交流，相互成长。

您写的书本，解决了我项目上很多问题。您在群里不厌其烦地告诉我们学习，也是我能坚持下来的重要因素，还有每次提问您都能解答疑惑，让我始终能戒骄戒躁。恩师，

**秘诀：
简历指导+ 狠狠卷**

2022年涨到35K

VIP76

解决了，限制 ip 频率。

谢谢老师

中午12:43

调整到了 35,加上这个月加班费，38

中午12:43

老师，我隔瑟来了。

晚上8:23

大大的赞

老师你这路子是对的。我就跟着你学习思路和方法，还有教程走的。

我和你一样的兴奋和喜悦

记得咱们去年改简历的时候，还是 10k

这种提升，已经太令人震撼啦

是 8K...

20 年 4 月份转行，就一路跟着你学习

6年小伙收60W年薪 一月速提3大offer

4.24号改简历

5.27号报喜

VIP1239 6年

4月24日 晚上19:10

预期的岗位: 60W

预期的岗位: go 和 java 的后端开

4月24日 晚上19:56

4月24日 晚上20:50

6年经验 1239 年薪 60W. 21.7 KB

前几天改ai, 今天改go

谢谢, 我根据这个模式, 再整理一下我简历, 到时候老师再帮我把关

ok

4月27日 下午14:37

老师, 再帮我看看

准备开始投简历了

简历-6年后端开发.pdf 210.1 KB

嗯, 我先对着简历准备些东西, 然后再开始投吧, 反正现在刚刚五一放假

上午8:01

来还愿咯, 3周斩获3个offer, 准备入职了

上午8:05

您太牛啦

简历优化后, 面试机会太多了, 拿完3个offer后, 还有许多公司在流程中的都拒绝了

这几天大动作不断, 联想, 阿里都在裁员, 您太牛啦

还是老师你强, offer中也达到了预期60w

非常不错

现在都上岸有offer就行, 薪酬还能达到预期, 已经超级牛啦

很多人, 连一个面试电话都没有, 崩溃的一塌糊涂

抖音上到处是这种

主要是简历优化后, 感觉如有神助, 每天基本3个面试, 除了字节一面没过, 其它都通过了

恭喜您

谢谢老师

后续有啥问题, 可以找我支援哈

好嘞, 学习圈一直在, 要持续提高自己

好的, 撸起袖子加油卷, 搞技术前途无限好

秘诀:
简历指导+ 狠狠卷

被裁后转架构, 逆涨 50% 8年小伙喜提年薪75W

4.16号改简历

5.6号报喜

VIP1236

最近面试了几个一轮游

捞了太多人上岸了

都是你这号

捞我

助力我一个月时间

绞杀 下钻 打破瓶颈

咱们开始不?

好

4月16日 下午15:04

预期岗位: 高级开发、架构

4月16日 下午15:12

预期薪酬: 60W

8-8年高级服务端-0404 - 副本(2). 30.2 KB

微信电脑版

4月16日 下午16:19

秘诀:
简历指导+ 狠狠卷

4月16日 下午19:21

通话时长 03:02:25

辛苦老师

尼架, 我决定要去上海了。

拿了几个offer?

两个

上海这个是架构师对吧? 还有一个呢?

还有个广州高级Java, 待遇40w左右, 老板比较喜欢我, 开了很多绿灯, 薪资可以再加, 但我还是想闯闯, 昨天拒绝了。

两年包多少呢?

就之前说的

那都快80个W了

我argue了下, 他们控制内部薪酬平衡有点难办到80, 但已经是标出来的上限了。

都快是高级java的两倍

你撤回了一条消息

75w也非常多了, 在现在的环境下

除开税, 差不多了, 主要是这个方向的潜在价值

关键对你来说, 这个是一个成长机会

是的, 寒意还是有的

您是我的贵人

是! 有个外接大脑就是爽

好好卷, 先祝贺您, 拿到年薪75W+

再预祝您, 2年之后, 年薪200W+

谢谢

9年 小伙伴拿到 年薪90W offer

9月11日改简历 11月29日晒offer

秘诀:
简历指导+ 狠狠卷

薪资高 稳

这个是你微调的

省略的地方, 需要你再补充一点

9月11日 下午17:38

上面你留着这个就行

Java 开发 - 9 年-修改.docx 34.1 KB

主要的工作是啥?

提升了自己的实力, 就不用怕

易所 关于数字货币的

po 也有 打盹的时候, 该裁员, 照样一个不少

年包比 po 多 19w

这么多

po 估计有 70 万

那你不是有 90 个 W?

po 给我 68

就是吗

小伙8年经验 年薪60w

7月12日改简历 8月10日晒offer

秘诀:
改简历+ 狠狠卷

明天晚上哈

好哒

恩哥, 今晚还改简历吗?

7月12日 晚上19:54

今晚还在外边应酬, 估计回去比较晚

要不, 咱们延迟到明天, 如何

明天白天也行

好的, 白天吧, 答应别人明天给他们简历了。

7月12日 晚上20:27

OK

那就上午11点左右哈

好的

之前 36*15, 现在这个 39*15

今年行情不太好, 还有一些 offer 基本都是平薪, 没降薪的。

OK

这个马上来

刚在指导简历

哈哈

等等哈

辛苦恩哥

这次找工作, 您的指导真的起到效果了。

我这次复习基本看的都是咱们课程的 面试题。

6年小伙伴 年薪40w

9月6日改简历 9月21日晒offer

秘诀:
简历指导+ 狠狠卷

Java - 6年.docx 24.7 KB

恩哥, 简历我改好了, 您再帮我看一下

9月6日 下午14:47

Java - 6年(1).docx 24.5 KB

恩哥, 看第二个吧

9月6日 晚上19:41

给你前面调整了一下

9月6日 晚上19:45

今年这行情, 也算可以了

总包多少呀, 让我也了解一下

大概 40w 吧

谢谢恩哥的指导和鼓励

是在深圳

深圳的行情尤其难

能有面试电话就不错啦

是的

太牛啦

哈哈哈哈哈, 恩哥的鼓励指导也很重要

5年小伙喜提3个offer 年薪 35个W

5月22日改简历 11月29日晒offer

恩恩老师晚上好, 汇报下最近的 offer 情况。最近面试收到了三个 offer。两个是平薪, 一个是跨境电商公司的 offer, 涨幅暂时不到 20%。通过这次面试也让我知道自己距离高级开发还有一点距离, 还要再多卷才能突破。

最后结合自己的情况, 先选择去跨境电商的公司再提升下

之前是 20k*13.5, 跨境电商这个薪资 23k* (14-15)

恭喜恭喜

独立寒冬, 能拿到 3 个 offer, 已经厉害了

很多小伙伴, 面试电话一个都接不到, 简历海投 7000 份, 只收到 3 次面试机会, 没有一个机会拿到最终 offer

您这个年薪, 算下来也有 35W 了吧

最终部分还要确认下, 大部分人听说只有两个月

这个时间点, 拿到这个水平, 挺不错的啦

持续加油卷哈

恩恩! 看看明年自己有没有能力冲击离开

把你视频发给我看看

辛苦老师再帮忙指导下哈

秘诀:
简历指导+ 狠狠卷

1.5年小伙搞定15K offer 就业寒冬涨100%

5月7日改简历

11月21日晒offer



卷王逆袭成功案例

6年小伙从很少面试机会到
搞定35K*14薪

3月5日改简历

4月11日拿offer



6年 经验小伙伴 喜收25K offer

3月12日改简历

12月1日晒offer



7年经验卷王 薪酬涨30%

7月11日改简历

9月1日晒offer



4年经验卷王逆袭 被毕业后，反涨24W

7月改简历 **8月30日晒offer**

**秘诀：
改简历 + 狠狠卷**

这就是你的简历
差得太多啦
ok
总共写了四个项目，最近一年的还没补充上
你是在职，还是离职呢？
离职
原因大概是啥？
项目被终止
方便语音沟通不
ok

是的 感谢你指导，非常重要
老哥 我八月十号开始找工作，今天已经入职了
现金基本持平，股票+24W
总计涨了多少呢
能涨24W
股票这个吧只能到手了才算
也不错啦
很多小伙伴，面试机会都没有
感谢老哥的指导👍👍，继续跟你卷技术
继续很厉害哈，马上就技术自由啦

小伙5月份"被毕业"，改简历后 斩获顶级央企Offer 涨薪7000+

5月29日改简历 **7月5日晒offer**

**秘诀：
简历指导+ 狠卷3高**

快速看书，就要不求甚解，把目录和场景大概一下，然后重点的地方，用划的地方，再去回顾
尼恩 我被"毕业"了
这周末或下周找你改一下简历
毕业了没有关系
ok，发我吧
it行业，就来跑去，太频繁啦
嗯，其实有点心理准备
5月29日 上午10:49
简历指导
5月29日 上午10:52
不太会写简历

尼恩 我拿到半职的offer了
涨20%，2家要多，结果人家都不还价的
看起来半职不差钱呀
超过了8000没
平均算下来
7000多
好的
有啥面试的心得吗
可以分享给其他小伙伴的
1.面试前多看看简历，2.面试时不要紧张，3.面试后要感谢面试官

卷王逆袭成功案例 武汉6年喜收4个优质offer 最高的年薪35W

2月9日改简历 **4月15日晒offer**

**面试法宝：
改简历 + 实操**

尼恩老师，新年好！
能帮忙修改下简历吗？
金三银四准备挑了
可以的
java开发-6年-简历-
拜托了，尼恩。希望能拿25k回来给你报喜
好，我加一下
还有吗？

恩恩，决赛圈offer了
截图是我目前认知能可出来的评分了，麻烦帮我参考下
选择大于努力，尼恩助我上岸
这么多offer，我看看哈
都是尼恩指点有方👍👍，本来还有个新能源汽车的，35W给拒了，主要太远了
跟着尼恩老师的时间太短了，目前实力也只能到这儿了
这边有个大数据的，感觉也不错

卷王逆袭成功案例 6年小伙喜提4个Offer 最高涨9k，年薪35W

4月14日改简历 **5月17日晒offer**

**涨薪法宝：
改简历 + 狠狠卷**

Java开发工程师_...
dock
52.5 KB
微信语音
你看着我给你改的
好的呀
好的
谢谢大佬
麻烦大佬了
这个你自己别哈
不对的，你自己别
那我照着这个改一下库存系统呀
一个简历，...
这么漂亮的简历，涨50%，已经没啥问题
只要准备好，不出大批量，基本没问题啦

保密押金收起来哈，你的offer最高涨了9k，多返现100
后面继续跟尼恩卷哈，感觉卷的时间越长，...
尼恩，...
我准备这一年的时间都看呢
加油卷哈
感觉自己学的不太透彻了
嗯嗯
跟着大佬一起
我周围好几个年薪百万的，都是这

卷王逆袭成功案例

5年经验小伙收2个offer 最高涨薪8k，年薪42W

5月9日改简历

5月30日晒offer

秘诀:
简历指导+ 狠卷3高

以此为例
大家狠狠卷
打造最卷IT社群

卷王逆袭成功案例

非全日制 6年经验卷王 喜提3个Offer，年包30W

5月9日改简历

5月18日晒offer

面试法宝:
改简历+ 狠狠卷

卷王逆袭成功案例

寒五冻六之际卷王大逆袭 收3大offer，涨30%

5月17日改简历

5月27日晒offer

秘诀:
简历指导+ 狠卷3高

卷王逆袭成功案例

4年卷王入职微软，涨50%

3月7日改简历

5月12日晒offer

涨薪法宝:
改简历+ 狠狠卷

4年小伙喜收百度、Boss直聘等N个顶级Offer 最高涨幅100%

6月27日改简历 9月19日晒offer

**秘诀：
改简历 + 狠狠卷**

有个offer选择问题

boss直聘和小满之间，boss那offer更好，工资也高，但是小满那个offer，虽然工资低一点，但是能学到很多东西，而且小满那个offer，还能接触到很多大牛，我觉得这个offer，还是值得去试试的。

还有没有其他offer的，还有一个券商的，

总体上看boss和小满之间选一个

了

boss比度小满年包多7w以上，

我这涨幅都接近百分之百了

太牛啦

卷王逆袭成功案例

4年卷王入收2个offer，涨50%

3月23日改简历 5月12日晒offer

offer决策图

offer决策图：n+1 电商erp，部门成熟，人数多，加班少，n+2 制造业中台内部系统，新部门，人数少，加班多

又搞到一个offer

能搞定两个offer，不尴尬

现在很多小伙伴面试机会都没有

涨了多少呀

地点在哪里

涨50%的样子

忘记你工作几年啦，大概工作几年啦

**涨薪法宝：
改简历 + 狠狠卷**

这个不用加粗

都是一样，加粗了反而没有效果

小伙大三暑期很焦虑 跟着尼恩卷一年 校招斩获顶级央企Offer

去年5月19日加入VIP群 今年7月5日晒offer

**秘诀：
狠狠卷书+视频**

邀请你加入群聊

尼恩老师

我校招去华润电力控股有限公司了

跟着你卷了一年 大学顺便拿了几个国奖

不错不错，这是央企

放空

这太牛啦

跟着你卷了一年 大学顺便拿了几个国奖

其实拿到手的也就一个a类国一

尼恩大佬 大三的暑期实习找不到 现在准备秋招应该没关系吧

看身边 总是很焦虑 自己算法这一块卡住

网盘里边有算法视频

去刷一刷吧

秋招来得及

谢谢大佬 不过经过几个月练习 看写的书比之前轻松多了

趁着还是学生这段时间 慢慢把知识吃透

嗯哪，我的书，比较深

期待大佬的下一本书，已经迫不及待去学习了

小伙高中学历 薪酬涨120%

5月6日改简历 7月22日晒offer

你这块估计要送你668

慢快的开发工作，就这个三个哈

哈哈，不用了师傅，师傅的请了就行 光今天晚上辅导就值几千了

老弟很感谢您

还有很多其他的模块

面试官提问

就是其他人做的

嗯哪，好

**秘诀：
改简历 + 狠狠卷**

之前你的工资是多少来的

翻了一翻

我得送你多少奖金来的

不知道，原价给老弟就行了

668.00

咱们得说实话呀

老弟拿着您的那个高并发改造亮点，所向披靡

ok

从头到尾给面试官讲得明明白白

后面继续狠狠卷哈

卷王逆袭成功案例

非全日制卷王 面试3家 收2个offer 涨薪30%

4月13日改简历

4月21日晒offer

面试法宝:
改简历 + 面试题

5年卷王喜收2大Offer

最高涨5K

5月19日改简历

9月13日晒offer

秘诀:
改简历 + 狠狠卷

卷王逆袭成功案例

3年经验卷王, 涨60%

4月16日改简历

5月11日晒offer

涨薪法宝:
改简历 + 狠狠卷

卷王逆袭成功案例

双非二本小伙春招大翻身 喜提9大offer

2月22日改简历

4月13日晒offer

面试法宝:
改简历 + IM实操

1	公司	部门	岗位	薪资结构	总包
2	字节跳动	电商数字产品部	java后端开发	18.5k+14.5k+5k+2000/月+500/月+500/月	22.4w
3	美团	交易研发部	java后端开发	18k+14k+5k+2000/月+500/月	22.5w
4	京东	待定	java后端开发	15k+15k+5k+2000/月+500/月	14.2w
5	滴滴	待定	java后端开发	11k+13k+5k+2000/月+500/月	14.2w
6	美团	待定	java后端开发	9k+10k+5k+2000/月+500/月	13.8w
7	美团	待定	java后端开发	9k+10k+5k+2000/月+500/月	13.8w
8	美团	待定	java后端开发	9k+10k+5k+2000/月+500/月	13.8w
9	美团	待定	java后端开发	9k+10k+5k+2000/月+500/月	13.8w

9大offer 最高年薪30万

修改简历找尼恩（资深简历优化专家）

- 如果面试表达不好，尼恩会提供 简历优化指导
- 如果项目没有亮点，尼恩会提供 项目亮点指导
- 如果面试表达不好，尼恩会提供 面试表达指导

作为 40 岁老架构师，尼恩长期承担技术面试官的角色：

- 从业以来，“阅历”无数，对简历有着点石成金、改头换面、脱胎换骨的指导能力。
- 尼恩指导过刚刚就业的小白，也指导过 P8 级的老专家，都指导他们上岸。

如何联系尼恩。尼恩微信，请参考下面的地址：

语雀：<https://www.yuque.com/crazymakercircle/gkkw8s/khigna>

码云：<https://gitee.com/crazymaker/SimpleCrayIM/blob/master/疯狂创客圈总目录.md>